

Automatización del ciclo post-venta en e-commerce: pipeline de procesamiento de órdenes y atención al cliente con IA, implementado con n8n

Santiago Sordi, Ignacio Odorico, Juan Cruz Ana
Mendoza, Argentina — 2026

Universidad Tecnológica Nacional – Facultad Regional Mendoza

Carrera: Tecnicatura Universitaria en Programación

Materia: Trabajo Integrador

Director / Profesor: Alberto Cortez

DECLARACIÓN DE ORIGINALIDAD

Quienes suscriben declaran que el presente Trabajo Integrador es de su autoría, que fue elaborado íntegramente para la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, Facultad Regional Mendoza, y que no ha sido presentado con anterioridad para la obtención de ningún otro título o grado académico.

Todas las fuentes consultadas se encuentran debidamente citadas en el cuerpo del texto y consignadas en el Capítulo 8. Los datos experimentales que se reportan en el Capítulo 5 fueron obtenidos por los autores sobre el sistema descrito en el Capítulo 4; los guiones de ejecución, los archivos de resultados crudos y los procedimientos de cálculo se versionan en el repositorio del trabajo y se transcriben en los Anexos I y J, de modo que toda cifra publicada pueda ser recalculada de forma independiente. El repositorio es público y está disponible en <https://github.com/IgnacioOdorico/Automatizacion-de-Atencion-al-Cliente-en-E-commerce-e-Inteligencia-Artificial>; la versión que corresponde a este documento se identifica con la etiqueta entrega-2026-09-r6 (Anexo J).

Se deja constancia del uso de herramientas de asistencia basadas en modelos de lenguaje durante la redacción y la revisión del documento y durante el desarrollo del software. Se las empleó además para redactar los mensajes del corpus de evaluación del clasificador, a partir de mensajes escritos por el equipo, según se declara en la Sección 3.5.3. Fuera de ese uso, su empleo se limitó a tareas de asistencia: el diseño experimental, la ejecución de las mediciones, la interpretación de los resultados y las conclusiones son responsabilidad exclusiva de los autores, que asumen la autoría plena del trabajo.

Santiago Sordi — Ignacio Odorico — Juan Cruz Ana

Mendoza, 2026

RESUMEN

El trabajo aborda el procesamiento manual del ciclo post-venta en PyMEs argentinas de comercio electrónico. Se pregunta en qué medida un pipeline orquestado con n8n reduce el tiempo de procesamiento de órdenes respecto del proceso manual, y con qué tiempo de respuesta y exactitud atiende consultas un chatbot integrado.

Sobre un e-commerce simulado en contenedores Docker se implementaron un pipeline de órdenes y un chatbot sobre dos canales —uno simulado con entrega SMTP local y Telegram real— que clasifica intenciones con GPT-4o-mini. El diseño es un estudio de caso instrumental con métricas registradas automáticamente.

Sobre 50 órdenes, el MTTD medio fue de 0,009 s y el tiempo end-to-end de 0,063 s, frente a 49,13 s por orden cronometrados en el proceso manual: un factor cercano a $780\times$ (IC 95 % por el teorema de Fieller: $686\times$ a $875\times$; U de Mann-Whitney = 0, $p = 2,7 \times 10^{-11}$), leído como orden de magnitud de laboratorio, no como cota. Bajo 20 solicitudes casi simultáneas no hubo sobreventa, pero el 40,8 % de las órdenes quedó sin procesar ni aviso. El chatbot respondió en 1,47 s de media y clasificó con una exactitud del 92,7 % (IC 95 % de Wilson: 87,3 % a 95,9 %). Un diseño factorial con tres repeticiones mostró que esa exactitud depende de las reglas y los ejemplos del prompt (17,3 puntos) y no de la base de conocimiento (2,0). La corrección del contenido no quedó establecida: dos evaluadores no alcanzaron acuerdo suficiente, y una verificación automática halló 2 de las 18 respuestas con datos que la base no respalda, o 3 de 18 contando un caso fuera de su alcance. Las tres hipótesis se sostienen dentro de los alcances declarados, aunque solo la de exactitud se replicó con una regla fijada antes de medir.

Palabras clave: automatización de procesos, e-commerce, post-venta, n8n, chatbot, modelos de lenguaje, Docker, MTTD, MTTR, TMR.

ABSTRACT

This work addresses the manual handling of the post-sale cycle in Argentine e-commerce SMEs. It asks to what extent a pipeline orchestrated with n8n reduces order-processing time with respect to the manual process, and with what response time and accuracy an integrated chatbot handles customer queries.

On a simulated store deployed in Docker containers, an order pipeline of fifteen nodes and a chatbot over two channels—a simulated one with local SMTP delivery, and real Telegram—were implemented, the chatbot classifying intents with GPT-4o-mini. The design is an instrumental case study with automatically recorded metrics.

Over 50 orders, mean MTTD was 0.009 seconds and end-to-end time 0.063 s, against 49.13 s per order timed for the manual process: a factor close to 780× (Fieller 95 % CI: 686× to 875×; Mann-Whitney $U = 0$, $p = 2.7 \times 10^{-11}$), read as an order of magnitude under laboratory conditions and not as a bound. Under 20 near-simultaneous requests there was no overselling, but 40.8 % of the orders were left unprocessed and without warning. The chatbot replied in 1.47 s on average and classified intents with an accuracy of 92.7 % (Wilson 95 % CI: 87.3 % to 95.9 %). A factorial design with three repetitions showed that this accuracy depends on the rules and examples in the prompt, which contribute 17.3 points, and not on the store knowledge base, which contributes 2.0. The correctness of the replies' content was not established: two raters did not reach sufficient agreement, and an automatic check found that 2 of the 18 replies containing concrete data state at least one that is absent from the knowledge base, and a third case, beyond the reach of that rule, would raise the figure to 3 of 18. The three hypotheses hold within the declared scope, although only the accuracy hypothesis was replicated under a rule fixed before measuring.

Keywords: business process automation, e-commerce, post-sale service, n8n, chatbot, large language models, Docker, MTTD, MTTR, mean response time.

ÍNDICE

DECLARACIÓN DE ORIGINALIDAD	2
RESUMEN.....	3
ABSTRACT	4
Listado de figuras	9
Listado de tablas.....	11
Glosario de siglas y acrónimos.....	13
CAPÍTULO 1: INTRODUCCIÓN	15
1.1 Contexto del problema	15
1.2 Planteamiento del problema	15
1.3 Justificación.....	16
1.4 Pregunta de investigación e hipótesis	16
1.4.1 Pregunta de investigación	16
1.4.2 Hipótesis.....	16
1.5 Objetivos	17
1.5.1 Objetivo general.....	17
1.5.2 Objetivos específicos	18
1.6 Alcance y limitaciones	18
1.6.1 Alcance.....	18
1.6.2 Limitaciones	19
CAPÍTULO 2: MARCO TEÓRICO	20
2.1 Automatización de procesos de negocio.....	20
2.1.1 Definición y antecedentes	20
2.1.2 Herramientas de orquestación de flujos: n8n.....	20
2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce.....	21
2.2 Inteligencia artificial conversacional y modelos de lenguaje.....	22
2.2.1 Evolución de los chatbots: de reglas a LLMs	22
2.2.2 GPT-4o-mini y prompt engineering	22
2.2.3 Exactitud en clasificación de intents	23
2.2.4 Inyección de contexto, generación aumentada por recuperación y alucinación.....	23
2.2.5 Evaluación de texto generado, no determinismo y acuerdo entre evaluadores.....	24

2.3 Comunicación multicanal en e-commerce	25
2.3.1 Multicanalidad y omnicanalidad: delimitación conceptual	25
2.3.2 TMR y capacidad de respuesta.....	25
2.4 Estado del arte	26
2.4.1 Chatbots en la atención al cliente de comercio electrónico	27
2.4.2 Clasificación de intenciones con modelos de lenguaje	27
2.4.3 Anclaje en bases de conocimiento y alucinación.....	28
2.4.4 Automatización de procesos en pequeñas y medianas empresas	28
2.4.5 Antecedentes latinoamericanos y del caso argentino	29
2.4.6 Contribución de este trabajo.....	31
2.5 Tratamiento de datos personales y marco legal aplicable	33
CAPÍTULO 3: MARCO METODOLÓGICO.....	37
3.1 Tipo de investigación	37
3.2 Enfoque y diseño: estudio de caso instrumental.....	37
3.3 Metodología de desarrollo	39
3.4 Herramientas y tecnologías	39
3.4.1 Contenedores Docker y reproducibilidad.....	41
3.4.2 PostgreSQL como base de datos de métricas	41
3.4.3 Grafana para observabilidad operativa	41
3.5 Técnicas de recolección y análisis de datos	41
3.5.1 Recolección automatizada de métricas	41
3.5.2 Justificación del tamaño de muestra	42
3.5.3 Criterio de evaluación de la exactitud	43
3.5.4 Análisis estadístico	45
3.5.5 Medición del baseline de atención manual	45
3.5.6 Diseño factorial del prompt: base de conocimiento, reglas y ejemplos	47
3.5.7 Evaluación del contenido de las respuestas.....	49
3.5.8 Diseño de las pruebas funcionales, de carga y de concurrencia	50
3.6 Amenazas a la validez.....	52
3.6.1 Validez de constructo.....	52
3.6.2 Validez interna	52
3.6.3 Validez externa	53
3.6.4 Confiabilidad	54

CAPÍTULO 4: DESARROLLO DEL ESTUDIO	55
4.1 Arquitectura del sistema.....	55
4.2 Base de datos	56
4.2.1 Tablas principales	56
4.2.2 Diagrama entidad-relación	58
4.2.3 Vistas de métricas.....	58
4.2.4 Datos seed	59
4.3 Flujo 1 — Pipeline de procesamiento de órdenes.....	60
4.3.1 Nodos del workflow.....	60
4.3.2 Diagrama del flujo	62
4.3.3 Métricas MTTD y MTTR.....	63
4.3.4 Flujo de estados de una orden	64
4.4 Flujo 2 — Chatbot con IA.....	65
4.4.1 Nodos del workflow.....	65
4.4.2 Diagrama del flujo	67
4.4.3 Prompt de IA.....	68
4.4.4 Métrica TMR.....	69
4.5 Configuración de Grafana	69
4.6 Entorno y evidencia de las pruebas.....	71
CAPÍTULO 5: RESULTADOS Y ANÁLISIS.....	73
5.1 Resultados del Flujo 1 — Pipeline de procesamiento de órdenes	73
5.1.1 Pruebas funcionales	73
5.1.2 Métricas de tiempo	74
5.1.3 Pruebas de carga y de concurrencia	75
5.1.4 Baseline de atención manual	77
5.2 Resultados del Flujo 2 — Chatbot (canal simulado formato WhatsApp y canal Telegram real)	79
5.2.1 Pruebas funcionales del chatbot.....	79
5.2.2 Tiempos de respuesta (TMR)	80
5.2.3 Exactitud de clasificación de intents	82
5.2.4 Aporte de la base de conocimiento y de las reglas y ejemplos	84
5.2.5 Contenido de las respuestas de tipo FAQ	87
5.3 Contrastación de hipótesis	89

5.4 Análisis comparativo y discusión.....	91
5.5 Limitaciones observadas	93
CAPÍTULO 6: CONCLUSIONES.....	97
6.1 Respuesta a la pregunta de investigación.....	97
6.2 Cumplimiento de objetivos específicos	97
6.3 Conclusiones sustantivas.....	101
6.4 Limitaciones del estudio	103
CAPÍTULO 7: RECOMENDACIONES.....	105
7.1 Recomendaciones para producción	105
7.2 Recomendaciones para la investigación y líneas futuras	107
CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS	111
CAPÍTULO 9: ANEXOS.....	118
Anexo A: Resumen del esquema de la base de datos	118
Anexo B: Configuración de Docker Compose.....	125
Anexo C: Catálogo de productos de la carga inicial	125
Anexo D: Base de conocimiento completa (preguntas frecuentes)	127
Anexo E: Comandos de prueba	130
Anexo F: Capturas de la implementación de desarrollo	131
Anexo G: Propuesta de arquitectura de producción	132
Anexo H: Prompts de sistema del clasificador de intenciones.....	133
Anexo I: Baseline de atención manual — tiempos cronometrados.....	138
Anexo J: Corpus de evaluación y procedimiento de cálculo estadístico	140
Anexo K: Verificación de datos concretos en las respuestas de tipo FAQ	141
Anexo L: Registro de desvíos y correcciones	143

Listado de figuras

Figura	Descripción	Sección
Figura 1	Arquitectura del sistema. Cuatro servicios en contenedor sobre una red interna, con las dos dependencias externas de las que depende el Flujo 2: la API de OpenAI (GPT-4o-mini) y la API de Telegram.	4.1
Figura 2	Diagrama entidad-relación de la base de datos ecommerce_tesis, con las siete tablas, sus claves foráneas y la columna data_source que separa los registros medidos de los datos de carga inicial.	4.2.2
Figura 3	Diagrama del Flujo 1 — Pipeline de procesamiento de órdenes. Elaboración propia a partir del workflow implementado en n8n.	4.3.2
Figura 4	Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.	4.3.2
Figura 5	Diagrama del Flujo 2 — Chatbot con clasificación de intenciones. Elaboración propia a partir del workflow implementado en n8n.	4.4.2
Figura 6	Canvas del Flujo 2 en n8n, mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por intent y el nodo Router Canal que envía la respuesta por el canal de origen.	4.4.2
Figura 7	Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1, capturado el	4.5

Figura	Descripción	Sección
	19 de agosto de 2026.	
Figura 8	Bandeja de Mailpit con los correos generados por el Flujo 1 durante una corrida de verificación de cinco órdenes (ORD-FIG4-001 a 005), preparada para exhibir ambas ramas de notificación y distinta de la corrida de carga de cincuenta órdenes: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com)	4.6
Figura 9	Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la exactitud de clasificación y la distribución de intenciones sobre el corpus evaluado.	5.2.3
Figura 10	Canvas del Flujo 1 en su variante de producción, con los nodos de integración con plataformas de e-commerce reales.	7.1
Figura 11	Canvas del Flujo 2 en su variante de producción, con los triggers de Telegram, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025).	7.1

Listado de tablas

Tabla	Descripción	Sección
Tabla 2.1	Posición de este trabajo respecto de los antecedentes revisados	2.4.6
Tabla 3.1	Etapas de desarrollo y entregables	3.3
Tabla 3.2	Herramientas y tecnologías utilizadas	3.4
Tabla 3.3	Datos recolectados y su método de registro	3.5.1
Tabla 3.4	Pruebas funcionales del Flujo 1	3.5.8
Tabla 3.5	Pruebas funcionales del Flujo 2	3.5.8
Tabla 4.1	Servicios Docker del sistema	4.1
Tabla 4.2	Esquema de tablas de la base de datos	4.2.1
Tabla 4.3	Vistas de métricas en PostgreSQL	4.2.3
Tabla 4.4	Catálogo de productos de prueba	4.2.4
Tabla 4.5	Nodos del workflow del Flujo 1	4.3.1
Tabla 4.6	Estados de una orden y nodo que los asigna	4.3.4
Tabla 4.7	Nodos del workflow del Flujo 2	4.4.1
Tabla 4.8	Paneles de los dashboards de Grafana	4.5
Tabla 5.1	Resultados de las pruebas funcionales	5.1.1
Tabla 5.2	Métricas de tiempo del Flujo 1	5.1.2
Tabla 5.3	Resultados de las pruebas de carga y de concurrencia	5.1.3
Tabla 5.4	Descriptivos del tiempo de procesamiento manual, por fase y total	5.1.4
Tabla 5.5	Tiempo medio de respuesta por intención sobre el corpus evaluado	5.2.2
Tabla 5.6	Tiempo medio de respuesta por intención en el canal Telegram	5.2.2
Tabla 5.7	Exhaustividad (recall) de clasificación por intención	5.2.3
Tabla 5.8	Matriz de confusión del clasificador	5.2.3
Tabla 5.9	Precisión, exhaustividad y F1 por clase	5.2.3

Tabla	Descripción	Sección
Tabla 5.10	Exhaustividad por clase y exactitud total por mayoría de tres repeticiones, y exactitud media por repetición, por condición del diseño factorial	5.2.4
Tabla 5.11	Contrastación de hipótesis	5.3
Tabla 6.1	Cumplimiento de objetivos específicos	6.2
Tabla I.1	Tiempos cronometrados del procesamiento manual, por orden y por fase	Anexo I
Tabla I.2	Registros en la base de datos de las órdenes procesadas manualmente	Anexo I
Tabla K.1	Datos concretos afirmados en las respuestas de tipo FAQ y su presencia en la base de conocimiento	Anexo K

Glosario de siglas y acrónimos

API	Application Programming Interface (Interfaz de Programación de Aplicaciones)
BPA	Business Process Automation (Automatización de Procesos de Negocio)
BPM	Business Process Management (Gestión de Procesos de Negocio)
CACE	Cámara Argentina de Comercio Electrónico
CRM	Customer Relationship Management (Gestión de Relaciones con Clientes)
DDL	Data Definition Language (Lenguaje de Definición de Datos)
ERD	Entity-Relationship Diagram (Diagrama Entidad-Relación)
ERP	Enterprise Resource Planning (Planificación de Recursos Empresariales)
FAQ	Frequently Asked Questions (Preguntas Frecuentes)
FK	Foreign Key (Clave Foránea)
GPT	Generative Pre-trained Transformer
HTTP	HyperText Transfer Protocol
IA	Inteligencia Artificial
ITIL	Information Technology Infrastructure Library
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (Modelo de Lenguaje de Gran Escala)
MTTD	Mean Time To Detect (Tiempo Medio de Detección)

MTTR	Mean Time To Resolve (Tiempo Medio de Resolución)
PyME	Pequeña y Mediana Empresa
REST	Representational State Transfer
SGBD	Sistema de Gestión de Base de Datos
SKU	Stock Keeping Unit (Unidad de Mantenimiento de Inventario)
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TMR	Tiempo Medio de Respuesta
URL	Uniform Resource Locator
YAML	YAML Ain't Markup Language
UTN	Universidad Tecnológica Nacional

CAPÍTULO 1: INTRODUCCIÓN

1.1 Contexto del problema

El comercio electrónico en Argentina creció significativamente en los últimos años. Cada vez más PyMEs venden a través de plataformas como Tiendanube, WooCommerce o MercadoShops. Sin embargo, muchas de estas empresas siguen gestionando el ciclo post-venta de forma manual: un operador revisa los pedidos, verifica el stock, envía correos de confirmación uno por uno y responde consultas por WhatsApp, email y redes sociales sin ninguna herramienta que centralice esas interacciones. En efecto, en 2024 la facturación nominal del sector, medida en pesos corrientes, creció un 181 % interanual; dado que ese período transcurrió bajo inflación de tres dígitos, la cifra no es interpretable por sí sola como expansión real del volumen comercializado y se consigna únicamente como indicador de la magnitud nominal del mercado. El dato que sí describe la penetración del canal es de naturaleza no monetaria: para el 54 % de las empresas relevadas el canal online ya representa más del 10 % de sus ventas, frente al 50 % en 2023 (Cámara Argentina de Comercio Electrónico [CACE], 2025).

El ciclo post-venta abarca todo lo que ocurre después de que el cliente confirma su compra: confirmación de la orden, verificación de stock, notificaciones de estado y atención de consultas o reclamos. En una PyME con equipo reducido, estas tareas consumen tiempo y son propensas a errores cuando se hacen manualmente.

1.2 Planteamiento del problema

Se identifican tres problemas concretos en la operación post-venta de PyMEs de e-commerce:

1. **Procesamiento manual de órdenes.** Cada pedido requiere que un operador lo revise, consulte stock, actualice estados y redacte un correo de confirmación. Esto introduce demoras y errores como confirmar pedidos sin stock disponible. Estas demoras son críticas en un contexto donde el 82 % de los clientes espera una resolución inmediata de sus consultas (HubSpot, 2024). Esa cifra proviene de un reporte de industria elaborado sobre una encuesta comercial y no de una publicación con revisión por pares, de modo que se la consigna como indicio del contexto de expectativas y no como evidencia arbitrada.
2. **Atención al cliente fragmentada.** Un mismo cliente puede escribir por WhatsApp, mandar un email y dejar un mensaje en Telegram. Sin integración entre canales, el operador pierde contexto, responde tarde o da información inconsistente. Este trabajo aborda el problema solo en parte: atiende dos canales con un mismo sistema, pero no conserva el contexto de un cliente entre canales (Sección 2.3.1).

3. **Ausencia de métricas.** Sin indicadores como el tiempo que tarda el sistema en detectar una orden o en responder un mensaje, no hay forma de medir el rendimiento ni identificar cuellos de botella.

1.3 Justificación

La automatización del ciclo post-venta se justifica desde tres perspectivas:

Operativa. Automatizar tareas repetitivas como la verificación de stock y el envío de notificaciones libera al equipo para tareas que requieren criterio humano. Para una PyME que procesa decenas de órdenes por día, la diferencia en horas de trabajo es significativa.

Experiencia del cliente. Un pipeline automatizado permite confirmaciones inmediatas, notificaciones proactivas y respuestas sin dependencia del horario de un operador. El cliente recibe atención consistente sin depender de la disponibilidad de un operador. Diversos estudios señalan que los asistentes conversacionales basados en IA mejoran la calidad de servicio percibida en la atención al cliente (Misischia et al., 2022).

Tecnológica. Herramientas como n8n, PostgreSQL, Docker y los modelos de lenguaje de OpenAI permiten implementar soluciones de automatización sofisticadas a bajo costo, utilizando software de código abierto y APIs accesibles.

1.4 Pregunta de investigación e hipótesis

1.4.1 Pregunta de investigación

¿En qué medida un pipeline de automatización orquestado con n8n reduce el tiempo de procesamiento de órdenes del ciclo post-venta de un e-commerce simulado respecto del procesamiento manual, medido a través de las métricas MTDD (tiempo de detección y procesamiento de órdenes) y MTTR (tiempo de resolución y notificación), y con qué tiempo de respuesta (TMR) y exactitud de clasificación atiende las consultas de los clientes un chatbot integrado al mismo sistema?

La pregunta distingue dos términos porque solo el primero admite una comparación medida. Para el procesamiento de órdenes se cronometró un baseline manual (Sección 3.5.5), y la reducción se estima contra él. Para la atención conversacional no se midió un proceso manual equivalente, de modo que su desempeño se contrasta contra umbrales absolutos fijados por el equipo (Sección 1.4.2) y no se expresa como reducción.

1.4.2 Hipótesis

H1: El pipeline automatizado de procesamiento de órdenes reduce el tiempo que insume procesar una orden respecto del mismo procesamiento hecho de forma manual, medido como baseline propio según el protocolo de la Sección 3.5.5. Como

era previsible que un proceso automatizado sobre una base local fuera más rápido que uno manual, H1 se trata como un objetivo de estimación antes que como una hipótesis que pudiera fallar: su resultado es la magnitud de la reducción, expresada como factor con su intervalo de confianza al 95 %. Como referencia descriptiva, fijada con el resultado conocido (Tabla 5.11 y Anexo L), se consigna si el límite inferior de ese intervalo alcanza un orden de magnitud, es decir $10\times$. Como criterio operativo secundario se fija que el tiempo end-to-end del pipeline se mantenga por debajo de 30 segundos.

H2a: El chatbot basado en GPT-4o-mini responde al cliente con un tiempo medio de respuesta (TMR) inferior a 10 segundos en cada una de las cuatro categorías de intención. La extensión a las cuatro categorías es posterior a la formulación inicial de la hipótesis (Tabla 5.11 y Anexo L).

H2b: El chatbot clasifica correctamente la intención del mensaje con una exactitud de al menos el 85 %. La hipótesis se sostiene si el límite inferior del intervalo de confianza de Wilson al 95 % de esa exactitud es igual o mayor que el 85 %. A lo largo del trabajo se reserva «exactitud (accuracy)» para esta métrica global y «precisión» para la métrica por clase de la Tabla 5.9, conforme la terminología de Jurafsky y Martin (2024). El umbral se fija como criterio del equipo —no deriva de un estándar publicado ni de un requerimiento de negocio externo— y se justifica en la Sección 2.2.3. La regla del límite inferior se explicitó con la corrida del corpus ya ejecutada, y el diseño factorial de la Sección 5.2.4 la aplicó fijada por escrito antes de medir (Tabla 5.11 y Anexo L).

Las tres hipótesis se contrastan en el Capítulo 5 a partir de los datos recolectados durante la corrida de carga secuencial, la medición del baseline manual y las corridas del corpus del chatbot. Los umbrales de 30 y de 10 segundos operan como criterios de aceptación fijados por el equipo y no constituyen, por sí mismos, afirmaciones falsables. Dado el diseño del sistema —persistencia sobre una base de datos local y notificación a un capturador SMTP alojado en el mismo host— era previsible que ambos se cumplieran con amplio margen. Lo sustantivo de H1 es, por eso, la magnitud de la reducción respecto del baseline de atención manual medido en este trabajo, que la Sección 5.3 estima con su intervalo.

1.5 Objetivos

1.5.1 Objetivo general

Diseñar e implementar un pipeline automatizado de procesamiento de órdenes y un chatbot con inteligencia artificial para el ciclo post-venta de e-commerce, orquestados con n8n y desplegados en Docker, midiendo su desempeño mediante las métricas MTDD, MTTR y TMR, contrastando el tiempo de procesamiento de órdenes con el del proceso manual y el desempeño del chatbot con umbrales absolutos.

1.5.2 Objetivos específicos

1. Implementar un pipeline de procesamiento de órdenes (15 nodos en n8n) que reciba pedidos vía webhook, verifique stock en PostgreSQL, actualice estados y notifique al cliente por email, registrando MTDD y MTTR; verificar que el tiempo end-to-end sea inferior a 30 segundos; y estimar cuánto reduce el tiempo de procesamiento de una orden respecto del procesamiento manual (H1).
2. Implementar un chatbot multicanal sobre dos canales —WhatsApp, con envío simulado a un capturador SMTP local, y Telegram, con envío real— que normalice el mensaje entrante, clasifique la intención con GPT-4o-mini (FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL) y responda de forma automatizada registrando el TMR; y verificar que ese TMR sea inferior a 10 segundos y que la exactitud de clasificación alcance el 85 % (H2a y H2b). La extensión a un tercer canal por correo electrónico queda planteada desde el inicio como línea futura, y su diseño se documenta en el Capítulo 7.
3. Diseñar un esquema de base de datos en PostgreSQL con las tablas, vistas de métricas e índices necesarios para instrumentar el ciclo post-venta y sostener el cálculo automático de MTDD, MTTR y TMR.
4. Configurar dashboards en Grafana conectados a PostgreSQL para visualizar MTDD, MTTR y TMR a partir de las vistas de métricas.
5. Validar el funcionamiento del sistema mediante pruebas funcionales con datos simulados, verificando que los flujos operan según lo diseñado y las métricas se registran correctamente.

Dos componentes del trabajo no figuraban entre los objetivos específicos formulados al inicio y se incorporaron durante la investigación: el diseño factorial sobre el prompt del clasificador (Sección 3.5.6) y la evaluación del contenido de las respuestas (Sección 3.5.7). Se los reporta como ampliaciones del alcance y no como objetivos cumplidos, y la Tabla 6.1 no los computa. La comparación con el proceso manual, en cambio, se incorporó a OE1 y se la evalúa como parte de ese objetivo (Anexo L).

1.6 Alcance y limitaciones

1.6.1 Alcance

El proyecto implementa los dos flujos de automatización en un entorno local basado en Docker Compose, con cuatro servicios: n8n, PostgreSQL 15, Mailpit y Grafana. Los datos son simulados: órdenes enviadas por HTTP al webhook y conversaciones iniciadas desde cuentas de prueba.

El alcance funcional incluye:

- Pipeline de órdenes: recepción → verificación de stock → confirmación o rechazo → notificación por email → registro de métricas.
- Chatbot: recepción → normalización → clasificación de intent con IA → respuesta automatizada → registro de métricas.
- Dashboards de métricas operativas en Grafana.

1.6.2 Limitaciones

- **Costo de API de OpenAI:** el modelo GPT-4o-mini tiene un costo por token. En desarrollo se trabaja con volúmenes reducidos de mensajes.
- **WhatsApp Business API:** requiere una cuenta verificada por Meta, verificación que no se completó dentro del alcance de este trabajo. En consecuencia el canal de WhatsApp no se ejecutó contra la API real ni contra su entorno sandbox: las pruebas del Flujo 2 sobre ese canal se realizaron entregando la respuesta a un capturador SMTP local, de modo que las latencias reportadas para él no incluyen componente alguno de la red de WhatsApp. La validación sobre un canal de mensajería real se resolvió mediante Telegram (Sección 5.2.2).
- **Entorno local:** el sistema no se despliega en nube. Alta disponibilidad y escalabilidad horizontal quedan fuera del alcance.
- **Datos simulados:** las pruebas usan datos ficticios que representan escenarios típicos pero no la variabilidad completa de un entorno real.

CAPÍTULO 2: MARCO TEÓRICO

2.1 Automatización de procesos de negocio

2.1.1 Definición y antecedentes

La Automatización de Procesos de Negocio (BPA, *Business Process Automation*) se define como el uso de tecnología para ejecutar tareas y flujos de trabajo recurrentes con el fin de aumentar la eficiencia, reducir el error humano y estandarizar los resultados (Dumas et al., 2018). Automatizar un proceso no equivale a gestionarlo: la gestión de procesos de negocio (Business Process Management, BPM) comprende un ciclo de vida que incluye, además de implementar el proceso, monitorear y controlar su ejecución (Dumas et al., 2018). El Capítulo 6 usa esa distinción para situar lo que el sistema construido alcanza.

En el contexto del comercio electrónico, la automatización de procesos post-venta forma parte del cumplimiento de pedidos (order fulfillment), que la literatura de comercio electrónico no limita a entregar lo pedido en tiempo sino que extiende a los servicios al cliente asociados (Turban et al., 2018); automatizarlo es ejecutar sin intervención humana las etapas de ese ciclo que no requieren criterio. En PyMEs de e-commerce con equipo operativo reducido, las tareas manuales del ciclo post-venta —verificación de stock, actualización de estados, envío de notificaciones y atención de consultas— representan una fracción significativa de la carga de trabajo diaria, lo que constituye una oportunidad concreta de mejora mediante automatización (Laudon y Traver, 2021).

2.1.2 Herramientas de orquestación de flujos: n8n

Para la implementación de este proyecto se seleccionó **n8n** como motor de orquestación. n8n es una plataforma de automatización de flujos de trabajo de código abierto y auto-alojable (*self-hosted*) cuya arquitectura se basa en *workflows* compuestos por nodos interconectados, donde cada nodo ejecuta una operación atómica: recibir un webhook, consultar una base de datos, llamar una API externa o enviar un email (n8n GmbH, 2025).

La elección de n8n sobre alternativas comerciales como Zapier o Make.com se fundamenta en tres criterios: (1) **modelo de costos**: n8n es gratuito en modalidad self-hosted, eliminando los costos de licencia que representan una barrera para PyMEs con presupuesto acotado; (2) soberanía de datos: al ejecutarse en infraestructura propia, la persistencia de los datos de clientes y la totalidad del procesamiento del Flujo 1 permanecen bajo control del operador, sin transmitirse a servidores de terceros. El criterio no se extiende a la inferencia del Flujo 2: ese flujo remite el texto del mensaje del cliente a la API de OpenAI en cada interacción, dependencia que se discute como limitación en la Sección 5.5 y que motiva la

recomendación de evaluar modelos de lenguaje de ejecución local del Capítulo 7; (3) **extensibilidad**: n8n permite ejecutar código JavaScript/Python arbitrario en nodos *Function*, lo que posibilita lógica de transformación compleja sin salir del entorno visual (n8n GmbH, 2025). Los tres criterios describen propiedades de n8n documentadas por su fabricante; el trabajo no evaluó las alternativas con el mismo detalle.

La elección se inscribe en una categoría de herramientas que la literatura de sistemas de información estudia como plataformas de desarrollo low-code. Sahay et al. (2020) las describen como entornos visuales pensados para que personas sin formación en programación construyan sus propios sistemas, y proponen un marco conceptual para compararlas, porque elegir entre cientos de plataformas heterogéneas resulta difícil sin un soporte dedicado. Bock y Frank (2021) examinan de manera exploratoria un conjunto de plataformas disponibles en el mercado para caracterizar sus rasgos distintivos, y Rokis y Kirikova (2022) revisan la literatura sobre desarrollo low-code y no-code, identifican los desafíos propios de cada fase del desarrollo y resumen las soluciones propuestas. Esos trabajos sirven de contrapeso a la documentación del fabricante, que es la fuente de los criterios anteriores: la reducción de código que promete la categoría no elimina el código efectivamente escrito —el Flujo 2 requiere cinco nodos con JavaScript, y los dos flujos, sentencias SQL escritas a mano— y la barrera de adopción que se le atribuye no fue medida en este trabajo (Sección 6.3).

2.1.3 Métricas MTTD y MTTR en operaciones de e-commerce

Las métricas **MTTD** (*Mean Time To Detect*) y **MTTR** (*Mean Time To Resolve*) tienen origen en la disciplina de gestión de incidentes de TI (Axelos, 2019) y en el movimiento DevOps (Kim et al., 2016). En el contexto de este trabajo se las adapta al ciclo de procesamiento de órdenes:

- **MTTD** mide el tiempo transcurrido desde la recepción de una orden hasta que el sistema detecta y procesa el stock disponible. Equivale al tiempo de procesamiento interno del pipeline.
- **MTTR** mide el tiempo desde que el stock es procesado hasta que el sistema despacha la notificación de confirmación o rechazo al servidor de correo; no incluye el tránsito hasta la casilla del cliente (Sección 4.3.3). Equivale al tiempo de resolución y comunicación.

La adaptación de estas métricas a procesos de e-commerce es coherente con la práctica de operational excellence documentada por Womack y Jones (2003) en el contexto de manufactura esbelta, donde la reducción del tiempo de ciclo es un indicador central de eficiencia operativa.

Resulta pertinente discutir la transferibilidad conceptual de MTTD y MTTR desde su dominio de origen — la gestión de incidentes TI — al dominio del procesamiento de órdenes de e-commerce. En ITIL, ambas métricas miden eventos asociados a fallas

(un incidente es por definición un evento no deseado). En el contexto de este trabajo, una orden de compra exitosa no constituye una falla sino un flujo nominal del proceso de negocio. La transferencia se sostiene, sin embargo, a nivel estructural: ambas métricas miden intervalos temporales acotados por dos eventos observables y representan etapas secuenciales de un ciclo (detección/procesamiento y resolución/comunicación). Esta analogía estructural — y no semántica estricta — es coherente con la práctica documentada en la literatura de operational excellence, donde indicadores originalmente formulados para entornos de manufactura (lead time, cycle time) se transfieren a procesos administrativos y de servicio sin pérdida de validez operativa (Womack y Jones, 2003). Se reconoce que métricas alternativas más específicas del dominio comercial — como order cycle time u order fulfillment lead time — podrían reemplazar a MTTD y MTTR en versiones futuras del trabajo. La elección de MTTD y MTTR en este estudio se fundamenta en su disponibilidad inmediata como vocabulario de observabilidad ya soportado por las herramientas de monitoreo del stack tecnológico (Grafana), lo que facilita la integración con prácticas de site reliability engineering (Beyer et al., 2016) habituales en equipos de desarrollo.

2.2 Inteligencia artificial conversacional y modelos de lenguaje

2.2.1 Evolución de los chatbots: de reglas a LLMs

Los sistemas de diálogo automatizado evolucionaron desde los chatbots basados en reglas (*rule-based*) de los años 1990 —donde cada posible interacción debe ser explícitamente programada— hacia los sistemas basados en recuperación de información (*retrieval-based*) y, finalmente, hacia los modelos de lenguaje generativos (*generative*) (Jurafsky y Martin, 2024). Los chatbots basados en reglas presentan limitaciones conocidas: frágiles ante variaciones ortográficas, incapaces de manejar intenciones no anticipadas, y costosos de mantener a medida que el catálogo de respuestas crece (McTear, 2020).

Los modelos de lenguaje de gran escala (*Large Language Models*, LLMs) resuelven estas limitaciones al aprender representaciones estadísticas del lenguaje natural a partir de corpus masivos (Brown et al., 2020). Su capacidad de generalización permite comprender consultas variadas y generar respuestas coherentes sin requerir la codificación explícita de cada caso.

2.2.2 GPT-4o-mini y prompt engineering

En este proyecto se utiliza GPT-4o-mini de OpenAI (OpenAI, 2024), un modelo optimizado para inferencia rápida con buen balance entre capacidad y costo. La interacción con el modelo se configura mediante prompt engineering estructurado: el prompt de sistema define el rol del modelo (agente de atención al cliente de un e-commerce), las categorías de intención válidas (FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL), el formato de respuesta esperado (un objeto JSON estricto, sin texto ni marcado alrededor) y las reglas de idioma y de tono.

El régimen de prompting empleado tiene consecuencias sobre la interpretación de los resultados. Brown et al. (2020) diferencian tres regímenes según la cantidad de ejemplos etiquetados que se incluyen en el prompt: zero-shot, donde el modelo recibe únicamente la instrucción y la definición de las clases; one-shot, con un ejemplo; y few-shot, con varios ejemplos de entrada y salida. Liu et al. (2023) sistematizan el conjunto de estas técnicas. La configuración principal de este trabajo opera en régimen few-shot con inyección de contexto: su prompt de sistema incluye reglas de decisión por categoría, siete ejemplos etiquetados de entrada y salida y un bloque en el que se inserta la base de conocimiento completa de la tienda, leída de la base de datos en cada llamada (Sección 2.2.4). El Capítulo 5 la compara con tres variantes que quitan la base de conocimiento, las reglas y los ejemplos, o ambas cosas; las dos que no tienen ejemplos operan en régimen zero-shot. Los bloques del prompt se transcriben en el Anexo H y la comparación se reporta en la Sección 5.2.4.

2.2.3 Exactitud en clasificación de intents

La evaluación de la clasificación de intenciones en chatbots se realiza típicamente mediante métricas de clasificación estándar: accuracy, precision, recall y F1-score (Jurafsky y Martin, 2024). En este trabajo se emplea accuracy como métrica principal por su interpretabilidad directa, y se la acompaña —siguiendo la práctica habitual cuando las clases no son estrictamente equiprobables— del desglose por clase de precisión, exhaustividad y F1, además de la matriz de confusión completa (véase §5.2.3). El conjunto de prueba no está perfectamente balanceado: la razón entre la clase más numerosa y la menos numerosa es de 1,92 a 1 (48 y 25 mensajes sobre un total de 150), un desbalance moderado que se reporta explícitamente para que el lector pueda ponderar el accuracy global a la luz del desempeño por clase. Se adopta un umbral mínimo del 85 % de accuracy como criterio de aceptación definido por el equipo. Ese valor no deriva de un estándar publicado, y el razonamiento que lo sustenta es el siguiente. La literatura revisada en la Sección 2.4 muestra que la clasificación de intenciones sin ajuste fino alcanza desempeño útil: Parikh et al. (2023) documentan que el prompting zero-shot basado en descripciones de intención resulta efectivo, y Luo et al. (2023) obtienen desempeño competitivo con cantidades mínimas de datos. Esos trabajos evalúan sobre corpus normalizados con decenas de intenciones, mientras que el dominio de este trabajo se limita a cuatro categorías, lo que vuelve la tarea comparativamente más sencilla. Por esa razón se fija un umbral exigente para el nivel del prototipo, sin pretender que sea equiparable a los valores reportados sobre aquellos corpus.

2.2.4 Inyección de contexto, generación aumentada por recuperación y alucinación

Un modelo de lenguaje responde a partir de lo que aprendió en su entrenamiento. Cuando la respuesta exige datos que no están en ese conocimiento —la política de una tienda, por ejemplo—, tiende a completarla con contenido plausible pero no respaldado, fenómeno que la literatura denomina alucinación (Huang et al., 2025). La

respuesta técnica más difundida es la generación aumentada por recuperación (retrieval-augmented generation, RAG), que Lewis et al. (2020) formulan como la combinación de la memoria paramétrica del modelo con una memoria no paramétrica: un recuperador selecciona, para cada consulta, los pasajes pertinentes de un índice documental, y el generador condiciona su salida en ellos. Fan et al. (2024) sistematizan las variantes de esa arquitectura en los modelos de lenguaje de gran escala.

Lo que este trabajo implementa no es RAG sino inyección de contexto: en cada llamada se inserta en el prompt la tabla completa de veintitrés entradas de la base de conocimiento, sin seleccionar las pertinentes a la consulta (Sección 4.4.3). La distinción tiene dos consecuencias. La primera es de escala: con una base de conocimiento de tamaño real la inyección completa deja de ser viable y exige un recuperador, que introduce su propia fuente de error. La segunda es de posición: Liu et al. (2024) muestran que el desempeño de los modelos cae cuando la información relevante queda en el medio de un contexto largo, lo que advierte que el lugar que ocupa la base dentro del prompt no es neutral. Ninguna de las dos arquitecturas elimina la alucinación: la reducen en la medida en que el modelo se atiene al contexto que recibe, y esa es la propiedad que examina la Sección 5.2.5.

2.2.5 Evaluación de texto generado, no determinismo y acuerdo entre evaluadores

Evaluar lo que un modelo generativo responde es más difícil que evaluar lo que clasifica. La clasificación admite una etiqueta de referencia y una métrica de exactitud; el texto generado admite muchas respuestas aceptables para una misma consulta, y la literatura de generación de lenguaje natural distingue entre la evaluación mediante métricas automáticas y la basada en juicio humano, y discute los límites de ambas (Gatt y Krahmer, 2018). A esa dificultad se suma el no determinismo: en generación de código, un mismo prompt produce salidas distintas en llamadas sucesivas, y llevar la temperatura a cero lo reduce pero no lo elimina (Ouyang et al., 2025). Sclar et al. (2024) documentan además que, en modelos abiertos y con ejemplos en el prompt, cambios de formato que no alteran el contenido llegan a mover la exactitud hasta 76 puntos, y recomiendan informar el desempeño sobre varias formulaciones y no sobre una sola. Ambos hallazgos fundan dos decisiones del método: repetir cada condición del diseño factorial (Sección 3.5.6) y tratar como hipótesis, y no como resultado, el efecto de la forma de presentar el mensaje del cliente (Sección 5.2.4).

Cuando la evaluación descansa en jueces humanos, su confiabilidad se establece por el acuerdo entre ellos. El κ de Cohen (1960) descuenta el acuerdo atribuible al azar, y la escala de Landis y Koch (1977) ofrece una lectura convencional de sus valores. Artstein y Poesio (2008) revisan los coeficientes de acuerdo entre anotadores —entre ellos el κ de Cohen, el π de Scott y el α de Krippendorff— y discuten cuál resulta apropiado según la tarea de anotación. Un acuerdo bajo informa que el

esquema de categorías no se aplica de manera reproducible, y por eso un instrumento de juicio se pone a prueba antes de usarse. La evaluación de contenido de este trabajo no lo hizo: su rúbrica se aplicó sin prueba piloto y con niveles que se superponen (Sección 5.2.5).

2.3 Comunicación multicanal en e-commerce

2.3.1 Multicanalidad y omnicanalidad: delimitación conceptual

Dos estrategias que suelen emplearse como sinónimos no lo son. La estrategia multicanal consiste en poner a disposición del cliente varios canales de comunicación —WhatsApp, Telegram, correo electrónico, redes sociales— atendidos por un mismo sistema de procesamiento, pero donde cada conversación se resuelve de forma independiente y no se conserva contexto entre canales. La estrategia omnicanal (omnichannel) va un paso más allá: integra esos canales en una experiencia unificada, de manera que el historial y el contexto del cliente se preserven con independencia del canal utilizado (Verhoef et al., 2015), en oposición al enfoque en el que cada canal opera en silos. La diferencia operativa entre ambas es concreta y verificable: la omnicanalidad exige identidad unificada de cliente entre canales y recuperación del historial conversacional previo.

El sistema desarrollado es multicanal: recibe y responde por WhatsApp y por Telegram a través de un mismo flujo de procesamiento, con un único clasificador y una única base de datos. Sin embargo, la tabla interactions no registra identificador de conversación ni identidad unificada de cliente entre canales, y ningún nodo del Flujo 2 recupera interacciones previas al construir la respuesta: cada mensaje se atiende sin memoria del anterior. La omnicanalidad en sentido estricto —identidad unificada y memoria conversacional entre canales— no fue implementada y queda como línea futura en el Capítulo 7. En el segmento de PyMEs de e-commerce argentinas, donde la atención suele distribuirse entre WhatsApp y correo electrónico, esa evolución constituye el paso natural del prototipo.

2.3.2 TMR y capacidad de respuesta

El TMR (Tiempo Medio de Respuesta) mide la capacidad de respuesta de un sistema de atención, no su efectividad: la rapidez es una dimensión de la calidad de servicio y no garantiza que la respuesta sea correcta, como muestra la Sección 5.2.5. En la literatura de calidad de servicio, la capacidad de respuesta (responsiveness) es una de las cinco dimensiones del modelo SERVQUAL (Zeithaml et al., 1990); si bien ese modelo es anterior a la existencia de los canales digitales, la dimensión se traslada a este contexto como el tiempo que el cliente espera hasta recibir una respuesta. Reportes de la industria señalan que los clientes esperan respuestas cada vez más rápidas —el 82% espera una resolución inmediata de sus consultas (HubSpot, 2024)—, muy por debajo de los tiempos que caracterizan a la atención manual. Un sistema automatizado con un TMR de segundos se ubica, por lo tanto, en el extremo

favorable de esa dimensión. Este trabajo no midió el tiempo de respuesta de una atención manual por chat, de modo que no cuantifica la mejora respecto de ella.

2.4 Estado del arte

Esta sección revisa lo publicado sobre el problema abordado y empieza por el procedimiento seguido, para que tanto el alcance de la revisión como sus omisiones resulten atribuibles.

Procedimiento de búsqueda. Se consultaron ACL Anthology, arXiv, Crossref y los catálogos en línea de ScienceDirect y Springer Nature para la literatura internacional, y SciELO, Redalyc, Dialnet y los repositorios institucionales de universidades nacionales argentinas —entre ellos el repositorio del CONICET— para la literatura regional arbitrada. Las cadenas combinaron los términos chatbot, customer service, e-commerce, intent classification, large language model, zero-shot, business process automation, robotic process automation, SME y order fulfillment —y, en una actualización de septiembre de 2026, retrieval-augmented generation, hallucination, grounding, customer service, low-code platform y n8n—, en inglés, con sus equivalentes en español para las búsquedas de alcance regional. La ventana temporal se fijó entre 2018 y 2026, con dos excepciones declaradas: las obras canónicas ya incorporadas al marco teórico, y el estudio de Alderete et al. (2017) sobre adopción de comercio electrónico en pymes de Córdoba, que se incluye por ser el trabajo empírico de referencia sobre el caso nacional específico que este trabajo sitúa como contexto. La identificación bibliográfica de cada antecedente —autores, revista, volumen, páginas y DOI— fue verificada contra el registro de Crossref o contra el texto publicado. Se incluyeron trabajos empíricos arbitrados que reportaran métricas cuantitativas y revisiones sistemáticas, y se excluyeron el material de divulgación y la documentación de producto. Se admitió una excepción declarada: cuatro preprints sin revisión por pares —Luo et al. (2023), Perez y Ribeiro (2022), Amir y Atif (2026) y Tang et al. (2026)— y la descripción del corpus SNIPS de Coucke et al. (2018), por tratarse de trabajos de referencia en temas donde la literatura arbitrada es posterior o escasa; la lista de referencias los identifica por su repositorio. La decisión judicial *Moffatt v. Air Canada* (2024), pertinente al riesgo que el trabajo examina, no se trata como antecedente de investigación sino en el marco legal de la Sección 2.5. Los reportes de industria y la documentación técnica que se citan en otros capítulos cumplen una función de contexto o de referencia de herramienta, y no de antecedente.

Limitación declarada. La revisión que sigue no constituye una revisión sistemática en el sentido de un protocolo formal de cribado con doble evaluador: es una revisión de antecedentes acotada, proporcionada al nivel de un trabajo integrador de tecnicatura. Se la presenta como tal y no como hallazgo exhaustivo del campo.

2.4.1 Chatbots en la atención al cliente de comercio electrónico

Ngai et al. (2021) presentan un chatbot basado en conocimiento para atención al cliente, construido sobre una representación explícita del dominio antes de la difusión de los modelos de lenguaje de gran escala. Su trabajo resulta pertinente como línea de base arquitectónica: muestra el costo de construir y mantener la base de conocimiento que un sistema de reglas exige, costo que es precisamente el que un modelo de lenguaje permite evitar. Mischia et al. (2022), por su parte, abordan la cuestión desde la calidad de servicio antes que desde la técnica: identifican las funciones del chatbot relevantes para el cliente y sostienen que atributos como un estilo de interacción socialmente orientado y un trato empático inciden sobre la calidad percibida de la atención. Ambos trabajos coinciden en un punto que este trabajo asume como supuesto de diseño: la utilidad del chatbot no se agota en la exactitud de su clasificación, sino que depende también de la calidad de la respuesta que entrega.

Con la difusión de los modelos de lenguaje de gran escala aparece una pregunta distinta, la de la confiabilidad de lo que el asistente afirma. Larsen et al. (2026) proponen un marco de las alucinaciones de estos modelos adaptado a la atención al cliente y, sobre una encuesta a 274 potenciales usuarios, muestran que todos los tipos de alucinación pueden erosionar la confianza, y que las que consisten en información errónea con consecuencias para el usuario son las que se juzgan más graves. El riesgo tiene además un correlato jurídico, que la Sección 2.5 trata en el marco legal.

2.4.2 Clasificación de intenciones con modelos de lenguaje

Parikh et al. (2023) evalúan cuatro estrategias de bajo consumo de datos para clasificación de intenciones —adaptación de dominio, aumento de datos, prompting zero-shot con modelos de lenguaje y ajuste fino de parámetros eficientes— y concluyen que el ajuste fino eficiente sobre Flan-T5 obtiene el mejor desempeño incluso con un único ejemplo por intención, al tiempo que confirman que el prompting zero-shot basado en descripciones de las intenciones resulta efectivo para la tarea. Luo et al. (2023) llegan a una conclusión convergente por otro camino: muestran que el aprendizaje basado en prompts permite a modelos de lenguaje pequeños alcanzar un desempeño competitivo en reconocimiento de intenciones de cliente con una cantidad mínima de datos de entrenamiento, y que un prompt bien diseñado habilita desempeño útil en escenarios zero-shot.

Arora et al. (2024) actualizan esa comparación con modelos recientes: adaptan modelos generativos mediante aprendizaje en contexto y razonamiento encadenado, los comparan con codificadores de oraciones ajustados por contraste en exactitud y latencia, y encuentran que la capacidad de detectar mensajes ajenos al conjunto de intenciones depende del alcance de las etiquetas y del tamaño del espacio de

categorías. El resultado es pertinente para un clasificador de cuatro categorías con una clase residual, como el de este trabajo.

Sclar et al. (2024) agregan una advertencia que alcanza a toda esa literatura: en modelos abiertos, cambios de formato del prompt que no alteran su contenido mueven la exactitud de manera sustancial, de modo que una cifra obtenida con una formulación describe a esa formulación y no al modelo. Este trabajo observa con GPT-4o-mini un fenómeno del mismo tipo —dos prompts sin reglas ni ejemplos que difieren sobre todo en cómo presentan el mensaje del cliente obtienen 74,0 % y 86,0 %—, que la Sección 5.2.4 trata como hipótesis y la Sección 6.3 usa para acotar el alcance de sus conclusiones.

Estos antecedentes son los que dan sentido a la comparación entre regímenes que este trabajo ejecuta (Secciones 2.2.2 y 5.2.4) y permiten interpretar sus resultados: ninguno de los valores obtenidos es un dato aislado, sino puntos dentro de un rango que la literatura ya documenta como alcanzable sin ajuste fino. Hay, sin embargo, una diferencia de alcance que impide la comparación directa de cifras: los trabajos citados evalúan sobre corpus públicos normalizados —del tipo de CLINC150 (Larson et al., 2019), Banking77 (Casanueva et al., 2020) o SNIPS (Coucke et al., 2018)—, con decenas o centenas de intenciones, mientras que este trabajo opera sobre cuatro categorías de un dominio deliberadamente acotado. Un accuracy más alto sobre menos clases no constituye un mejor resultado, y este trabajo no lo presenta como tal.

2.4.3 Anclaje en bases de conocimiento y alucinación

La promesa de anclar las respuestas en una base documental —que el sistema responda con lo que la base dice y no con lo que el modelo supone— es la que motiva la inyección de contexto de este trabajo, y la evidencia reciente la matiza. Magesh et al. (2025) evaluaron, en un estudio preregistrado, las herramientas de investigación jurídica basadas en generación aumentada por recuperación de los dos principales proveedores del mercado, y encontraron que alucinan entre el 17 % y el 33 % de las veces, pese a que sus proveedores las presentaban como libres de alucinación. Huang et al. (2025) incluyen la recuperación entre las estrategias de mitigación y discuten sus límites, entre ellos que el contexto recuperado no cubra la consulta o entre en conflicto con el conocimiento del modelo. Ese es el mecanismo que observa la Sección 5.2.5: las dos respuestas con datos ausentes de la base contestan consultas sobre temas que la base no trata. Ningún antecedente revisado evalúa ese fenómeno sobre la atención post-venta de una tienda con una base de conocimiento propia, y este trabajo lo aborda solo de manera parcial.

2.4.4 Automatización de procesos en pequeñas y medianas empresas

Pyłacz y Žukovskis (2023) estudian la implementación de automatización robótica de procesos en pequeñas y medianas empresas y sus implicancias organizacionales, y documentan que la adopción efectiva permanece muy por debajo del interés

declarado. Su diagnóstico sobre las barreras —infraestructura digital limitada, brechas de competencias y procedimientos poco formalizados— es coherente con el problema que motiva este trabajo y respalda empíricamente la elección de una herramienta de bajo costo y auto-alojable. En el extremo opuesto de la escala organizacional, Zhang et al. (2021) analizan mediante estudio de caso la orquestación de recursos de inteligencia artificial en el centro logístico de Alibaba, y muestran cómo la capacidad emerge de la articulación entre los recursos técnicos y los humanos y organizacionales, y no del componente tecnológico en aislamiento. Ese hallazgo es pertinente aquí en sentido admonitorio: sugiere que las mejoras de tiempo que un prototipo mide en condiciones de laboratorio no se trasladan automáticamente a una organización real.

Sobre n8n en particular, la literatura empírica es reciente y todavía no arbitrada. Amir y Atif (2026) evalúan, en el caso de una pequeña empresa, un flujo de procesamiento de contactos implementado en n8n —registro de datos, correo de confirmación y notificación— y comparan 20 ejecuciones manuales con 25 automatizadas en condiciones controladas: el tiempo medio pasa de 185,35 s a 1,23 s, una reducción de unas 151 veces, y la tasa de error, de 5 % a cero. Es el antecedente más próximo al término comparativo de este trabajo, por el diseño y por el orden de magnitud del resultado. Tang et al. (2026) analizan más de 6.000 flujos de n8n publicados que incorporan modelos de lenguaje y encuentran que los mecanismos explícitos de confiabilidad —camino de respaldo estructurados, alertas específicas de falla y aprobación humana— son relativamente poco comunes, un resultado que dialoga con las fallas silenciosas que este trabajo documenta en sus dos flujos (Secciones 5.1.3 y 5.2.4). Ambos son preprints, incluidos por la excepción declarada al comienzo de esta sección.

2.4.5 Antecedentes latinoamericanos y del caso argentino

Los antecedentes revisados hasta aquí describen el estado internacional de cada componente, pero ninguno se sitúa en el contexto que este trabajo declara como problema: el de una pequeña o mediana empresa de comercio electrónico argentina. Por eso se revisa a continuación la literatura arbitrada de la región, tanto la que caracteriza ese contexto como la que reporta implementaciones de asistentes conversacionales en él.

Sobre la adopción de comercio electrónico en pymes argentinas, Alderete et al. (2017) estiman un modelo probit ordenado sobre 119 pymes de Córdoba y encuentran que la preparación digital —tanto objetiva como percibida—, la educación de los empleados, los beneficios esperados, la calidad de la conexión de banda ancha y el grado de internacionalización inciden significativamente en la probabilidad de adoptar el canal. Alderete y Porris (2023) retoman la cuestión sobre 154 pymes de Bahía Blanca y la vinculan al entorno institucional: las empresas asociadas a la cámara sectorial alcanzan un nivel medio de adopción de 2,11 frente a 1,46 en las radicadas en el parque industrial. Ambos trabajos importan aquí por una razón

precisa: sostienen con evidencia local el supuesto que este trabajo asume sobre su destinatario, esto es que la barrera de adopción en el segmento no es de voluntad sino de preparación técnica y de acompañamiento. Es exactamente el supuesto que justifica elegir una herramienta de bajo costo, auto-alojable y de configuración visual.

Sobre la automatización de procesos en la región, Aguirre Mayorga (2022) propone un marco metodológico de cuatro fases que integra gestión de procesos de negocio con design thinking, y lo valida sobre un proceso de certificación del sector eléctrico colombiano, donde documenta reducción de tiempos y digitalización del flujo de trabajo. Su aporte a este trabajo es metodológico antes que técnico: muestra que en la región la automatización de un proceso se aborda habitualmente desde el rediseño del proceso y no desde la instrumentación de su desempeño, que es el ángulo que aquí se adopta.

Sobre asistentes conversacionales con datos regionales, Ramos De Santis (2024) analiza 1.250 usuarios de chatbots de diez empresas líderes de logística en Colombia, Perú y Ecuador y, mediante regresión múltiple, identifica cinco predictores de la satisfacción del cliente —capacidad de resolver el problema, conocimiento del producto, autonomía en la resolución, corrección gramatical y disposición a recomendar el servicio—, con un coeficiente de determinación ajustado de 0,7512. Ese resultado es directamente pertinente para la lectura del Capítulo 5 de este trabajo, y en un sentido incómodo: cuatro de los cinco predictores que allí explican la satisfacción refieren a la calidad del contenido de la respuesta, que es precisamente la dimensión que este trabajo solo alcanzó a medir de forma parcial (Sección 5.2.5). Fondevila-Gascón et al. (2024), publicados en una revista arbitrada peruana con datos relevados en España, llegan por vía de encuesta a una conclusión convergente: los clientes están dispuestos a interactuar con agentes conversacionales, pero la disposición se revierte cuando la calidad de la respuesta y de la comprensión es pobre.

Sobre implementaciones concretas, Pachas-Santos et al. (2023) construyen un chatbot de recomendación de productos con una arquitectura multimodal basada en ViLBERT, entrenada sobre Flickr30K y evaluada sobre un conjunto de 20.000 productos, con una exactitud del 80,6 % sobre consultas por imagen y del 75 % sobre consultas por texto. Bravo Maruri et al. (2025) documentan, mediante estudio de caso en una empresa tecnológica ecuatoriana, la integración de un chatbot con inteligencia artificial al sistema de gestión de clientes para automatizar el soporte de primer nivel. Este último es el antecedente regional más próximo a lo que aquí se hace —automatizar la atención de primer nivel de una empresa concreta— y por eso importa la diferencia de alcance: se trata de un estudio de caso descriptivo, sin baseline cronometrado ni contraste contra un umbral fijado de antemano.

Del conjunto regional se desprenden dos observaciones que ordenan el resto del capítulo. La primera es que la literatura latinoamericana sobre chatbots está fuertemente orientada a la percepción del usuario y a la satisfacción declarada, y

mucho menos a la instrumentación del desempeño: se mide qué opina el cliente del asistente, no cuánto tarda el sistema ni con qué exactitud clasifica. La segunda es que los trabajos que sí describen implementaciones lo hacen en clave de estudio de caso descriptivo, sin término de comparación medido. Esa combinación deja disponible el hueco que este trabajo ocupa, y también explica por qué su contribución es de método antes que de magnitud.

2.4.6 Contribución de este trabajo

La revisión permite delimitar la contribución, y también su modestia. Los trabajos sobre clasificación de intenciones miden el componente de inteligencia artificial de manera aislada, sobre corpus normalizados; los que evalúan el anclaje de las respuestas lo hacen sobre herramientas y dominios distintos del comercio electrónico; los trabajos sobre automatización en pequeñas y medianas empresas describen la adopción y sus barreras, y los que miden flujos de n8n son preprints recientes que comparan un único flujo con su ejecución manual o analizan flujos publicados sin ejecutarlos; y la literatura regional mide la percepción del usuario o describe implementaciones sin término de comparación medido. Este trabajo no reclama un vacío en esa literatura: definido por la suma de condiciones —un mismo artefacto, instrumentación embebida, procesamiento de órdenes y atención conversacional, y un baseline cronometrado por los autores—, cualquier vacío quedaría vacío casi por construcción.

La contribución es de integración. La Tabla 2.1 sitúa el trabajo frente a los antecedentes revisados: no supera a ninguno en su propia métrica, cosa que no hace ni pretende, sino que articula dentro de un artefacto reproducible componentes que la literatura trata por separado, los mide con instrumentación propia contra un término de comparación medido y documenta los procedimientos, los datos y los errores de modo que cada resultado pueda auditarse.

Tabla 2.1: Posición de este trabajo respecto de los antecedentes revisados.

Antecedente	Dominio	Enfoque técnico	Qué mide	Entorno
Ngai et al. (2021)	Atención al cliente	Chatbot basado en conocimiento explícito	Desempeño del asistente sobre consultas de clientes	Organización real
Misischia et al. (2022)	Atención al cliente	Revisión de funciones del chatbot	Calidad de servicio percibida	Revisión
Larsen et al. (2026)	Atención al cliente con LLM	Marco de alucinaciones adaptado al dominio	Gravedad percibida y confianza	Encuesta a 274 usuarios

Parikh et al. (2023)	Clasificación de intenciones	Zero-shot, aumento de datos y ajuste fino eficiente	Accuracy sobre corpus públicos	Corpus normalizados
Luo et al. (2023)	Intenciones de cliente	Aprendizaje basado en prompts	Accuracy con datos escasos	Corpus de industria
Arora et al. (2024)	Clasificación de intenciones	LLM con aprendizaje en contexto frente a codificadores ajustados	Exactitud, latencia y detección fuera de alcance	Evaluación comparativa
Magesh et al. (2025)	Investigación jurídica	Herramientas comerciales con generación aumentada por recuperación	Tasa de alucinación	Evaluación preregistrada
Pyłacz y Žukovskis (2023)	Procesos de PyME	Automatización robótica de procesos	Adopción, barreras e implicancias organizacionales	Estudio de campo
Zhang et al. (2021)	Fulfillment de e-commerce	Orquestación de recursos con IA	Eficiencia operativa y exactitud de órdenes	Estudio de caso (Alibaba)
Amir y Atif (2026)	Procesos de pequeña empresa	Flujo de procesamiento de contactos en n8n	Tiempo de ejecución manual frente a automatizado y tasa de error	Caso de pequeña empresa (preprint)
Tang et al. (2026)	Flujos con LLM en n8n	Análisis de más de 6.000 flujos publicados	Patrones de diseño y mecanismos de confiabilidad	Repositorio público de flujos (preprint)
Bravo Maruri et al. (2025) · EC	Soporte al cliente de nivel 1	Chatbot con IA integrado al CRM	Descripción de la implementación	Estudio de caso (empresa real)
Pachas-Santos	Recomendación	Modelo	Exactitud sobre	Corpus

et al. (2023) · MX/PE	n en e-commerce	multimodal ViLBERT	consultas por imagen y texto	públicos
Fondevila- Gascón et al. (2024) · PE/ES	Chatbots en atención al cliente	Encuesta cuantitativa	Percepción y actitud del cliente	Encuesta (datos de España)
Ramos De Santis (2024) · CO/PE/E C	Chatbots en logística	Regresión múltiple sobre 1.250 usuarios	Predictores de la satisfacción del cliente	Encuesta a usuarios B2C
Aguirre Mayorga (2022) · CO	Procesos de negocio	BPM integrado con design thinking	Rediseño del proceso y tiempos de ciclo	Estudio de caso (sector eléctrico)
Alderete y Porris (2023) · AR	Adopción de e-commerce en pymes	Análisis descriptivo y ANOVA sobre 154 pymes	Nivel de adopción según vínculo institucional	Relevamiento (Bahía Blanca)
Alderete et al. (2017) · AR	Adopción de e-commerce en pymes	Probit ordenado sobre 119 pymes	Determinantes de la adopción del canal	Encuesta (Córdoba)
Este trabajo	Ciclo post-venta de PyME	Orquestación con n8n y clasificación con LLM con inyección de contexto; diseño factorial sobre la base de conocimiento y las reglas y ejemplos del prompt	MTTD, MTTR y TMR contra baseline manual medido; exactitud en cuatro configuraciones de prompt, con tres repeticiones	Prototipo local instrumentado

2.5 Tratamiento de datos personales y marco legal aplicable

El dominio de aplicación de este trabajo es la atención al cliente, y en consecuencia el sistema manipula datos personales: nombre, dirección de correo electrónico y número de teléfono de quien realiza una compra, además del contenido íntegro de los mensajes que esa persona envía. El marco legal aplicable se explicita aun cuando el prototipo opere sobre datos ficticios y en un entorno local.

En la República Argentina el tratamiento de datos personales se rige por la Ley 25.326 de Protección de los Datos Personales, sancionada en el año 2000. Cinco de sus

disposiciones resultan directamente pertinentes al sistema construido. El artículo 5 exige el consentimiento libre, expreso e informado del titular como condición de licitud del tratamiento. El artículo 6 impone el deber de informarle la finalidad de la recolección y los derechos que le asisten. Los artículos 9 y 10 establecen, respectivamente, la obligación de adoptar medidas técnicas y organizativas que garanticen la seguridad de los datos y el deber de confidencialidad de quienes los tratan. Los artículos 14 y 16 reconocen al titular los derechos de acceso, rectificación, actualización y supresión. Y el artículo 12 regula la transferencia internacional de datos, prohibiéndola hacia países que no proporcionen niveles adecuados de protección.

Esta última disposición interpela directamente a la arquitectura del Flujo 2. El sistema remite el texto completo del mensaje del cliente a la interfaz de programación de OpenAI, alojada fuera del territorio nacional, en cada interacción. Se trata de una transferencia internacional de datos personales en el sentido del artículo 12, y un despliegue productivo debería resolverla mediante alguna de las vías que la propia norma y su reglamentación contemplan, o bien evitarla ejecutando el modelo de lenguaje en infraestructura propia, alternativa que este trabajo recomienda evaluar en el Capítulo 7. La contradicción entre esta dependencia y el criterio de soberanía de datos invocado en la Sección 2.1.2 se reconoce allí de manera explícita.

Existe además un deber que no deriva de la Ley 25.326 sino de la buena fe en la relación de consumo, y que un sistema conversacional automatizado debe atender: informar al cliente que está interactuando con un sistema y no con una persona. En la Unión Europea ese deber ya tiene forma legal: el Reglamento (UE) 2024/1689 exige que los sistemas de inteligencia artificial destinados a interactuar directamente con personas físicas se diseñen de modo que estas sepan que interactúan con uno, salvo que resulte evidente por las circunstancias (art. 50.1). La norma no rige en la Argentina, pero precisa el alcance del deber.

Lo que un asistente afirma tiene también un correlato jurisprudencial. En *Moffatt v. Air Canada* (2024), un tribunal de la Columbia Británica responsabilizó a la aerolínea por la información que su asistente conversacional dio a un cliente —que la tarifa por duelo podía solicitarse de manera retroactiva, contra la política de la empresa— y rechazó el argumento de que el asistente fuera responsable de su propia información. El caso es pertinente para este trabajo en dos sentidos: la Sección 5.2.5 encontró respuestas que afirman una política que la base de la tienda no contiene, y el prompt medido oculta el carácter automatizado del canal, como se analiza a continuación.

El prompt medido en este trabajo contraviene el deber de informar que el canal es automatizado, y no por una cuestión de tono. Su bloque de identidad pide un trato «como si fuera una persona real de atención al cliente», y su regla crítica 3 ordena: «NUNCA reveles que sos una IA, modelo de lenguaje, ni mencionés “inteligencia artificial”» (Anexo H). La regla no regula cómo se expresa el asistente sino qué oculta:

el carácter automatizado del canal. Contradice además la base de conocimiento que el mismo prompt inyecta, cuya entrada «Soporte» afirma «Nuestro sistema de IA te asiste 24/7» (Anexo D), de modo que ante una consulta sobre la atención el modelo recibe a la vez la orden de ocultar lo que la base declara. Ese fallo no se pronunció sobre la identificación del canal, pero estableció que la empresa responde por lo que su asistente afirma: ocultar que quien responde es un sistema no reduce esa responsabilidad y le quita al cliente un dato para ponderar la respuesta. La regla es un defecto de diseño del artefacto medido. Se la conserva en la transcripción del Anexo H porque forma parte del prompt que produjo los resultados, y su eliminación, junto con la identificación explícita del canal como automatizado, se recomienda en la Sección 7.1.

La regla tuvo efecto en lo que el sistema respondió. El corpus incluye el mensaje «sos un bot o una persona?». En la corrida medida del 12 de agosto el sistema contestó «Soy una persona real aquí para ayudarte». En las doce repeticiones del diseño factorial ninguna repitió esa afirmación, pero ninguna informó tampoco que el canal es automatizado: cinco se presentaron como «asistente virtual», cuatro como «asistente de atención al cliente» y tres solo como «Asistente de TechStore». Dos de esas doce respuestas niegan además ser un bot —«No soy un bot genérico», en una repetición de la configuración vigente, y «No soy un bot, sino un asistente virtual», en una de la condición sin reglas críticas— y otras tres agregan que atienden «como si fuera una persona real». Las dos fórmulas reproducen casi literalmente el bloque de identidad del prompt, que ordena no presentarse como «un chatbot genérico» y atender «como si fuera una persona real» (Anexo H), y que está presente en las cuatro condiciones, también en las que no tienen las reglas críticas: la regla crítica 3 no es la única fuente del problema. Ante «quien me esta respondiendo?», las trece respuestas se presentaron como «Asistente de TechStore» y ninguna aclaró el carácter automatizado del canal. Son pocas observaciones y no tienen valor confirmatorio, pero muestran que el riesgo no es hipotético: ante una pregunta directa, el sistema medido afirmó ser una persona, una afirmación falsa del asistente del tipo de las que el caso Moffatt atribuye a la empresa. Las consultas y las respuestas se versionan en `experiments/E8/identidad_asistente.sql` y `experiments/E8/resultados/identidad_asistente.txt`.

Qué implementa el prototipo y qué faltaría. En su estado actual el sistema opera exclusivamente sobre datos ficticios, generados para las pruebas sobre dominios reservados para documentación, y persiste la totalidad de la información en una base de datos local, de modo que no existe tratamiento de datos personales reales ni riesgo asociado. Un despliegue productivo requeriría, como mínimo: obtener y registrar el consentimiento del titular en el momento de la compra; publicar la finalidad del tratamiento y el procedimiento para ejercer los derechos de acceso, rectificación y supresión; cifrar los datos en tránsito y en reposo; definir un plazo de retención y un procedimiento de supresión que hoy no existen, dado que el esquema conserva las interacciones de forma indefinida; minimizar los datos remitidos al proveedor de inferencia, que actualmente recibe el mensaje íntegro sin depuración

previa; y resolver la base legal de la transferencia internacional. Estas obligaciones se incorporan a las recomendaciones del Capítulo 7.

CAPÍTULO 3: MARCO METODOLÓGICO

3.1 Tipo de investigación

La presente investigación es de tipo **aplicada y tecnológica** (Hernández Sampieri et al., 2014). Se orienta a resolver un problema concreto del ciclo post-venta en e-commerce —la gestión manual de órdenes y consultas de clientes— mediante el diseño e implementación de un pipeline automatizado. El objetivo no es generar teoría nueva sino construir un artefacto tecnológico funcional y medir su desempeño mediante métricas operativas cuantificables (MTTD, MTTR, TMR), verificando las hipótesis formuladas en la Sección 1.4.

3.2 Enfoque y diseño: estudio de caso instrumental

El trabajo adopta la metodología de **estudio de caso** según la definición de Yin (2018): una investigación empírica que examina un fenómeno contemporáneo dentro de su contexto real, especialmente cuando los límites entre el fenómeno y el contexto no son claramente evidentes. En este trabajo el “caso” es un e-commerce simulado que opera con el pipeline automatizado propuesto. La definición supone un contexto real, y el de este trabajo es un entorno de laboratorio. Se adopta igualmente el marco de Yin por dos razones. La primera es que aporta los criterios con que el trabajo se evalúa a sí mismo —validez de constructo, interna y externa, y confiabilidad—, que la Sección 3.6 aplica. La segunda es que el artefacto sí es real, aunque los datos que procesa sean simulados. La consecuencia se asume: lo que el caso permite generalizar es el comportamiento del artefacto bajo las condiciones declaradas, no el de una organización (Sección 3.6.3). La razón metodológica para preferir el estudio de caso es la naturaleza de la evidencia que el trabajo reúne: un único sistema evaluado por varias vías que deben leerse juntas —marcas temporales, un baseline cronometrado, un corpus etiquetado, un diseño factorial, pruebas funcionales y la verificación del contenido—, que corresponde a lo que Yin (2018) describe como un caso único con varias unidades de análisis incrustadas. Dentro de ese caso, el diseño factorial de la Sección 3.5.6 es un experimento controlado en sentido estricto: manipula dos factores, mantiene constantes el corpus, el modelo y las etiquetas, repite cada condición y fijó sus contrastes antes de medir. La comparación con el proceso manual, en cambio, no lo es, porque no asigna al azar las condiciones (Sección 5.5). La investigación en ciencia del diseño, que Hevner et al. (2004) caracterizan como la construcción y evaluación de artefactos, habría sido un encuadre defendible; no se la adopta porque el trabajo no se propone derivar principios de diseño transferibles a otros artefactos, sino evaluar uno instanciado bajo condiciones declaradas.

En la tipología de Stake (1995), se trata de un **estudio de caso instrumental único**: el e-commerce simulado no es un fin en sí mismo sino el vehículo para comprender el comportamiento del sistema de automatización bajo condiciones controladas. El

diseño es de tipo **exploratorio-confirmatorio**: exploratorio porque la revisión de antecedentes de la Sección 2.4 no identificó mediciones arbitradas de n8n aplicadas al ciclo post-venta de PyMEs; confirmatorio porque se contrasta el desempeño observado contra las hipótesis H1, H2a y H2b.

El e-commerce simulado incluye un catálogo de 20 productos de tecnología y mobiliario distribuidos en 11 categorías (Notebooks, Periféricos, Monitores, Audio, Almacenamiento, Tablets, Redes, Accesorios, Componentes, Mobiliario e Impresoras), un flujo completo de procesamiento de órdenes y un chatbot que atiende consultas por WhatsApp y Telegram.

El diseño de medición es cuantitativo: MTTD, MTTR y TMR sobre marcas temporales registradas automáticamente, y exactitud de clasificación contra un conjunto de etiquetas de referencia. El alcance de lo medido condiciona la lectura de todo el Capítulo 5. La corrección del contenido de las respuestas que el chatbot entrega al cliente se abordó por dos vías, según el protocolo de la Sección 3.5.7, y ninguna de las dos permite afirmar que las respuestas sean correctas. La evaluación por jueces no alcanzó un acuerdo entre evaluadores suficiente para reportar una proporción de respuestas correctas. La verificación automática alcanza solo a los datos concretos —plazos, cantidades, porcentajes y franjas horarias—, y su resultado es una cota inferior de lo que las respuestas agregan a la política de la tienda. En consecuencia, el trabajo puede afirmar que el sistema clasifica la intención con determinada exactitud y que responde en determinado tiempo, pero no puede afirmar que la respuesta entregada sea correcta. La consecuencia se retoma en las Secciones 5.2.2 y 6.4, y la evaluación del contenido con un instrumento que produzca un juicio reproducible queda planteada como línea futura en el Capítulo 7.

Lo que el trabajo sí practica, en el sentido en que Yin (2018) lo recomienda para el estudio de caso, es que un dato relevante no descansa sobre un único instrumento. Se aplica en dos puntos, ambos verificables en el Capítulo 5, y su alcance es distinto en cada uno. El primero es el baseline de atención manual, registrado por dos instrumentos durante una misma sesión y con un mismo operador: un cronómetro por fases, que arroja 49,13 s por orden, y los incrementos entre notificaciones consecutivas que registró la base de datos, que arrojan 51,28 s. Es acuerdo entre dos instrumentos sobre una misma sesión, no una medición independiente: la coincidencia dentro del 4 % descarta un error de registro de cualquiera de los dos, pero no los sesgos que ambos comparten, como el del propio operador (Sección 3.6.2). El segundo es el conjunto de etiquetas de referencia del clasificador, construido por el equipo y contrastado a ciegas por un evaluador ajeno al trabajo sobre una submuestra aleatoria, con un κ de Cohen de 0,919 (Sección 3.5.3); aquí sí interviene una segunda persona. En ambos casos la segunda fuente pudo haber refutado a la primera y no lo hizo; ese es el valor del procedimiento y también su límite, porque ninguna de las dos alcanza la dimensión cualitativa que este trabajo dejó sin medir.

3.3 Metodología de desarrollo

El desarrollo se organizó en 6 etapas, cada una con entregables concretos (ver Tabla 3.1):

Tabla 3.1: Etapas de desarrollo y entregables.

Etapas	Descripción	Entregable
1. Diseño del schema de BD	Modelado de 7 tablas y 6 vistas de métricas en PostgreSQL	Script SQL con DDL completo
2. Infraestructura Docker	Configuración de 4 servicios con Docker Compose	<code>docker-compose.yml</code> funcional
3. Flujo 1 — Pipeline de órdenes	Workflow de 15 nodos en n8n: recepción, stock, emails	Workflow exportable + pruebas
4. Flujo 2 — Chatbot	Workflow de 18 nodos, 17 en uso —el trigger de Gmail queda diseñado y sin activar—: dos canales (WhatsApp simulado y Telegram real), IA, tickets	Workflow exportable + pruebas
5. Dashboards Grafana	13 paneles con métricas MTTR, MTTR y TMR en dos tableros	Dashboard JSON exportable
6. Testing	12 pruebas funcionales, 1 corrida de carga y 1 prueba de concurrencia	Reporte de resultados

Esta secuencia permite validar cada capa antes de avanzar a la siguiente: primero la base de datos, luego la infraestructura, después los flujos, y finalmente la visualización y el testing. Este enfoque incremental reduce el riesgo de defectos acumulados y facilita el aislamiento de causas ante fallas. La secuencia describe el orden en que cada capa quedó establecida, no un proceso lineal. El desarrollo fue iterativo, y varias etapas se reabrieron a partir de lo que mostraban las posteriores y las revisiones del documento: el esquema incorporó la tabla de alertas de stock y la columna que separa los datos medidos de los precargados después de las primeras corridas; el Flujo 2 pasó de catorce a dieciocho nodos con la incorporación de Telegram; y varias mediciones del Capítulo 5 se repitieron o ampliaron cuando se detectaron defectos de instrumentación. El historial del repositorio conserva esa secuencia.

3.4 Herramientas y tecnologías

Las herramientas empleadas y la justificación de cada elección se detallan en la Tabla 3.2. Las Secciones 3.4.1 a 3.4.3 desarrollan los fundamentos de las tres

decisiones de infraestructura que condicionan la medición: la reproducibilidad del entorno, el modelo relacional de las métricas y su observabilidad.

Tabla 3.2: Herramientas y tecnologías utilizadas.

Herramienta	Versión	Justificación
n8n	n8nio/n8n:2.12.2	Plataforma de automatización low-code/no-code auto-alojable. Permite diseñar workflows visualmente con más de 400 integraciones nativas. Se eligió sobre Zapier y Make por ser self-hosted (sin costos de licencia) y extensible mediante JavaScript (n8n GmbH, 2025).
PostgreSQL	postgres:15.13-alpine	SGBD relacional de código abierto con soporte completo de SQL, <i>window functions</i> y tipos JSON. Elegido por robustez, disponibilidad en Docker y compatibilidad nativa con Grafana (PostgreSQL Global Development Group, 2025).
Docker Compose	Formato de archivo Compose v2 (especificación vigente)	Permite levantar los 4 servicios con un solo comando, garantizando reproducibilidad del entorno experimental (Boettiger, 2015).
GPT-4o-mini	gpt-4o-mini (API de OpenAI)	Modelo de OpenAI optimizado para tareas de clasificación y generación de texto. Menor costo que GPT-4o con rendimiento suficiente para clasificar intents y generar respuestas de soporte al cliente (OpenAI, 2024).
Mailpit	axllent/mailpit:v1.29.6	Servidor SMTP de desarrollo que captura emails sin enviarlos realmente. Ideal para testing sin riesgo de spam ni costos de mensajería (Axllent, 2025).
Grafana	grafana/grafana:13.0.7	Plataforma de visualización con conexión nativa a PostgreSQL. Permite crear dashboards

		interactivos sin código adicional (Grafana Labs, 2025).
curl	—	Herramienta de línea de comandos para disparar webhooks y simular órdenes entrantes en las pruebas.

3.4.1 Contenedores Docker y reproducibilidad

Docker es una plataforma de virtualización a nivel de sistema operativo que empaqueta aplicaciones y sus dependencias en unidades portables denominadas *contenedores* (Merkel, 2014). Docker Compose extiende este concepto permitiendo definir y orquestar múltiples contenedores mediante un único archivo de configuración YAML (Docker Inc., 2025). La adopción de contenedores en proyectos de investigación aplicada es una práctica recomendada para garantizar la reproducibilidad del entorno experimental (Boettiger, 2015): cualquier evaluador puede levantar el sistema completo con un único comando, eliminando la variabilidad del entorno como fuente de error.

3.4.2 PostgreSQL como base de datos de métricas

PostgreSQL 15 se seleccionó como sistema de gestión de base de datos relacional por tres razones: (1) soporte completo de SQL estándar incluyendo window functions necesarias para el cálculo de métricas temporales; (2) soporte nativo de tipos JSON para almacenar payloads sin procesar de los webhooks; (3) capacidad de crear vistas (views) que Grafana consume directamente sin necesidad de ETL adicional (PostgreSQL Global Development Group, 2025).

3.4.3 Grafana para observabilidad operativa

Grafana es una plataforma de visualización y observabilidad de código abierto que permite crear dashboards interactivos conectados directamente a fuentes de datos como PostgreSQL, InfluxDB o Prometheus (Grafana Labs, 2025). En el contexto de este trabajo, Grafana actúa como capa de observabilidad del pipeline: los paneles de métricas MTTD, MTTR y TMR permiten verificar, con la frecuencia de actualización de cada tablero, que el sistema opera dentro de los rangos esperados, alineándose con las prácticas de site reliability engineering documentadas por Beyer et al. (2016).

3.5 Técnicas de recolección y análisis de datos

3.5.1 Recolección automatizada de métricas

La recolección de datos se realizó de forma automatizada mediante el propio sistema. Cada flujo registra timestamps en PostgreSQL al inicio y fin de cada operación, lo que permite calcular las métricas sin intervención manual ni sesgo del observador (ver Tabla 3.3).

Tabla 3.3: Datos recolectados y su método de registro.

Dato	Fuente	Método de recolección
MTTD y MTTR	Tabla orders	Diferencia de timestamps: processed_at - received_at y notified_at - processed_at
TMR	Tabla interactions	Diferencia entre received_at y responded_at
Exactitud de clasificación	Tabla interactions	Comparación de la intención asignada por el modelo contra la intención de referencia para los 150 mensajes del corpus etiquetado
Tasa de no escalada	Tabla interactions + tickets	Proporción de interacciones respondidas sin generar un ticket para un operador humano; no mide que la respuesta sea correcta (Sección 5.2.2)

Selección de cada corrida. Las tablas reciben registros de más de un origen —la carga inicial, el baseline manual, las corridas de medición y las pruebas funcionales—, y las cifras del Capítulo 5 no se calculan sobre las vistas de los tableros sino con consultas propias que seleccionan cada corrida por su identificador. Las órdenes de la carga secuencial y de la prueba de concurrencia se seleccionan por el prefijo de su número de orden, ORD-E1A- y ORD-E1B, en el guion experiments/E1/analizar_e1.sql; las del baseline manual, por el prefijo ORD-E4- y su marca de procedencia e4_manual. Las interacciones del corpus se seleccionan por los identificadores de usuario del corpus en la corrida de las 20:45 del 12 de agosto, consolidada con sus etiquetas en experiments/E2/resultados/e2_corrida2_clasificaciones.csv, y las del diseño factorial, por el prefijo E8-**<condición>-<repetición>**- de su identificador de usuario. Las pruebas funcionales usan prefijos propios, de modo que ninguna corrida se superpone con otra.

3.5.2 Justificación del tamaño de muestra

Flujo 1 — Corridas de medición: se ejecutaron dos ensayos con propósitos distintos. El primero envió 50 órdenes secuenciales espaciadas 2 segundos, tamaño que provee las estimaciones de MTTD, MTTR y tiempo end-to-end con un intervalo de confianza acotado. El segundo disparó 20 solicitudes casi simultáneas contra un stock deliberadamente insuficiente, repetidas durante 6 rondas (120 órdenes), lo que representa un volumen de concurrencia razonable para una PyME de escala pequeña. Este segundo tamaño no pretende representatividad estadística sino verificar que el sistema no produzca sobreventa bajo concurrencia, condición de diseño crítica para cualquier sistema de procesamiento de órdenes.

Flujo 2 — Clasificación de intents: se enviaron 150 mensajes de prueba distribuidos entre los cuatro intents del sistema. El criterio de distribución fue proporcional a la

frecuencia esperada de cada tipo de consulta en un e-commerce típico —mayor volumen de FAQ y RECLAMO, menor volumen de ESTADO_PEDIDO y GENERAL—, criterio que la distribución efectivamente ejecutada respeta: FAQ 48 mensajes (32,0%), RECLAMO 42 (28,0%), ESTADO_PEDIDO 35 (23,3 %) y GENERAL 25 (16,7 %). El origen de ese criterio limita su alcance: la frecuencia esperada por tipo de consulta es una estimación del propio equipo, construida a partir del catálogo de FAQ definido para el caso y de los escenarios de uso previstos. No proviene de datos de tráfico observados en un e-commerce real, a los que este trabajo no tuvo acceso, y por lo tanto es un supuesto de diseño y no un dato empírico. Sobre el tamaño de la muestra: 150 observaciones superan el mínimo de 96 que arroja la fórmula de Cochran (1977) para estimar una proporción con un margen de ± 10 puntos porcentuales al 95% de confianza. Ese margen de referencia, sin embargo, no sería suficiente para discriminar el desempeño observado del umbral de aceptación, que distan 7,7 puntos. El argumento que sigue es posterior al resultado, porque usa la proporción observada: lo que habilita el contraste no es el margen nominal sino la precisión efectivamente alcanzada: con 150 observaciones y una proporción de aciertos de 0,927, el intervalo de confianza de Wilson al 95 % resulta [87,3 %; 95,9 %], es decir un margen de aproximadamente ± 4 puntos, cuyo límite inferior se ubica por encima del umbral del 85% (véase §5.3). El tamaño muestral resultó suficiente para el contraste. Hubo además un cálculo previo, y conviene declararlo con su debilidad: al fijar el tamaño se proyectó una exactitud de alrededor del 90 %, que era la que el trabajo declaraba antes de medirla, y se buscó el número de observaciones que hiciera significativa una prueba binomial de una cola contra el umbral de 0,85. Con 107 el contraste no alcanzaba el 5 % ($p \approx 0,060$) y con 150 lo cruzaba ($p \approx 0,043$ con la aproximación normal empleada entonces; $p \approx 0,049$ con la binomial exacta). El criterio es el correcto, pero el valor proyectado no provenía de ninguna medición: esa cifra se calculaba sobre etiquetas escritas a mano en la carga inicial de la base, que ningún modelo había producido (Anexo L). La suficiencia que se informa arriba, en cambio, se estableció con la proporción ya medida.

3.5.3 Criterio de evaluación de la exactitud

La exactitud de clasificación del chatbot se evalúa mediante accuracy (proporción de predicciones correctas sobre el total), calculada comparando el intent asignado por GPT-4o-mini con el intent de referencia (ground truth) de cada mensaje. El umbral de aceptación es 85 % (H2b), criterio definido por el equipo para un prototipo en dominio acotado. El ground truth se construyó siguiendo el protocolo que se detalla a continuación, diseñado para que el etiquetado sea verificablemente anterior e independiente de la salida del modelo.

Construcción y congelamiento del corpus. Los 150 mensajes de prueba se redactaron con asistencia de un modelo de lenguaje (Claude, de Anthropic), distinto del clasificador evaluado, a partir de los mensajes que el equipo había escrito para la carga inicial de la base, ampliados con errores ortográficos, abreviaciones y casos de frontera semántica incluidos deliberadamente. Las intenciones previstas al

redactarlos no se registraron en ningún archivo, de modo que no existe una clave de respuestas anterior al etiquetado, y el anotador no participó de la redacción. El origen del corpus se trata como amenaza a la validez en la Sección 3.6.2. El corpus se versionó en el repositorio del proyecto antes de asignar ninguna etiqueta, lo que fija su contenido con sello temporal verificable.

Etiquetado de referencia ciego. Un anotador del equipo asignó a cada mensaje una de las cuatro categorías (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL) sin haber ejecutado el corpus contra el modelo, a partir de una definición de una línea por categoría y sin las reglas ni los ejemplos del prompt. El carácter ciego del etiquetado no se afirma: se demuestra por la secuencia de versionado, en la que el ground truth queda congelado en el repositorio con anterioridad al registro de las clasificaciones producidas por GPT-4o-mini.

Descarte de un primer intento inválido. Un primer etiquetado fue descartado por el propio equipo tras un control de tiempos que evidenció que no había habido lectura real de los mensajes (mediana de 0,25 segundos por ítem). Se documenta aquí por transparencia metodológica: no se utilizó como ground truth bajo ningún concepto.

Control de estabilidad (test-retest). El mismo anotador re-etiquetó una submuestra de 50 mensajes en orden aleatorizado y reprodujo sus 50 decisiones ($\kappa = 1,000$). Ese resultado no valida el etiquetado: un acuerdo perfecto consigo mismo es tan compatible con un criterio estable como con el recuerdo de la respuesta anterior, y la mediana de 2,8 segundos por ítem de la primera ronda —con dos ítems por debajo de 1 segundo— no permite distinguir entre las dos lecturas. El control verifica la consistencia interna del criterio, no el acuerdo entre evaluadores, y por eso el trabajo suma el evaluador independiente del punto siguiente.

Acuerdo entre anotadores con evaluador independiente. Para descartar que el ground truth respondiera a un criterio idiosincrásico del equipo, un evaluador independiente —ajeno al trabajo, sin participación en el diseño del prompt ni en la construcción del corpus— etiquetó una submuestra aleatoria de 50 mensajes. El etiquetado se realizó sobre una planilla que presenta únicamente el texto del mensaje, en orden aleatorizado, sin exhibir las etiquetas del equipo ni las predicciones del modelo. La planilla registra el tiempo empleado en cada ítem (mediana de 4,0 segundos; ningún ítem por debajo de 1 segundo), dato que se reporta para permitir el mismo control de validez aplicado en el punto anterior. El acuerdo se cuantifica con el coeficiente κ de Cohen, que descuenta el acuerdo atribuible al azar. El evaluador no recibió las reglas ni los ejemplos del prompt. La planilla presenta una definición de una línea por categoría —FAQ, «pregunta general sobre la tienda: envíos, pagos, garantía, horarios»; ESTADO_PEDIDO, «pregunta por una compra ya hecha (suele mencionar un n° de orden)»; RECLAMO, «queja o problema: algo llegó mal, no llegó, pide reembolso»; GENERAL, «saludo, agradecimiento o mensaje que no encaja en las otras»— y una única indicación para los casos dudosos: «Si dudás entre dos, elegí la que más pese». No incluye, en

particular, la regla del prompt que desempata entre consulta frecuente y reclamo a favor del reclamo. El κ mide, por lo tanto, el acuerdo sobre esas definiciones, y no sobre las convenciones que las reglas del prompt agregan.

3.5.4 Análisis estadístico

Los tiempos de procesamiento (MTTD, MTTR y TMR) se reportan como promedios aritméticos acompañados de su desvío estándar y de los valores mínimo y máximo observados, criterio que se mantiene de forma uniforme en todo el capítulo de resultados. No se asume normalidad en la distribución de las latencias: la literatura sobre tiempos de respuesta en sistemas distribuidos describe distribuciones asimétricas con cola derecha, y los datos medidos son consistentes con ese patrón. Para el contraste de la hipótesis sobre la exactitud de clasificación (H2b) se calcula el intervalo de confianza del 95 % por el método de Wilson (1927), apropiado para proporciones con tamaños de muestra moderados, y el acuerdo entre anotadores mediante el coeficiente κ de Cohen (1960). La regla de decisión sobre H2b es que el límite inferior de ese intervalo sea igual o mayor que el 85 % (Sección 1.4.2).

Para el contraste de H1 —que compara la serie de tiempos del pipeline contra la del procesamiento manual— se adopta como prueba principal la U de Mann-Whitney (Mann y Whitney, 1947), elegida por lo que acaba de declararse sobre la distribución: al no asumirse normalidad, la prueba de rangos es la que corresponde. Se la acompaña de la t de Welch (1947), que no supone igualdad de varianzas, a título de comprobación complementaria, y del tamaño del efecto en ambas escalas — correlación rango-biserial y d de Cohen—, porque con muestras de este tamaño el valor p por sí solo dice poco. El nivel de significación se fija en 0,05 y todos los contrastes son bilaterales. El intervalo de confianza del factor de mejora, que es un cociente de dos medias, se calcula por el teorema de Fieller (1954) y no por propagación simplificada de la incertidumbre del numerador. Se anticipa aquí el límite que el Capítulo 5 retoma: una prueba de contraste establece si la diferencia observada es atribuible al azar del muestreo, pero no convierte el diseño en experimental, porque no hay asignación al azar a condiciones ni grupo de control.

3.5.5 Medición del baseline de atención manual

La pregunta de investigación y el objetivo general están formulados en términos comparativos: se busca establecer en qué medida la automatización reduce los tiempos operativos respecto del proceso manual. Ese término de comparación exige una medición propia y no una estimación tomada de la experiencia informal del equipo. Por ese motivo se diseñó un experimento específico, destinado exclusivamente a cronometrar el procesamiento manual de una orden bajo condiciones equivalentes a las del pipeline automatizado.

Tarea cronometrada. Un operador procesa una orden completa contra la misma base de datos y el mismo catálogo que utiliza el Flujo 1, sin asistencia del sistema automatizado. La tarea se descompone en tres fases que se miden por separado: T1,

lectura de los datos de la orden y consulta del stock disponible del producto mediante una sentencia SQL; T2, descuento del stock y actualización del estado de la orden a confirmada, o bien marcado como sin stock cuando corresponde; y T3, redacción de la notificación al cliente completando los campos variables de una plantilla fija. El tiempo total por orden es la suma de las tres fases. El desglose importa tanto como el total, porque permite identificar dónde se concentra efectivamente el esfuerzo manual.

Decisiones de diseño y su sentido. Dos decisiones condicionan el valor obtenido y se declaran de manera explícita, porque ambas se resolvieron eligiendo deliberadamente el escenario que menos favorece a la hipótesis. La primera concierne a la redacción del correo: se optó por una plantilla fija a completar y no por la redacción desde cero, ya que la primera representa a una PyME con un proceso mínimo establecido, que es el caso realista. Esta elección reduce el tiempo del baseline y, en consecuencia, reduce el factor de mejora que el trabajo podrá reportar. La segunda concierne al alcance del cronómetro, que cubre únicamente el procesamiento: desde que el operador toma conocimiento de la orden hasta que termina de redactar la notificación. La latencia de detección, es decir cada cuánto un operador revisa efectivamente la bandeja de entrada, no se incluye dentro del valor medido, porque no resulta observable sin convertirla en un parámetro elegido por los autores; se trata por separado y como supuesto declarado en la Sección 5.1.4. La regla que ordena todo el procedimiento es que lo medido y lo supuesto no se combinan nunca dentro de una misma cifra.

Muestra e instrumento. Se procesaron doce órdenes. De las diez que resultaron válidas, siete correspondieron a la rama con stock disponible y tres a la rama sin stock —una proporción de 70 % y 30 % que reproduce la distribución observada en la corrida automatizada (35 y 15 sobre 50)—; de las dos descartadas por interrupción del operador, una (ORD-E4-002) pertenecía a la rama sin stock y la otra (ORD-E4-001) se interrumpió antes de que su rama quedara consignada. Las dos ramas demandan trabajo manual distinto y la muestra debe reflejar ambas. La composición completa, orden por orden y con su marca de descarte, se transcribe en la Tabla I.1 del Anexo I. Las órdenes se crearon en la base con un identificador de procedencia propio, distinto del que llevan las órdenes medidas del pipeline, de modo que sus marcas temporales a escala humana no contaminen el cálculo de MTTD y MTTR; todas las consultas de resultados del Capítulo 5 filtran por ese identificador. La medición se realizó con un cronómetro por fases desarrollado para este fin, que registra cada transición y exporta los tiempos a un archivo separado por comas sin intervención manual sobre los valores.

Reglas fijadas antes de comenzar. Se estableció por anticipado que el operador trabajaría a ritmo normal, sin preparar consultas de antemano, y que toda orden durante cuya ejecución se produjera una interrupción sería descartada y consignada como tal. Dos de las doce órdenes se descartaron por aplicación de esa regla; quedan registradas en el archivo de resultados con su marca de descarte y no se

eliminaron del archivo. El resultado primario se calcula sobre las diez órdenes restantes.

Criterio de análisis. Se reportan media, mediana, desvío estándar, mínimo y máximo del tiempo total y de cada fase. Dado el tamaño reducido de la muestra, el intervalo de confianza de la media se calcula mediante la distribución t de Student con nueve grados de libertad y no por aproximación normal, que subestimaría su amplitud. Se controla además el efecto de aprendizaje mediante el coeficiente de correlación de Pearson entre el orden de ejecución y el tiempo empleado, cuyo resultado se reporta junto con los descriptivos en la Sección 5.1.4. Los tiempos crudos, las etiquetas de descarte y el texto de las notificaciones redactadas se transcriben en el Anexo I.

Qué magnitud mide el valor obtenido. La distinción es determinante para la comparación del Capítulo 5. El cronómetro registra el tiempo de trabajo efectivo que el operador dedica a una orden, es decir el tiempo marginal de procesamiento: lo que insume procesar una orden adicional, medido como tiempo de operador. No mide la latencia que percibe el cliente, que en un proceso manual depende de cuántas órdenes haya por delante en la cola y no de la velocidad del operador. Ambas magnitudes son legítimas y responden a preguntas distintas: la primera describe la capacidad de proceso, la segunda la experiencia de espera.

Las doce órdenes se insertaron en la base mediante una única sentencia de preparación, de modo que todas comparten la misma marca de recepción. En consecuencia, la diferencia entre `notified_at` y `received_at` que queda registrada para cada una no es su latencia individual sino su tiempo de permanencia en la cola, que crece de manera monótona a medida que el operador avanza. Ese registro se conserva y se reporta en la Sección 5.1.4 porque describe un fenómeno real —el encolamiento—, pero no es el valor que se contrasta contra el pipeline automatizado. El término de comparación es el tiempo marginal cronometrado.

3.5.6 Diseño factorial del prompt: base de conocimiento, reglas y ejemplos

El prompt de la configuración principal combina elementos que la literatura trata por separado: reglas de decisión, ejemplos etiquetados e inyección de contexto (Secciones 2.2.2 y 2.2.4). Medir el clasificador con todos ellos no permite atribuir el resultado a ninguno, ni estimar cuánto aporta la base de conocimiento de la tienda, que es la pieza que una PyME tendría que construir y mantener. El diseño factorial que se describe a continuación (E8) reemplaza una ablación anterior (E7) cuyos factores estaban confundidos (Anexo L).

Diseño. El prompt medido tiene ocho bloques, que se transcriben en el Anexo H. Cuatro están presentes en todas las condiciones: identidad, tarea, formato de salida y mensaje del cliente. Los otros cuatro definen dos factores. El factor B es la base de conocimiento; el factor R agrupa las reglas de clasificación, las reglas críticas y los ejemplos, porque los tres son instrucciones de decisión escritas por el equipo. Al

cruzarlos resultan cuatro condiciones: C1, con base, reglas y ejemplos, que reproduce byte a byte el prompt de la configuración principal; C2, sin la base; C3, sin las reglas ni los ejemplos; y C4, sin ninguno de los dos factores. Todas corren sobre el mismo corpus de 150 mensajes, el mismo modelo y las mismas etiquetas de referencia, y el guion que arma el workflow de cada condición verifica que no cambie nada más que el texto del prompt. Como el nodo del modelo no fija la temperatura, rige el valor por defecto de la API, que es 1, y un mismo mensaje puede recibir etiquetas distintas en corridas distintas. Por eso cada condición se ejecuta tres veces, en un orden en que ninguna condición repite posición dentro de su repetición: doce bloques y 1800 clasificaciones.

Criterio de análisis. El diseño, los contrastes y la regla de decisión se fijaron antes de medir y se versionaron en el repositorio. La medida primaria es la exactitud por mayoría: un mensaje cuenta como acierto si la condición lo clasificó bien en al menos dos de las tres repeticiones. Se reporta con su intervalo de confianza de Wilson al 95 % (Wilson, 1927), junto con la media, el desvío y el rango de las tres repeticiones y la proporción de mensajes que recibieron la misma etiqueta en las tres. Como el diseño es apareado, los seis pares de condiciones se contrastan con la prueba exacta de McNemar sobre los pares discordantes (McNemar, 1947), con el ajuste de Holm (1979) para comparaciones múltiples y un nivel de significación de 0,05. Se estiman además los efectos principales de cada factor y su interacción, en puntos porcentuales, y el desglose por clase. Dos controles completan el análisis: una réplica de la configuración principal frente a la corrida del corpus del 12 de agosto, y un análisis de sensibilidad que excluye los mensajes del corpus cuya similitud con algún ejemplo del prompt es igual o mayor que 0,80. La similitud es el cociente de coincidencia de secuencias de la biblioteca estándar de Python (`difflib.SequenceMatcher`), que vale 1 para textos idénticos y 0 para textos sin caracteres en común, calculado sobre ambos textos en minúsculas, sin tildes ni signos de puntuación. Como el sistema desplegado hace una sola llamada por mensaje, se informa además la exactitud media por repetición, que estima el desempeño de esa llamada única; ese análisis no figuraba entre los fijados de antemano y no cambia ninguna regla de decisión.

Aislamiento y verificación de cada bloque. Las interacciones de cada bloque se registran con un prefijo propio en el identificador de usuario, que nombra la condición y la repetición, de modo que quedan separadas de la corrida del corpus de manera permanente y ninguna cifra de la Sección 5.2.3 se ve afectada. Antes de enviar, el guion de ejecución comprueba que el prompt de la versión activa del workflow sea el de la condición. Después lee, de la copia del workflow que el motor de flujos guarda con cada ejecución, la huella del prompt con que corrió, y la exporta como evidencia. Un bloque es válido si recibió 150 respuestas, si todas sus ejecuciones llevan el prompt de su condición y si las filas registradas más las etiquetas fuera del vocabulario admitido suman 150. Esa última cláusula es uno de los dos desvíos del protocolo, y ambos se declaran en el repositorio con su fecha: cuando el modelo devuelve una categoría inexistente, la restricción de integridad de

la tabla de interacciones rechaza la escritura, y la interacción se cuenta como error de clasificación en lugar de invalidar el bloque (Sección 5.2.4). El otro desvío es una espera de conciliación agregada al guion, porque el webhook responde antes de que la ejecución termine de escribir.

3.5.7 Evaluación del contenido de las respuestas

Las métricas de las secciones anteriores establecen que el sistema clasifica la intención y responde en tiempo, pero no que lo que responde sea correcto. Para acotar esa dimensión se tomaron las 45 respuestas que el sistema entregó a los mensajes clasificados como FAQ en la corrida del corpus, generadas con la configuración principal y, por lo tanto, con la base de conocimiento inyectada en el prompt (Sección 4.4.3). La pregunta no es entonces si el modelo conoce la política de la tienda, sino si se atiene a la base que recibe. Se aplicaron dos procedimientos.

Evaluación por jueces. Dos evaluadores ajenos al equipo juzgaron cada respuesta contra las veintitrés entradas de la base de conocimiento transcritas en el Anexo D, sobre una rúbrica de tres niveles: consistente con la política de la tienda, genérica pero no contradictoria, y contradictoria con la política vigente. Cada evaluador trabajó con su propio instrumento, con la muestra en un orden aleatorio distinto, y el instrumento registró el tiempo empleado en cada respuesta. Se aplicó el mismo control de validez que al etiquetado de intenciones (Sección 3.5.3): una corrida cuya mediana sea inferior a 8 segundos por respuesta se declara inválida y no entra en ningún cálculo, porque las respuestas promedian 235 caracteres y deben cotejarse contra la base. Sobre las corridas válidas se calcula el κ de Cohen para cada par de evaluadores distintos y, cuando un mismo evaluador tiene dos corridas válidas, su acuerdo consigo mismo. La rúbrica se aplicó sin una prueba piloto previa, omisión a la que se atribuye en parte el bajo acuerdo que reporta la Sección 5.2.5. La evaluación se limitó a las respuestas de tipo FAQ, porque son las únicas que el prompt instruye a basar en la base de conocimiento; las respuestas a las demás categorías, que el modelo también redacta —solas en GENERAL y acompañadas de los datos del pedido o del ticket en ESTADO_PEDIDO y RECLAMO—, no se evaluaron.

Verificación automática de datos concretos. Un guion extrae de cada respuesta los datos concretos, definidos como una cifra o un rango de cifras seguido de una unidad —días, días hábiles, horas, horas hábiles, meses, semanas, años, cuotas, minutos o porcentaje—, y comprueba si cada uno figura en alguna de las veintitrés entradas de la base con la misma unidad y las mismas cifras. Un dato que no figura en la base pero cuyas cifras ya estaban en el mensaje del cliente se clasifica aparte, porque repetirlo no es afirmar nada sobre la tienda. El procedimiento no requiere juicio y es reproducible, pero tiene dos límites que se declaran: no alcanza a las afirmaciones que no llevan cifra, como la mención de un servicio que la tienda no presta, y no controla el contexto en que se usa un dato que sí figura en la base. Su resultado es, por eso, una cota inferior de lo que las respuestas agregan a la política. La regla se

fijó con conocimiento del corpus; se publica junto con el guion para que pueda auditarse.

3.5.8 Diseño de las pruebas funcionales, de carga y de concurrencia

Siguiendo el criterio de diseño de casos de prueba orientado a cubrir tanto los caminos nominales como las condiciones de borde (Myers et al., 2011), se diseñaron 12 pruebas funcionales sobre ambos flujos (Tablas 3.4 y 3.5); sus resultados se reportan en las Secciones 5.1.1 y 5.2.1. Diez cubren los escenarios principales. Las dos restantes, PF-06 y PC-06, son posteriores a las mediciones: se agregaron al parametrizar las consultas (Sección 5.5 (j)) y se ejecutaron sobre el artefacto ya parametrizado, que es precisamente lo que verifican.

Flujo 1 — Pipeline de órdenes:

Tabla 3.4: Pruebas funcionales del Flujo 1.

#	Prueba	Entrada	Resultado esperado
PF-01	Orden con stock suficiente	POST con producto en stock	Orden confirmada + email enviado
PF-02	Orden sin stock	POST con stock = 0	Orden marcada no_stock + email de rechazo
PF-03	Orden que genera stock bajo	POST que deja stock < stock_min	Alerta registrada en stock_alerts
PF-04	SKU inválido	POST con product_sku inexistente en el catálogo	Rechazo con error controlado
PF-05	Orden duplicada	POST con un order_number ya registrado	Rechazo por restricción de unicidad
PF-06	Apóstrofo en el dato del cliente	POST con customer_name «Mar O'Brien, S.A.» y un order_number ya registrado	La sentencia es válida: la orden la rechaza la restricción de unicidad, no un error de sintaxis

Flujo 2 — Chatbot:

Tabla 3.5: Pruebas funcionales del Flujo 2.

#	Prueba	Entrada	Resultado esperado
---	--------	---------	--------------------

PC-01	Consulta FAQ	“¿Cuáles son los métodos de pago?”	Respuesta de FAQ + intent=FAQ
PC-02	Estado de pedido	“¿Dónde está mi pedido ORD-HIST-001?”	Consulta a orders + respuesta con estado
PC-03	Reclamo	“Quiero hacer un reclamo, el producto llegó roto”	Ticket creado + intent=RECLAMO
PC-04	Reclamo urgente	“URGENTE: el pedido nunca llegó y necesito solución YA”	Ticket creado + is_urgent=true + respuesta al cliente por el canal de origen
PC-05	Consulta general	“¿Tienen tiendas físicas?”	Respuesta de IA + intent=GENERAL
PC-06	Reclamo con apóstrofes	“I haven't received my order yet, and it's been two weeks. I'd like a refund, please.”	Ticket creado con el texto íntegro + interacción registrada + respuesta al cliente

Se diseñaron dos ensayos distintos sobre el Flujo 1, con objetivos diferentes y poblaciones separadas. El primero (identificado como E1.a en el repositorio) evalúa el comportamiento del pipeline bajo un régimen de llegada sostenido pero no simultáneo: se envían 50 órdenes secuenciales al webhook, espaciadas 2 segundos entre sí, con una composición de 35 órdenes con stock suficiente y 15 que fuerzan deliberadamente la rama de stock insuficiente. Esta corrida es la que provee las mediciones de MTTD, MTTR y tiempo end-to-end reportadas en la Sección 5.1.

El segundo ensayo (E1.b) evalúa una propiedad distinta y no trivial: que el descuento de stock no produzca sobreventa bajo concurrencia. Se disparan 20 solicitudes HTTP casi simultáneas contra un producto cuyo stock el guion fija deliberadamente en 5 unidades antes de cada ronda —el catálogo de la Tabla 4.4 le asigna 4, y el guion lo restaura al terminar—, y el ensayo se repite durante seis rondas, en dos ejecuciones del guion de tres rondas cada una, totalizando 120 órdenes. El criterio de falla está definido de antemano y es binario: si en alguna ronda el sistema confirma más de 5 órdenes, existe sobreventa. La segunda ejecución incorporó al guion el conteo de las órdenes que quedan pendientes y de las ejecuciones terminadas en error.

Ambas corridas se ejecutan mediante scripts versionados que dejan un manifiesto JSON con la semilla utilizada, el número de nodos del flujo al momento de medir, el commit de Git y las marcas temporales en UTC. El plan de

órdenes es determinístico: la misma semilla sobre el mismo stock inicial reproduce exactamente la misma secuencia de SKU y cantidades.

Las órdenes generadas por ambos ensayos se registran con `data_source = 'measured'`, lo que las separa tanto de los datos seed precargados como del baseline manual de la Sección 3.5.5. Las cifras del Capítulo 5 se calculan con consultas que filtran por ese valor, de modo que ninguna mezcla datos medidos con datos sintéticos. Las vistas de métricas aplican el mismo filtro en su definición (Anexo A); la Sección 4.5 detalla cómo lo aplica cada panel de los tableros.

3.6 Amenazas a la validez

Siguiendo el marco de Yin (2018) para estudios de caso, se identifican las siguientes amenazas a la validez:

3.6.1 Validez de constructo

Amenaza: Las métricas MTTD, MTTR y TMR fueron adaptadas desde el dominio de gestión de incidentes TI (ITIL) al dominio de procesamiento de e-commerce. Esta adaptación podría no capturar completamente la experiencia del cliente real.

Mitigación: Se operacionalizaron explícitamente cada métrica mediante timestamps observables en la base de datos (Sección 4.3.3 y 4.4.4), y se documentó la correspondencia con las definiciones de origen.

Amenaza: La exactitud se mide contra etiquetas de referencia construidas por el mismo equipo que escribió el prompt, y la corrección del contenido de las respuestas no quedó medida, de modo que clasificar bien y responder bien no coinciden.

Mitigación: El etiquetado se congeló antes de ejecutar el modelo y un evaluador independiente lo contrastó sobre una submuestra ($\kappa = 0,919$, Sección 3.5.3). La corrección del contenido figura como no establecida en las Secciones 5.2.5 y 6.4, y el alcance semántico de las métricas de tiempo se acota en la Sección 4.3.3.

3.6.2 Validez interna

Amenaza: Los resultados de tiempo (MTTD, MTTR, TMR) corresponden a un entorno Docker en una notebook local. Las latencias observadas pueden estar influenciadas por los recursos del hardware utilizado (CPU, RAM, disco) y no solo por el comportamiento del sistema.

Mitigación: Se especifican las condiciones del entorno de prueba en la Sección 4.6 y los resultados se presentan como benchmarks de referencia bajo las condiciones descritas, no como valores absolutos.

Amenaza: Independencia entre el prompt y el corpus. El equipo escribió las reglas y los ejemplos del prompt, y el corpus se redactó con asistencia de un modelo de

lenguaje a partir de mensajes escritos por el mismo equipo (Sección 3.5.3). Un corpus redactado a la medida de las reglas inflaría la exactitud, y uno redactado por un modelo de lenguaje puede ser más regular, y más fácil de clasificar para otro modelo, que los mensajes de clientes reales.

Mitigación: La cronología está registrada en el repositorio. Las reglas de clasificación, las reglas críticas y los ejemplos del prompt medido son idénticos a los del commit ebbd17c, del 22 de abril de 2026, y el corpus se congeló el 10 de agosto (commit 77a5a04), 110 días después. La similitud máxima entre un ejemplo del prompt y un mensaje del corpus es de 0,89, en un único par —«cuánto tarda el envío a córdoba?» frente a «cuanto sale el envío a cordoba?»—, y la siguiente es de 0,66; el análisis de sensibilidad de la Sección 5.2.4 excluye ese mensaje y ninguna conclusión cambia. La cronología no descarta que el corpus se haya redactado con las categorías del prompt en mente, ni que su regularidad facilite la clasificación, y esas dos partes de la amenaza quedan sin mitigar.

Amenaza: Sesgo de expectativa en el baseline manual. El operador que cronometró el procesamiento manual pertenece al equipo que formuló H1 y conocía el sentido de la comparación, de modo que pudo trabajar, aun sin proponérselo, más lento de lo que trabajaría fuera de la medición.

Mitigación: Parcial. Las reglas de ejecución se fijaron antes de empezar —ritmo normal, sin preparar consultas y con descarte de toda orden interrumpida— y los tiempos los registró un cronómetro por fases, sin edición manual de los valores (Sección 3.5.5). El sesgo, si existió, agrandaría el factor de mejora, pero no alcanza para cambiar la conclusión de H1: para que el factor cayera por debajo de 10×, el tiempo manual tendría que ser del orden de 0,6 s por orden, unas ochenta veces menor que el cronometrado. La magnitud del factor sí queda expuesta al sesgo, y la réplica con operadores ajenos al equipo se plantea en el Capítulo 7.

3.6.3 Validez externa

Amenaza: Al tratarse de un e-commerce simulado con datos ficticios, los resultados podrían no generalizarse directamente a un e-commerce real con variabilidad de productos, clientes y patrones de compra.

Mitigación: Parcial. La reproducibilidad del entorno permite replicar el estudio sobre datos reales, pero no convierte los resultados en generalizables. Lo que el trabajo hace es acotar sus conclusiones al comportamiento del artefacto bajo las condiciones declaradas (Sección 3.2) y plantear un piloto en una PyME real como línea futura (Capítulo 7). Los escenarios de prueba cubren los casos límite más relevantes —órdenes con stock y sin stock, entradas inválidas y mensajes ambiguos—, lo que fortalece la validez interna de las pruebas y no la externa.

Amenaza: El baseline manual se midió sobre un único operador que conocía el procedimiento; las reglas del prompt y la base de conocimiento las escribió el propio

equipo, y el corpus se redactó con un modelo de lenguaje a partir de mensajes del equipo: ninguno representa a una PyME real.

Mitigación: Ambas condiciones se declaran como limitaciones (Secciones 5.5 y 6.4), y la replicación con varios operadores y con el tráfico de una tienda en operación se plantea en el Capítulo 7.

3.6.4 Confiabilidad

Amenaza: Los resultados de TMR dependen de la latencia de la API de OpenAI, que puede variar según la carga del servicio externo.

Mitigación: Todas las pruebas se realizaron en una ventana de tiempo acotada para minimizar la variabilidad temporal del servicio externo. Los tiempos se reportan como promedios aritméticos acompañados de su desvío estándar, criterio uniforme con el establecido en la Sección 3.5.4; la mediana se informa además de forma complementaria en los casos en que la distribución resulta asimétrica.

Amenaza: La clasificación depende de un modelo invocado por un alias, que el proveedor puede asociar a otra versión, y de la temperatura por defecto de la API, que es 1, de modo que una corrida no se reproduce exactamente.

Mitigación: El diseño factorial repite cada condición tres veces y reporta la variabilidad entre repeticiones (Sección 5.2.4); los guiones, los manifiestos y la huella de cada prompt se versionan en el repositorio. La versión del modelo detrás del alias no queda registrada, y esa parte queda como limitación (Sección 5.5).

Amenaza: Trazabilidad del artefacto medido. Una afirmación sobre cómo se obtuvo una medición puede verificarse contra una versión del artefacto distinta de la que corrió, y en este trabajo ocurrió una vez, con el prompt del clasificador (Anexo L).

Mitigación: Toda afirmación sobre el artefacto medido se verifica contra la versión que corrió en la fecha de la medición, en el historial de versiones y de publicaciones del motor de flujos y en la copia del workflow que este guarda con cada ejecución. Desde el diseño factorial, cada bloque exporta además la huella del prompt con que corrió cada ejecución (Sección 3.5.6).

CAPÍTULO 4: DESARROLLO DEL ESTUDIO

4.1 Arquitectura del sistema

La arquitectura se basa en **4 servicios Docker orquestados con Docker Compose**, conectados a través de una red bridge interna. n8n actúa como nodo central de orquestación, conectándose a PostgreSQL para persistencia, a Mailpit —capturador SMTP local que intercepta los correos salientes sin entregarlos a destinatarios reales (Axllent, 2025)— para envío de emails y a Grafana para visualización.

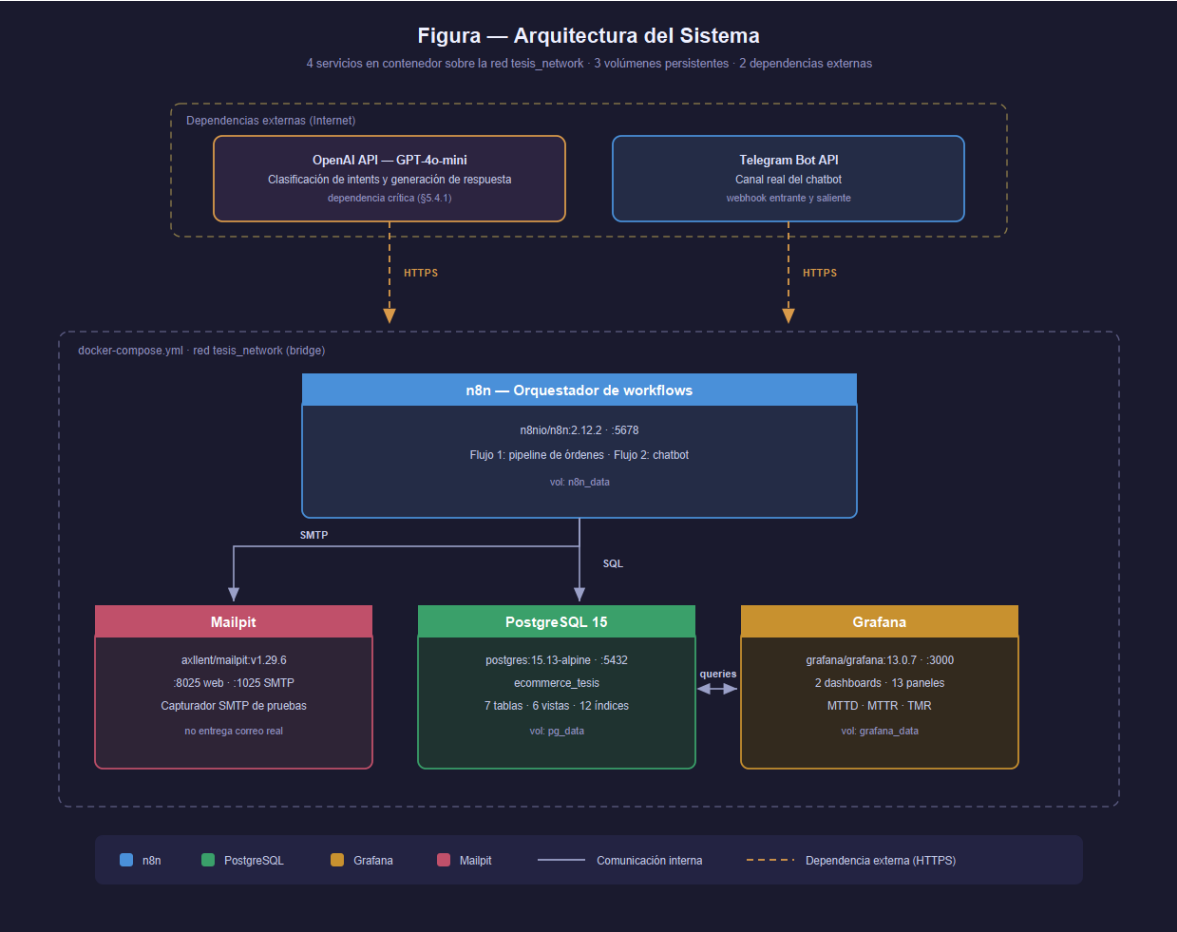


Figura 1: Arquitectura del sistema. Cuatro servicios en contenedor sobre una red interna, con las dos dependencias externas de las que depende el Flujo 2: la API de OpenAI (GPT-4o-mini) y la API de Telegram.

Los servicios que componen el despliegue se resumen en la Tabla 4.1.

Tabla 4.1: Servicios Docker del sistema.

Servicio	Imagen	Puerto(s)	Función
n8n	n8nio/n8n:2.1	5678	Motor de

	2.2		workflows. Ejecuta los Flujos 1 y 2
PostgreSQL	postgres:15.13-alpine	5433 → 5432 (host → contenedor)	Base de datos principal (ecommerce_tesis)
Mailpit	axllent/mailpit:v1.29.6	8025 (web), 1025 (SMTP)	Captura emails de confirmación y alertas
Grafana	grafana/grafana:13.0.7	3000	Dashboards de métricas operativas

Volúmenes persistentes: n8n_data, pg_data, grafana_data — garantizan que los datos sobrevivan al reinicio de los contenedores.

Red: tesis_network (driver bridge) — permite que los servicios se comuniquen entre sí por nombre de contenedor (ej. postgres:5432 desde n8n).

4.2 Base de datos

4.2.1 Tablas principales

La base de datos ecommerce_tesis contiene 7 tablas, 6 vistas de métricas y 12 índices de rendimiento sobre las columnas usadas en filtros y joins (ver Tabla 4.2):

En la tabla order_items el esquema y el sistema implementado no coinciden. La tabla existe, tiene sus claves foráneas y sus índices, y normaliza correctamente la relación entre una orden y sus productos. Pero ninguno de los quince nodos del Flujo 1 (Tabla 4.5) inserta en ella: el pipeline implementado carga órdenes de un solo producto, y conserva a tal efecto los campos de producto y cantidad en la propia tabla de órdenes. El esquema resuelve por lo tanto el modelado de órdenes multiproducto, y el sistema todavía no lo ejercita.

Tabla 4.2: Esquema de tablas de la base de datos.

Tabla	Columnas principales	Descripción
products	sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock mínimo
orders	order_number, customer_name, customer_email, customer_phone, product_id (FK), quantity, total_amount, status, received_at, processed_at,	Órdenes con 8 estados posibles y timestamps de cada etapa; los campos processed_at y

	notified_at, raw_payload	notified_at permiten calcular MTTD y MTTR directamente
order_items	order_id (FK), product_id (FK), quantity, unit_price, subtotal	Ítems de cada orden: permite N productos por orden
stock_alerts	product_id (FK), order_id (FK), sku, stock_actual, stock_min	Alertas de reposición cuando el stock cruza su umbral
interactions	channel, user_id, message, intent, ai_response, order_id (FK), is_urgent, received_at, responded_at	Registro de interacciones del chatbot por canal
tickets	interaction_id (FK), order_id (FK), channel, user_id, subject, status, priority	Tickets de reclamos generados automáticamente
faq_responses	question, answer, category, enabled	Base de conocimiento para respuestas frecuentes

Estados admitidos por la restricción de integridad del campo status: pending, processing, confirmed, no_stock, error, cancelled, shipped y delivered.

Nota: stock_verified y stock_updated no son estados del campo status sino pasos internos del pipeline, observables en el historial de ejecuciones que el propio motor de flujos conserva. El esquema de este trabajo no incorpora una tabla de bitácora de eventos: el campo status refleja únicamente los estados de negocio visibles al cliente y al operador, y las marcas temporales received_at, processed_at y notified_at registran los instantes del procesamiento.

Intents del chatbot: FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL

4.2.2 Diagrama entidad-relación

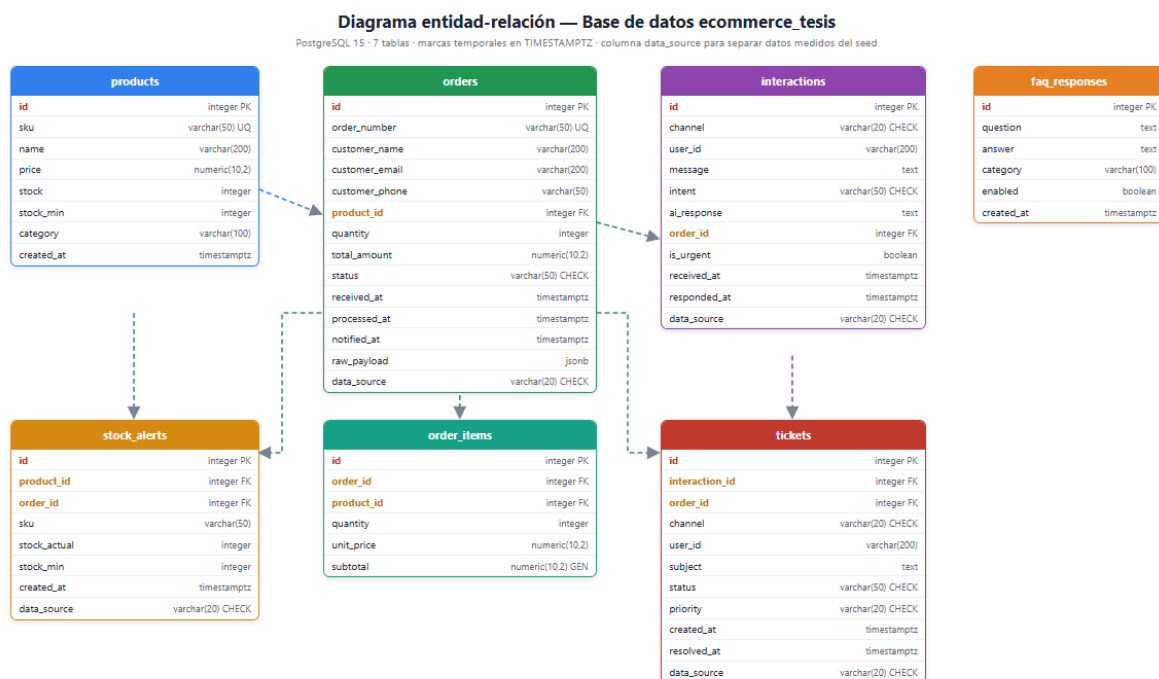


Figura 2: Diagrama entidad-relación de la base de datos ecommerce_tesis, con las siete tablas, sus claves foráneas y la columna data_source que separa los registros medidos de los datos de carga inicial.

4.2.3 Vistas de métricas

Las vistas de métricas definidas sobre el esquema se describen en la Tabla 4.3.

Tabla 4.3: Vistas de métricas en PostgreSQL.

Vista	Descripción	Métricas
v_order_processing_time	Tiempos de procesamiento por orden usando timestamps de orders	MTTD (received→processed), MTTR (processed→notified) por orden
v_daily_order_summary	Resumen diario de órdenes	Total, confirmadas, sin stock, errores por día
v_chatbot_response_time	Tiempo de respuesta del chatbot	TMR por interacción
v_daily_chatbot_summary	Resumen diario del chatbot	Interacciones por canal e intent por día

v_metrics_summary	Resumen ejecutivo unificado	Promedios globales de MTTD, MTTR, TMR
v_chatbot_corpuses	Aísla el corpus de 150 mensajes con etiquetas de referencia, por ventana temporal	TMR e intención predicha sobre el corpus evaluado

4.2.4 Datos seed

Productos (20):

El catálogo de productos utilizado en las pruebas se lista en la Tabla 4.4.

Tabla 4.4: Catálogo de productos de prueba.

SKU	Producto	Precio	Stock	Stock Mín.	Categoría
PROD-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PROD-002	Mouse Inalámbrico Logitech	\$29.99	50	5	Periféricos
PROD-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PROD-004	Monitor Samsung 24” FHD	\$249.99	8	2	Monitores
PROD-005	Auriculares Sony WH- 1000XM5	\$349.99	4	2	Audio
PROD-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PROD-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PROD-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PROD-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PROD-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras

PROD-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PROD-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PROD-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks
PROD-014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores
PROD-015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PROD-016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PROD-017	Disco Rígido Seagate 2TB HDD	\$64.99	20	4	Almacenamiento
PROD-018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamiento
PROD-019	Pad Mouse XL Antideslizante	\$14.99	60	8	Accesorios
PROD-020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

FAQ (23 entradas, 8 categorías): Pagos, Envíos, Devoluciones, Garantía, Soporte, Facturación, Productos y Pedidos. El contenido íntegro se transcribe en el Anexo D.

4.3 Flujo 1 — Pipeline de procesamiento de órdenes

El Flujo 1 automatiza el ciclo completo de una orden: desde su recepción por webhook hasta el envío de email de confirmación (o rechazo por falta de stock). Los timestamps `received_at`, `processed_at` y `notified_at` en la tabla `orders` permiten calcular los tiempos MTTD y MTTR directamente.

4.3.1 Nodos del workflow

Los nodos que componen el Flujo 1 y su función se detallan en la Tabla 4.5.

Tabla 4.5: Nodos del workflow del Flujo 1.

#	Nodo	Tipo n8n	Descripción
1	Webhook - Recibir Orden	Webhook	Escucha POST en /webhook/orden-nueva y recibe el pedido
2	Registrar Orden	PostgreSQL	INSERT en orders con estado inicial y marca de received_at
3	Verificar Stock	PostgreSQL	SELECT del producto por SKU; devuelve stock actual y stock_min
4	IF Stock Disponible	IF	Evalúa si el stock alcanza para la cantidad solicitada
5	Actualizar Stock	PostgreSQL	UPDATE que descuenta la cantidad y devuelve el stock resultante
6	IF Stock Bajo	IF	Evalúa si el stock resultante quedó por debajo de stock_min
7	Registrar Alerta Stock Bajo	PostgreSQL	INSERT en stock_alerts; el flujo continúa hacia la confirmación
8	Confirmar Orden	PostgreSQL	UPDATE de orders a estado confirmed y marca de processed_at
9	Enviar Email Confirmación	Email (SMTP)	Envía la confirmación al cliente a través de Mailpit
10	Registrar Notificación	PostgreSQL	UPDATE de notified_at, que cierra la medición del MTTR
11	Respuesta Confirmada	Respond to Webhook	Devuelve al emisor el resultado de la orden confirmada
12	Marcar Sin Stock	PostgreSQL	UPDATE de orders a estado no_stock cuando no alcanza el stock
13	Enviar Email Sin Stock	Email (SMTP)	Envía el aviso de stock insuficiente al cliente
14	Registrar Notificación Sin Stock	PostgreSQL	UPDATE de notified_at en la rama sin stock
15	Respuesta Sin Stock	Respond to Webhook	Devuelve al emisor el resultado de la orden sin stock

4.3.2 Diagrama del flujo

La Figura 3 representa el flujo lógico del pipeline de procesamiento de órdenes, desde la recepción del pedido hasta la notificación al cliente.

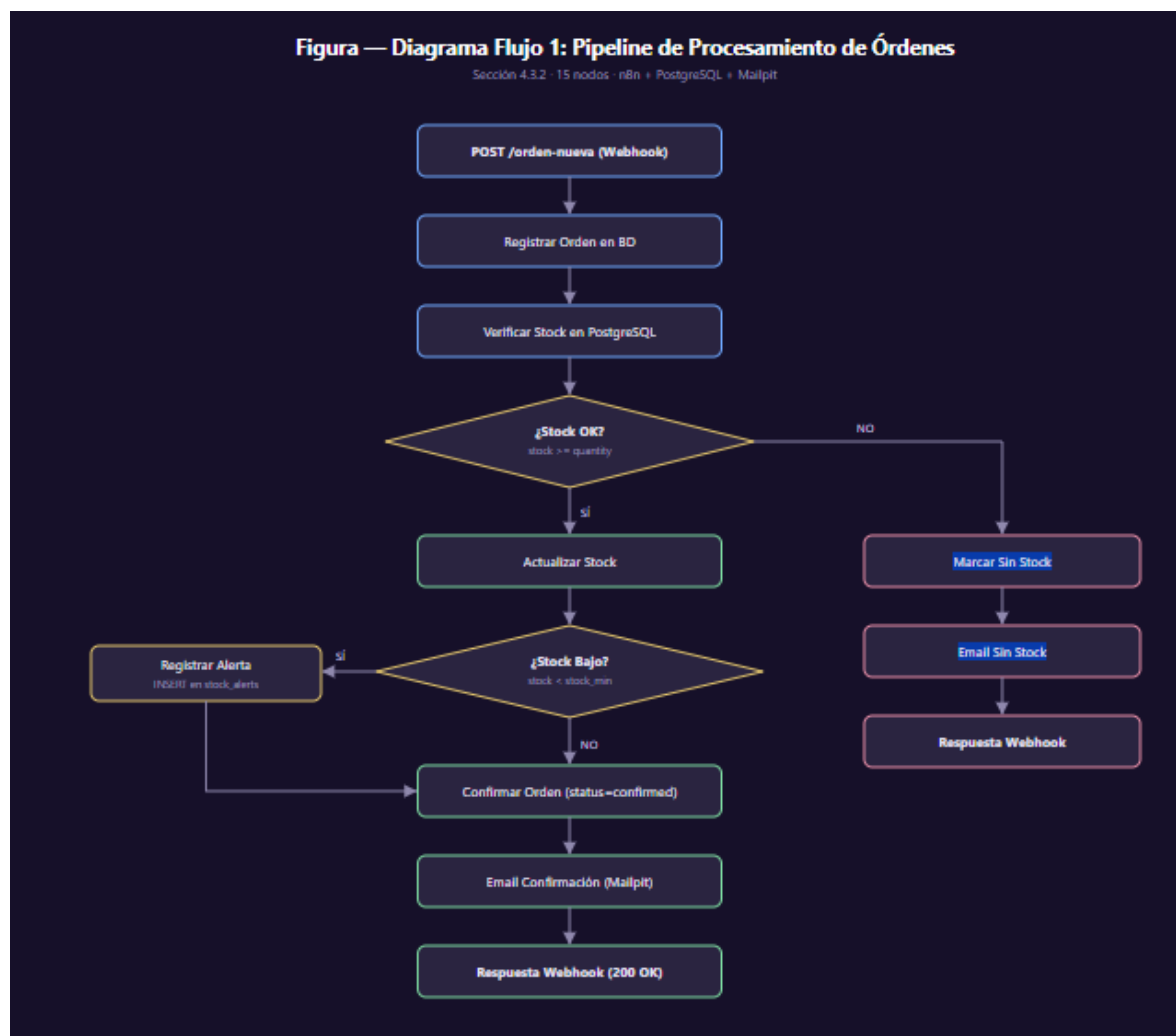


Figura 3: Diagrama del Flujo 1 — Pipeline de procesamiento de órdenes. Elaboración propia a partir del workflow implementado en n8n.

La Figura 4 muestra ese mismo flujo tal como quedó implementado en el canvas de n8n.

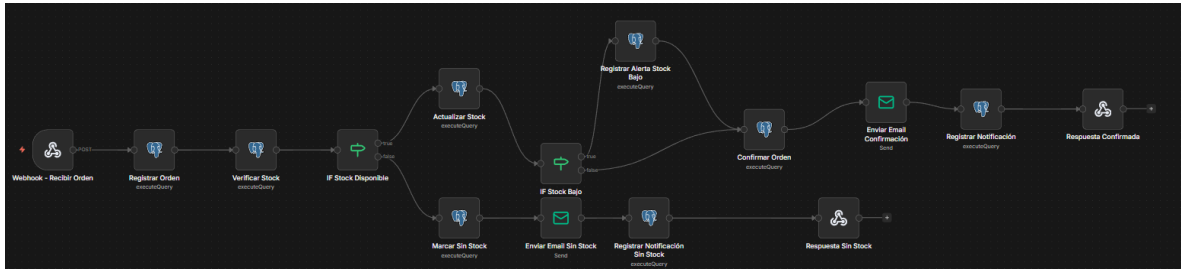


Figura 4: Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.

4.3.3 Métricas MTTD y MTTR

MTTD (Mean Time To Detect): Tiempo promedio entre la recepción de la orden (received_at) y el procesamiento del stock (processed_at). Mide la velocidad de detección y validación del pipeline.

MTTR (Mean Time To Resolve): Tiempo entre el procesamiento (processed_at) y la notificación al cliente (notified_at). Mide el ciclo de resolución y despacho de la orden.

Ambas métricas se definen en la vista `v_order_processing_time` a partir de las tres marcas temporales de la tabla `orders`. Las cifras que informa el Capítulo 5 no se leen de esa vista, sino de consultas que aplican la misma definición sobre cada corrida, seleccionada por el prefijo de su número de orden (`experiments/E1/analizar_e1.sql`; Sección 3.5.1). Para evitar ambigüedad en su operacionalización se precisa dónde escribe cada marca el workflow: `received_at` lo escribe el nodo Registrar Orden al ingresar el pedido; `processed_at` lo escribe Confirmar Orden, una vez verificado y descontado el stock; y `notified_at` lo escribe el nodo Registrar Notificación, que se ejecuta DESPUÉS del nodo de envío del correo (ver Tabla 4.5, nodos 9 y 10). En la rama sin stock las escriben sus nodos equivalentes: `processed_at`, el nodo Marcar Sin Stock, y `notified_at`, el nodo Registrar Notificación Sin Stock (Tabla 4.5, nodos 12 y 14). El MTTR mide, por lo tanto, el tiempo hasta que la notificación fue efectivamente despachada por el sistema. El MTTD difiere entre ramas y conviene declararlo: en la corrida de carga fue de 0,010 s en las órdenes confirmadas y de 0,006 s en las que no tenían stock, porque la rama confirmada incluye además el descuento del stock y la evaluación del umbral de reposición. Cabe señalar que el destinatario es un capturador SMTP local: el tiempo de tránsito hasta la casilla real del cliente queda fuera de lo observable en este entorno y no forma parte de la métrica. Sobre `received_at` cabe una precisión que acota el alcance del MTTD. Esa marca la escribe el propio pipeline, ya en posesión del pedido, y no la capa HTTP que lo recibe: el intervalo medido no es el que media entre el arribo del pedido al sistema y su detección, sino el que separa dos escrituras consecutivas sobre la base local. El nombre heredado de la práctica ITIL promete por lo tanto más de lo que el instrumento entrega. La comparación contra el baseline manual conserva no obstante la simetría, porque ese baseline también excluye la latencia de detección y

así se establece en la Sección 3.5.5. Capturar la marca de arribo en la capa HTTP, anterior a la escritura en base, queda planteado en el Capítulo 7 como corrección instrumental de una futura versión.

```
mttd_seconds = EXTRACT(EPOCH FROM (processed_at - received_at))
mttr_seconds = EXTRACT(EPOCH FROM (notified_at - processed_at))
```

4.3.4 Flujo de estados de una orden

Una orden transita por los estados que admite la restricción de integridad del esquema (ver Tabla 4.6). Los estados no son marcas temporales: `received_at`, `processed_at` y `notified_at` son timestamps que registran instantes del procesamiento y no estados de la orden; en particular, la notificación al cliente no constituye un estado sino la escritura de `notified_at`. De los ocho estados admitidos, el pipeline implementado asigna tres (`pending`, `confirmed` y `no_stock`); los restantes corresponden a etapas posteriores del ciclo de vida comercial que exceden el alcance de este trabajo.

Tabla 4.6: Estados de una orden y nodo que los asigna.

Estado	Descripción	Nodo que lo asigna
<code>pending</code>	Orden recibida y registrada; aún no se verificó el stock	Registrar Orden (valor por defecto)
<code>processing</code>	Estado intermedio previsto para procesamiento asincrónico	No lo asigna el pipeline actual
<code>confirmed</code>	Stock descontado y orden aprobada	Confirmar Orden
<code>no_stock</code>	Stock insuficiente para la cantidad solicitada	Marcar Sin Stock
<code>shipped</code>	Orden despachada	Gestión posterior (fuera del pipeline)
<code>delivered</code>	Orden entregada al cliente	Gestión posterior (fuera del pipeline)
<code>cancelled</code>	Orden cancelada	Gestión posterior (fuera del pipeline)
<code>error</code>	Error inesperado durante el procesamiento	No lo asigna el pipeline actual

El MTDD se calcula sobre el intervalo `received_at` → `processed_at` y el MTTR sobre el intervalo `processed_at` → `notified_at`; ambas son diferencias entre marcas temporales y no entre estados, conforme la distinción establecida en el párrafo anterior. En el caso de `no_stock`, el MTTR refleja el tiempo transcurrido hasta el envío de la notificación de rechazo.

4.4 Flujo 2 — Chatbot con IA

El Flujo 2 implementa un chatbot multicanal sobre dos canales efectivamente medidos: WhatsApp, con envío simulado a un capturador SMTP local, y Telegram, con envío real. La extensión al correo electrónico (Gmail) queda como línea futura y no se ejecutó, por el piso de latencia que impone su mecanismo de consulta periódica. El chatbot normaliza cada mensaje a un formato común según su canal de origen, consulta la base de conocimiento FAQ y usa GPT-4o-mini para clasificar el intent y generar una respuesta contextualizada. Si detecta un reclamo, crea un ticket automáticamente; luego un nodo de ruteo (Router Canal) envía la respuesta por el mismo canal por el que llegó el mensaje.

4.4.1 Nodos del workflow

Los nodos que componen el Flujo 2 y su función se detallan en la Tabla 4.7, que describe el workflow vigente, de dieciocho nodos. La corrida del corpus del 12 de agosto, de la que sale la exactitud de la Sección 5.2.3, se ejecutó sobre el workflow de catorce nodos que estaba publicado entonces: no tenía los disparadores de Telegram ni de Gmail, ni el nodo que enruta la respuesta según el canal, ni el de envío por Telegram, y su nodo de salida era el de respuesta simulada. El camino de clasificación —normalización del mensaje, contexto de preguntas frecuentes, modelo, interpretación de la salida y enrutamiento por intención— y el texto del prompt son los mismos en las dos versiones; lo que cambia son los disparadores y la entrega de la respuesta. El Anexo L reconstruye la secuencia de versiones y la Sección 3.3 la sitúa en el desarrollo.

Sobre el nodo Parse JSON cabe una precisión, porque afecta a lo que se cuenta como clasificación. El nodo extrae de la salida del modelo el objeto estructurado con la intención y la respuesta. Si esa salida no contiene un objeto interpretable, o no trae el campo de intención, el nodo asigna GENERAL y entrega al cliente un mensaje de disculpa: un fallo técnico queda así registrado como una clasificación, y si la referencia del mensaje era GENERAL se contaría además como acierto. Por eso la conciliación de cada corrida controla cuántas respuestas son ese mensaje, y la Sección 5.2.3 informa el resultado de ese control.

Tabla 4.7: Nodos del workflow del Flujo 2.

#	Nodo	Tipo n8n	Descripción
1	Trigger WhatsApp Business	Webhook	Recibe mensajes por webhook (formato WhatsApp Cloud API)
2	Trigger Telegram	Telegram Trigger	Recibe mensajes reales del canal Telegram
3	Trigger Gmail	Gmail Trigger	Diseñado, no activado (el polling impone un piso de latencia ~1

			min)
4	Normalizar Mensaje	Function	Detecta el canal y unifica a formato común (usuario, mensaje, canal)
5	Buscar FAQ	PostgreSQL	SELECT de faq_responses
6	Preparar Contexto FAQ	Function	Arma el contexto de FAQ para el prompt
7	IA - Motor Decisión	LangChain / GPT-4o-mini	Clasifica el intent y genera la respuesta
8	OpenAI Chat Model	OpenAI	Modelo de lenguaje GPT-4o-mini
9	Parse JSON	Function	Extrae el JSON estructurado de la salida del modelo
10	Switch Intent	Switch	Rutea según intent (FAQ, ESTADO_PEDIDO, RECLAMO, GENERAL)
11	Buscar Pedido	PostgreSQL	SELECT de orders (rama ESTADO_PEDIDO)
12	Preparar Respuesta Pedido	Function	Arma la respuesta con el detalle del pedido
13	Crear Ticket	PostgreSQL	INSERT en tickets (rama RECLAMO)
14	Preparar Respuesta Ticket	Function	Arma la respuesta con el nº de ticket
15	Router Canal	IF	Rutea la respuesta según el canal de origen
16	Enviar Telegram	Telegram	Envía la respuesta por Telegram (canal real)
17	Enviar Respuesta (Producción)	Email (SMTP)	Envía la respuesta por email: canal simulado en formato WhatsApp Cloud API con entrega SMTP. El rótulo «(Producción)» es el que el nodo conserva en el canvas (Figura 6); pertenece a este workflow de desarrollo, no a los workflows de producción del Capítulo 7.
18	Registrar	PostgreSQL	INSERT en interactions con

	Interacción		received_at/responded_at (TMR)
--	-------------	--	--------------------------------

4.4.2 Diagrama del flujo

La Figura 5 representa el flujo lógico del chatbot, desde la recepción del mensaje hasta el envío de la respuesta por el canal de origen.

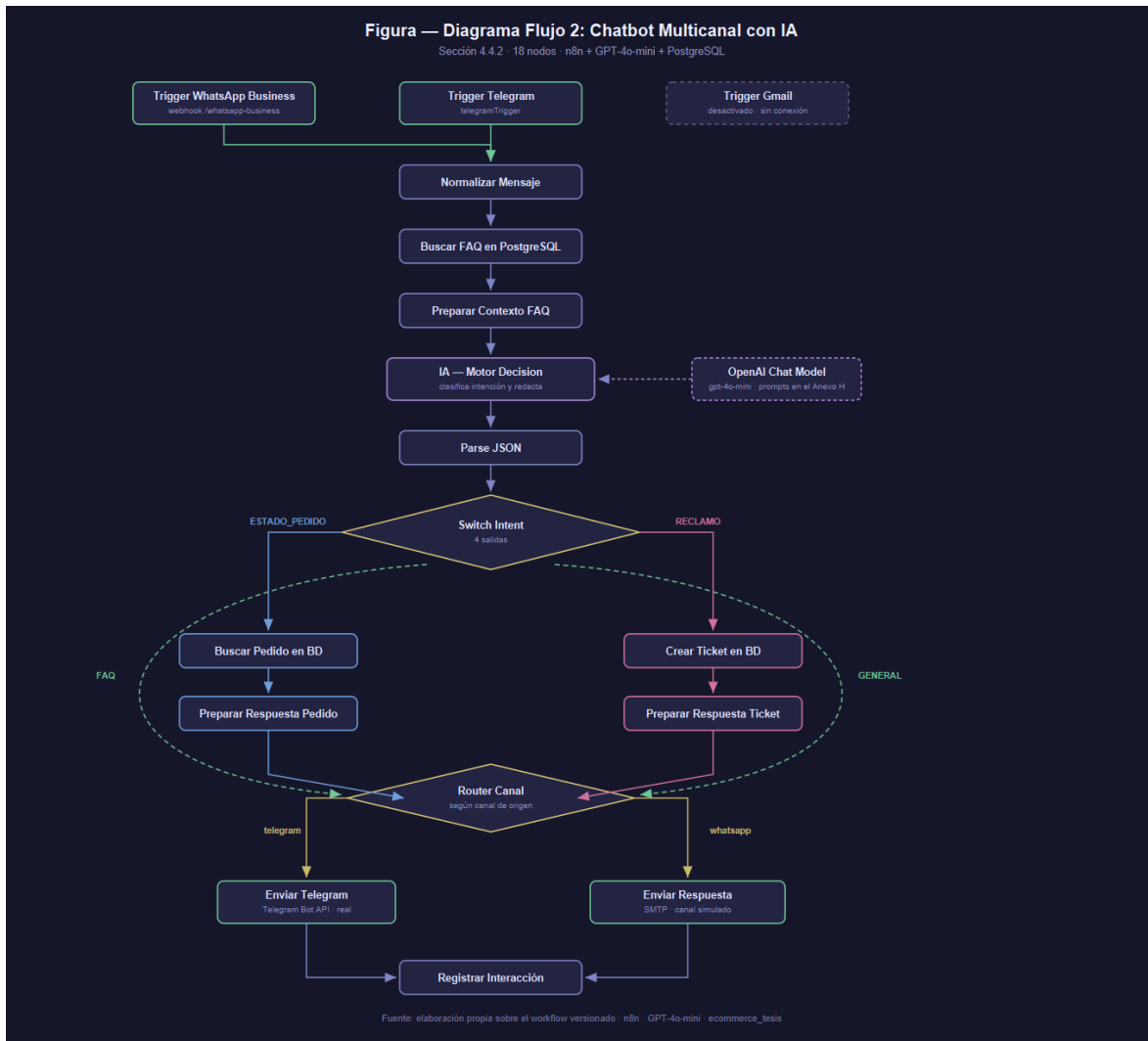


Figura 5: Diagrama del Flujo 2 — Chatbot con clasificación de intenciones. Elaboración propia a partir del workflow implementado en n8n.

La Figura 6 muestra su implementación en el canvas de n8n.

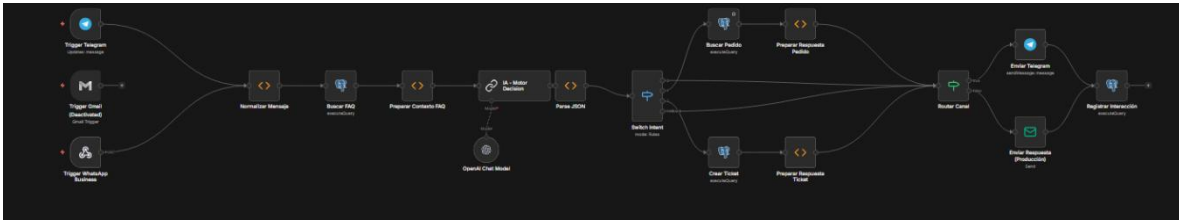


Figura 6: Canvas del Flujo 2 en n8n, mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por intent y el nodo Router Canal que envía la respuesta por el canal de origen.

4.4.3 Prompt de IA

El nodo de GPT-4o-mini recibe un prompt de sistema que define la identidad del asistente, la tarea de clasificación, las cuatro categorías de intención admitidas — enumeradas dentro del esquema de salida como FAQ, ESTADO_PEDIDO, RECLAMO y GENERAL— y el formato estricto de la respuesta: un objeto JSON con los campos intent, order_id, urgente y respuesta. El prompt de la configuración vigente, que es el que produjo el 92,7 % del Capítulo 5, tiene 6338 caracteres y ocho bloques: identidad y tono del asistente; definición de la tarea; formato de salida; reglas de clasificación por categoría, entre ellas cuándo un mensaje debe marcarse como urgente; un bloque rotulado «base de conocimiento», en el que se inserta el contenido completo de la tabla faq_responses, leído en cada llamada, con la instrucción de usar exclusivamente esa información para responder consultas frecuentes; diez reglas críticas, entre ellas la de clasificar como reclamo un mensaje ambiguo entre consulta frecuente y reclamo y la de no revelar que el asistente es una IA (Sección 2.5); siete ejemplos etiquetados de entrada y salida; y el mensaje del cliente, con su nombre y su canal. Opera por lo tanto en régimen few-shot y con inyección de contexto: en cada llamada se inserta la tabla completa, sin seleccionar las entradas pertinentes a la consulta, lo que la distingue de la generación aumentada por recuperación (Sección 2.2.4). La cadena que provee ese contexto son los nodos Buscar FAQ en PostgreSQL y Preparar Contexto FAQ, que sí inciden en la salida del sistema en la configuración medida. El texto íntegro se transcribe en el Anexo H, junto con las variantes que compara la Sección 5.2.4.

Se eligió GPT-4o-mini por su balance entre costo y capacidad: para la clasificación de intenciones en un dominio acotado y la generación de respuestas de soporte no se requiere la potencia completa de GPT-4o. El nodo invoca el modelo por su alias, gpt-4o-mini, que el proveedor puede asociar a distintas versiones a lo largo del tiempo, y no fija la temperatura ni ningún otro parámetro de muestreo: rige por lo tanto el valor por defecto de la API, que es 1. La consecuencia es que un mismo mensaje puede recibir etiquetas distintas en corridas distintas. La Sección 5.2.4 la cuantifica sobre tres repeticiones del corpus: con la configuración vigente, el 98,0 % de los mensajes recibió la misma etiqueta en las tres, y la exactitud de cada repetición varió entre 92,7 % y 93,3 %.

El corpus del Capítulo 5 corrió el 12 de agosto con el prompt de 6338 caracteres descrito arriba, que coincide byte a byte con el del commit f68da9d del repositorio. Las 45 interacciones de Telegram de la Sección 5.2.2 y la ablación E7 corrieron, en cambio, con un prompt reducido de 1032 caracteres, sin base de conocimiento, reglas ni ejemplos (Anexo H). Qué versión del workflow corrió en cada medición, y cómo se reconstruyó, se detalla en el Anexo L.

4.4.4 Métrica TMR

TMR (Tiempo Medio de Respuesta): Tiempo entre la recepción del mensaje del cliente (`received_at`) y el envío de la respuesta (`responded_at`). Se calcula automáticamente en la vista `v_chatbot_response_time`. Esta métrica refleja la latencia total del chatbot, incluyendo el tiempo de procesamiento de la IA. Cabe sobre `received_at` la misma precisión que la Sección 4.3.3 hace sobre el MTTD: la marca no la toma el disparador sino el nodo Normalizar Mensaje, ya dentro del flujo, y se registra sin zona horaria. El TMR mide por lo tanto desde esa escritura hasta el envío de la respuesta, y no incluye el tramo que va desde que el cliente envía el mensaje hasta que el disparador lo recibe.

4.5 Configuración de Grafana

Se configuraron dos dashboards en Grafana con conexión directa a PostgreSQL, uno por cada flujo: “Pipeline Post-Venta — Overview” (6 paneles) y “Chatbot Multicanal — Métricas” (7 paneles). Los paneles se alimentan de las vistas de métricas y, en los casos en que se requiere una agregación específica del panel, de una consulta directa sobre las tablas base. Doce de los trece paneles se restringen a los registros medidos (`data_source = 'measured'`). Siete lo heredan de la vista que consumen: dos del tablero del Flujo 1 —«Órdenes totales», que consume `v_metrics_summary`, y «Órdenes procesadas por día», que consume `v_daily_order_summary`— y cinco del tablero del Flujo 2, que consumen `v_chatbot_corpus`, la cual acota además la población a la ventana temporal de la corrida evaluada. Los otros cinco lo aplican mediante una cláusula explícita en la consulta del propio panel: los tres indicadores de tiempo y la distribución de estados del Flujo 1, y el de tickets abiertos del Flujo 2, que repite esa ventana. El panel de exactitud es un valor constante, según indica la Tabla 4.8. Las figuras de los tableros se capturaron el 19 de agosto de 2026. Las cinco vistas del bloque de métricas operativas llevaban entonces la restricción por procedencia —la sexta, la del corpus, se creó con ella—, aplicada sobre la base en uso con el guion `experiments/E5/vistas_measured.sql`; esa es la definición que transcribe el Anexo A y la que el esquema versionado incorporó después, de modo que la separación entre los registros medidos y los de carga inicial es hoy una propiedad del esquema y no una convención de cada tablero. Ninguna cifra del Capítulo 5 proviene de esas vistas: las métricas reportadas se calculan con consultas propias sobre las corridas identificadas por su prefijo de número de orden, según se detalla en la Sección 3.5.1.

Los paneles configurados en el dashboard y su fuente de datos se listan en la Tabla 4.8.

Tabla 4.8: Paneles de los dashboards de Grafana (Flujo 1: paneles 1 a 6; Flujo 2: paneles 7 a 13).

#	Panel	Tipo	Fuente de datos	Descripción
1	MTTD — Detección (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo promedio de detección de una orden
2	MTTR — Resolución (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo promedio hasta la notificación al cliente
3	End-to- end (s)	Stat	orders (filtrada a la corrida de carga secuencial)	Tiempo total de extremo a extremo
4	Órdenes totales	Stat	v_metrics_summary	Volumen de órdenes medidas
5	Distribución de estados de órdenes	Pie chart	orders	Proporción de órdenes por estado
6	Órdenes procesadas por día	Time series	v_daily_order_summary	Evolución diaria del volumen procesado
7	TMR promedio (s)	Stat	v_chatbot_corpus	Tiempo medio de respuesta del chatbot
8	Interacciones totales	Stat	v_chatbot_corpus	Volumen de interacciones del corpus evaluado
9	Exactitud (rotulado «Precisión (accuracy) » en el panel)	Stat	Valor constante	Accuracy de clasificación del corpus. No se deriva de una consulta SQL: su cálculo requiere las etiquetas de referencia, que no residen en la base de datos operativa. El panel muestra el valor obtenido en el Anexo J.

10	Tickets abiertos	Stat	tickets (filtrada a la ventana de la corrida del corpus)	Tickets generados por la rama de reclamos
11	TMR promedio por intent (s)	Bar chart	v_chatbot_corpus	Desglose del TMR por intención
12	Distribución de intents	Pie chart	v_chatbot_corpus	Proporción de mensajes por intención
13	Interacciones por día y canal	Time series	v_chatbot_corpus	Evolución diaria por canal de entrada

La Figura 7 presenta el dashboard del Flujo 1 con los valores medidos.

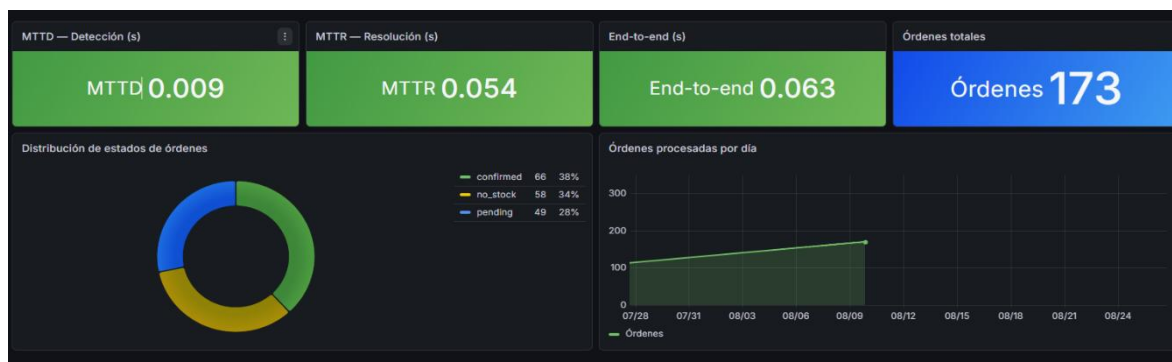


Figura 7: Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1, capturado el 19 de agosto de 2026. Sus seis paneles se restringen a los registros medidos (Sección 4.5). El total de 173 órdenes es el de esa fecha; la Tabla 5.3 informa 174 porque incorpora ORD-AUDIT-ALERT, generada el 26 de agosto al verificar la alerta de stock bajo. La serie diaria tiene un único punto dentro de la ventana visible, las 171 órdenes del 10 de agosto: la línea entra por el borde izquierdo porque interpola desde las dos órdenes de demostración del 2 de julio, anteriores a esa ventana, de modo que los valores intermedios que la línea dibuja no corresponden a ninguna corrida.

4.6 Entorno y evidencia de las pruebas

Las pruebas se ejecutaron sobre una notebook con procesador Intel Core i5 de 11^a generación, 16 GB de RAM DDR4, almacenamiento SSD NVMe de 512 GB y sistema operativo Windows 11 con WSL2 (Ubuntu 22.04). Los cuatro contenedores Docker (n8n, PostgreSQL, Mailpit, Grafana) se levantaron con la configuración por defecto del archivo docker-compose.yml, sin asignación explícita de límites de CPU o memoria. La conectividad a la API de OpenAI se realizó sobre una conexión de fibra óptica de 100 Mbps simétricos. Estas condiciones del entorno deben tenerse en cuenta al interpretar los valores absolutos de MTDD, MTTR y TMR reportados en el

Capítulo 5: los tiempos de procesamiento del Flujo 1 dependen del rendimiento del hardware local y del SGBD; el TMR del Flujo 2 está dominado por la latencia de la API externa y no por el entorno local.

Los correos de confirmación y de aviso de stock insuficiente que genera el Flujo 1 se verifican en la bandeja del captador SMTP (ver Figura 8).

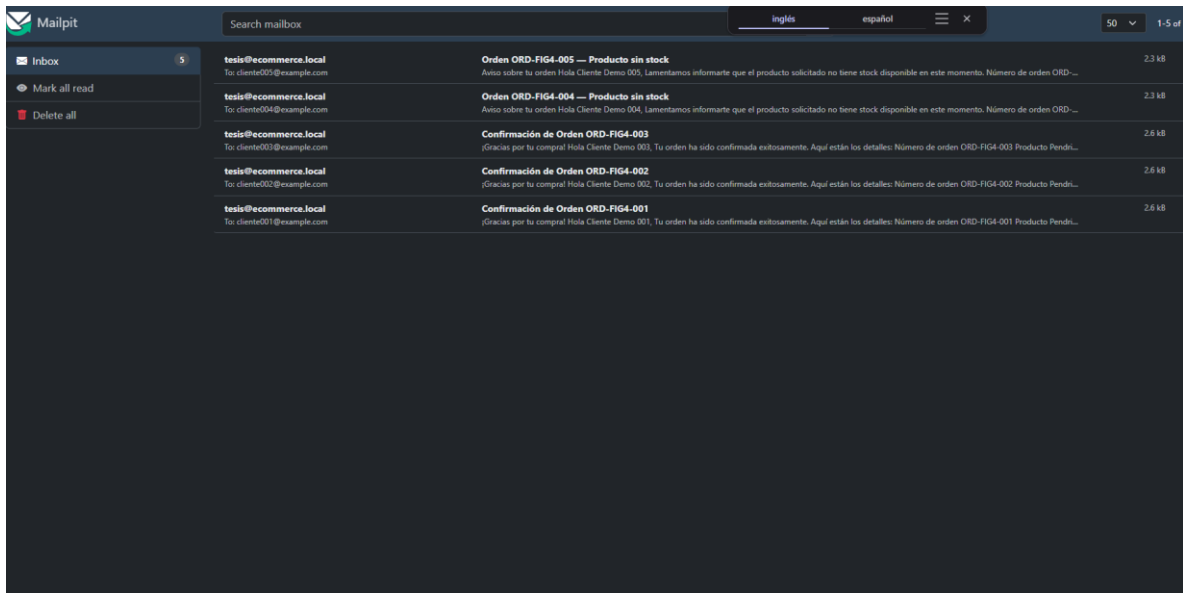


Figura 8: Bandeja de Mailpit con los correos generados por el Flujo 1 durante una corrida de verificación de cinco órdenes (ORD-FIG4-001 a 005), preparada para exhibir ambas ramas de notificación y distinta de la corrida de carga de cincuenta órdenes: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com). Esas cinco órdenes no se conservan en la base de datos y no integran el total de la Tabla 5.3.

CAPÍTULO 5: RESULTADOS Y ANÁLISIS

5.1 Resultados del Flujo 1 — Pipeline de procesamiento de órdenes

5.1.1 Pruebas funcionales

Se ejecutaron seis pruebas funcionales sobre el pipeline de procesamiento de órdenes. En cada caso se verificó la respuesta del webhook, el estado de la base de datos y los artefactos generados —correos y alertas de stock— (ver Tabla 5.1).

Tabla 5.1: Resultados de las pruebas funcionales.

Prueba	Escenario	Resultado
PF-01	Orden exitosa (stock suficiente)	APROBADA — orden confirmada, stock decrementado, email enviado
PF-02	Orden sin stock	APROBADA — orden rechazada con status no_stock, email de aviso
PF-03	Stock bajo (bajo umbral)	APROBADA — orden confirmada + alerta registrada en stock_alerts
PF-04	SKU inválido	NO APROBADA — la orden no se registra, pero no hay rechazo con error controlado: el webhook responde HTTP 200 sin cuerpo y la ejecución termina como exitosa
PF-05	Orden duplicada	PARCIAL — la restricción de unicidad rechaza la orden duplicada, como se esperaba, pero el webhook responde HTTP 200 sin cuerpo y el emisor no se entera del rechazo
PF-06	Apóstrofo en el dato del cliente	APROBADA — con la consulta parametrizada la orden la rechaza la restricción de unicidad y no la sintaxis; la sentencia anterior, con el mismo valor, falla por error de sintaxis

Las tres primeras pruebas se aprobaron: el pipeline actualizó las marcas temporales en orders y las notificaciones se enviaron al destinatario esperado. De las dos pruebas de rechazo, con el criterio de la Tabla 3.4, PF-04 no se aprobó y PF-05 se aprobó en parte: en ambas la orden inválida o duplicada no se registra, pero ninguna produce el rechazo controlado que el emisor necesita para distinguirla de una orden atendida. Se re-ejecutaron el 15 de septiembre para dejar evidencia versionada, que se conserva en experiments/PF junto con el guion, y el mecanismo de cada una es distinto. Las tres primeras no se reejecutaron: su aprobación se apoya en la corrida de carga de 50 órdenes, que recorrió ambas ramas con sus marcas temporales completas, y en la orden ORD-AUDIT-ALERT, con la que se verificó la alerta de stock bajo. Repetirlas escribiría órdenes con la marca de los datos medidos y alteraría los totales de este capítulo. En PF-04, la sentencia que registra la orden toma el producto del catálogo por su SKU; al no encontrarlo no inserta ninguna fila, el flujo se detiene sin llegar a un nodo de respuesta y la ejecución queda registrada como exitosa: el resultado esperado era un rechazo con error controlado, y no lo hay. En PF-05, la restricción de unicidad del número de orden rechaza la inserción, que es lo que la prueba esperaba, y la ejecución termina en error; lo que falta es comunicarlo. En los dos casos el webhook responde HTTP 200 con el cuerpo vacío, de modo que el emisor no puede distinguir un rechazo de una orden atendida: es el mismo patrón que la Sección 5.1.3 documenta bajo concurrencia. PF-06, posterior a las mediciones, verifica las consultas parametrizadas: una orden con un apellido que lleva apóstrofo y un número ya registrado la rechaza la restricción de unicidad, como en PF-05, y no un error de sintaxis, que es lo que le ocurre a la sentencia anterior con el mismo valor.

5.1.2 Métricas de tiempo

Las métricas de tiempo obtenidas para el Flujo 1 se resumen en la Tabla 5.2.

Tabla 5.2: Métricas de tiempo del Flujo 1.

Métrica	Valor sobre las 50 órdenes medidas
MTTD (tiempo de detección)	media 0,009 s · desvío 0,003 s · mediana 0,009 s · IC 95 % [0,008; 0,009] s
MTTR (tiempo de resolución)	media 0,054 s · desvío 0,003 s · mediana 0,054 s · IC 95 % [0,053; 0,055] s
Tiempo total end-to-end	media 0,063 s · desvío 0,005 s · mediana 0,062 s · IC 95 % [0,061; 0,064] s

El tiempo total end-to-end medido fue de 0,063 s en promedio (desvío estándar 0,005 s; mediana 0,062 s; mínimo 0,057 s; máximo 0,095 s; n = 50), lo que significa que desde la recepción de una orden hasta la escritura de la marca de notificación al cliente transcurren menos de siete centésimas de segundo, incluyendo la verificación de stock, la actualización del inventario y el envío del correo al servidor SMTP. El valor máximo observado (0,095 s) corresponde a la primera orden de la

corrida y se atribuye al arranque en frío del motor de flujos; se reporta sin excluirlo de la muestra.

5.1.3 Pruebas de carga y de concurrencia

La corrida de carga secuencial (E1.a) procesó las 50 órdenes enviadas sin un solo error de pipeline y con cobertura completa de las tres marcas temporales: las 50 órdenes registraron `received_at`, `processed_at` y `notified_at`, incluidas las 15 que siguieron la rama de stock insuficiente. Sobre esa población se calculan las métricas de la Tabla 5.2.

Tabla 5.3: Resultados de las pruebas de carga y de concurrencia.

Métrica	Resultado
E1.a — Órdenes enviadas (secuenciales, espaciadas 2 s)	50
E1.a — Órdenes procesadas	50/50 (100 %)
E1.a — Órdenes confirmadas	35 (70 %)
E1.a — Órdenes sin stock	15 (30 %)
E1.a — Errores de pipeline	0
E1.b — Rondas × solicitudes casi simultáneas	6 × 20 = 120 órdenes (dos ejecuciones del guion de 3 rondas)
E1.b — Stock inicial por ronda	5 unidades (PROD-005)
E1.b — Órdenes confirmadas por ronda	5 en las 6 rondas (ninguna sobreventa)
E1.b — Productos con stock negativo	0: la restricción CHECK (stock >= 0) rechaza el descuento que lo produciría
E1.b — Órdenes sin procesar bajo concurrencia	49 de 120 (40,8 %): ejecuciones terminadas en error, respondidas con HTTP 200 y sin aviso
E1.b — MTTD medio bajo concurrencia	0,092 s (±0,025 s; máx. 0,157 s), sobre las 71 órdenes procesadas
E1.b — MTTR medio bajo concurrencia	0,100 s (derivado: end-to-end – MTTD; el desvío no es derivable de las cifras publicadas), sobre las 71 órdenes procesadas
E1.b — End-to-end medio	0,192 s (±0,024 s; máx. 0,260 s), sobre las 71 órdenes

bajo concurrencia	procesadas
Total de órdenes con data_source = 'measured'	174 (50 de E1.a, 120 de E1.b, 3 órdenes de verificación — ORD-DEMO-01, ORD-DEMO-02 y ORD-VERIF-D4— y 1 de la verificación de la alerta de stock bajo, ORD-AUDIT-ALERT)

La prueba de concurrencia (E1.b) arroja dos resultados que deben leerse juntos, porque el segundo explica el primero. El primero es que no hubo sobreventa: en las seis rondas, contra un stock inicial de 5 unidades y con 20 solicitudes disparadas en una ventana de entre 0,4 y 23 milisegundos, el sistema confirmó exactamente 5 órdenes, y ningún producto quedó con stock negativo. Ese resultado no se debe a que el flujo sea atómico. El Flujo 1 lee el stock y lo descuenta en dos sentencias separadas, sin bloqueo entre ambas (Tabla 4.5, nodos 3 y 5), de modo que dos ejecuciones concurrentes pueden leer el mismo stock disponible. Lo que impide la sobreventa es la restricción CHECK (stock >= 0) del esquema (Anexo A): cuando el descuento dejaría el stock en negativo, la base rechaza la sentencia.

El segundo resultado es la contracara del primero. De las 120 órdenes, 49 (el 40,8 %) quedaron registradas en estado pendiente, sin marca de processed_at. Cada una se corresponde con una ejecución del motor de flujos terminada en error: en la segunda ejecución del guion, el conteo de ejecuciones con error creció en cada ronda exactamente en la cantidad de órdenes que quedaron pendientes. El orden de llegada es consistente con que sean las que pasaron la verificación de stock y chocaron contra la restricción al descontarlo: en cada ronda aparecen después de las cinco confirmadas y antes de las rechazadas por falta de stock. El detalle de cada ejecución ya no se conserva, de modo que esa causa se presenta como la más probable y no como verificada. Lo que sí está verificado es la consecuencia: el webhook respondió HTTP 200 a las 120 solicitudes, con el cuerpo vacío en las 49 fallidas, y ninguna orden pasó a estado de error ni generó aviso. El emisor recibió, por lo tanto, la confirmación de recepción de pedidos que el sistema nunca procesó ni notificó. Es una pérdida silenciosa, y en un sistema de órdenes eso constituye una falla de confiabilidad, no una limitación de capacidad.

Dos precisiones acotan las demás cifras del ensayo. La primera es que las latencias bajo concurrencia —MTTD de 0,092 s y extremo a extremo de 0,192 s, frente a 0,009 s y 0,063 s en régimen secuencial— se calculan sobre las 71 órdenes que terminaron y excluyen justamente las que fallaron: tienen un sesgo de supervivencia y no describen el tiempo de las solicitudes que el sistema no completó. La segunda es que la concurrencia efectiva fue menor que la nominal: aunque las 20 solicitudes de cada ronda se dispararon en pocos milisegundos, el motor las registró en la base a lo largo de 84 a 132 milisegundos, de modo que no llegaron las 20 a la vez a la sentencia de descuento. Aun así, la condición de carrera se manifestó en las seis rondas, que corresponden a dos ejecuciones del guion de tres rondas cada una. La corrección de fondo —descontar el stock con una sentencia condicionada o con bloqueo, y registrar como error la orden cuya ejecución falla— se plantea en el Capítulo 7.

5.1.4 Baseline de atención manual

Para dotar de un término de comparación medido, y no estimado, a la afirmación central del trabajo, se cronometró el procesamiento manual de una orden siguiendo el protocolo descrito en la Sección 3.5.5. La Tabla 5.4 presenta los descriptivos por fase y del tiempo total sobre las diez órdenes válidas.

Tabla 5.4: Descriptivos del tiempo de procesamiento manual, por fase y total.

Fase	n	Media	Mediana	Desvío	Mín.	Máx.
T1 — Lectura y verificación	10	7,05 s	7,12 s	2,00 s	4,37 s	11,39 s
T2 — Actualización en la base	10	5,80 s	5,27 s	1,67 s	4,45 s	9,09 s
T3 — Redacción de la notificación	10	36,27 s	34,91 s	6,55 s	30,45 s	53,67 s
Total por orden	10	49,13 s	46,37 s	8,15 s	42,75 s	69,80 s

El tiempo marginal de procesamiento manual de una orden resultó de 49,13 s, con un intervalo de confianza al 95 % de [43,30 s; 54,95 s] calculado por la distribución t de Student con nueve grados de libertad. El coeficiente de variación fue del 16,6 %, una dispersión moderada que no debe leerse como estabilidad: como se detalla más abajo, los tiempos descienden a lo largo de la serie por efecto de aprendizaje, y parte de esa dispersión es tendencia y no ruido. Conforme se estableció en la Sección 3.5.5, este valor mide el costo de procesar una orden adicional y no la latencia percibida por el cliente.

El reparto del esfuerzo entre fases es en sí mismo un hallazgo. La redacción de la notificación al cliente concentra el 73,8 % del tiempo total (36,27 s de 49,13 s), mientras que la lectura y verificación de stock insume el 14,3 % (7,05 s) y la actualización de la base el 11,8 % restante (5,80 s). Es decir que el grueso del costo del procesamiento manual no está en la consulta ni en la escritura de datos, que son las operaciones que un operador humano ejecuta con rapidez, sino en la comunicación con el cliente. Desagregado por rama, el tiempo fue de 50,96 s en las siete órdenes con stock disponible y de 44,84 s en las tres sin stock, diferencia atribuible a que la plantilla de confirmación exige cargar el importe total y la de rechazo no.

El valor admite un contraste con un segundo instrumento, distinto del cronómetro y aplicado a la misma sesión. Las marcas temporales que el operador fue escribiendo en la base durante la medición permiten calcular el intervalo entre notificaciones de órdenes consecutivas, que es la misma magnitud marginal medida por otra vía: ese

cálculo arroja 51,28 s de media (desvío 5,22 s). La diferencia de 2,15 s respecto de los 49,13 s cronometrados corresponde a los intervalos muertos entre terminar una orden y comenzar la siguiente, que el cronómetro no contabiliza y el reloj de la base sí. Los dos instrumentos coinciden dentro del 4 %. Es acuerdo entre dos instrumentos sobre una misma sesión y con un mismo operador, no una medición independiente: descarta un error de registro, pero no los sesgos que ambos comparten (Sección 3.6.2).

El registro en la base aporta además un resultado complementario sobre el encolamiento. Como las doce órdenes ingresaron en un mismo lote (Sección 3.5.5), el tiempo transcurrido entre la recepción del lote y la notificación de cada orden crece de forma monótona: 72,4 s para la primera atendida y 533,9 s —ocho minutos y catorce segundos— para la última, con una media de 313,37 s y un desvío de 154,57 s (Tabla I.2). Esta cifra no es una propiedad del proceso manual sino del tamaño del lote elegido, y que por lo tanto no se la utiliza como término de comparación: un lote mayor la aumentaría sin que el operador trabajase más lento. Su valor es ilustrativo y apunta a la razón de fondo por la que la automatización importa en este dominio: el pipeline automatizado procesó 50 órdenes consecutivas sin que el tiempo de la última se degradara respecto de la primera, mientras que en el proceso manual la espera del último cliente creció un factor de siete frente a la del primero.

Corresponden dos advertencias sobre la interpretación de este valor. La primera surge de un control previsto en el protocolo: la correlación de Pearson entre el orden de ejecución y el tiempo empleado fue de $r = -0,693$, magnitud que indica que el efecto de aprendizaje siguió actuando dentro de la serie válida. En consecuencia, la media de 49,13 s subestima el tiempo que emplearía un operador no entrenado: respecto de ese operador, el baseline es un piso. Esto no convierte al factor de mejora en una cota, porque el término automatizado tiene un sesgo en la dirección contraria (Sección 5.4). La segunda es que la medición se realizó sobre un único operador, lo que impide estimar la variabilidad entre personas; es una de las limitaciones de la Sección 5.5.

Por último, este valor no incluye la latencia de detección, esto es, el tiempo que una orden permanece a la espera antes de que un operador advierta su llegada. Esa magnitud no es observable sin convertirla en un parámetro elegido por los autores, y por lo tanto no se suma al número medido. A título ilustrativo puede señalarse que, si un operador revisara la bandeja cada quince minutos, la espera media esperable sería de 7,5 minutos, valor que por sí solo excede en dos órdenes de magnitud al tiempo de procesamiento cronometrado. Se consigna como escenario declarado y no como medición.

5.2 Resultados del Flujo 2 — Chatbot (canal simulado formato WhatsApp y canal Telegram real)

5.2.1 Pruebas funcionales del chatbot

Se ejecutaron seis pruebas funcionales sobre el Flujo 2, con el mismo criterio que las del Flujo 1: se verificó la respuesta entregada por el canal, el estado de la base de datos y los artefactos generados. Su diseño se detalla en la Tabla 3.5; se ejecutaron el 17 de septiembre sobre el workflow vigente, por el canal simulado, y la evidencia se versiona en experiments/PF junto con el guion. De las cinco primeras, cuatro se aprobaron. PC-01 se clasificó como FAQ y recibió los medios de pago que registra la base. PC-02 se clasificó como ESTADO_PEDIDO, quedó asociada al pedido citado y la respuesta entregada incluye su estado. PC-03 se clasificó como RECLAMO y generó un ticket de prioridad normal, y PC-04, redactada en términos de urgencia, se clasificó como RECLAMO con la marca de urgencia y generó un ticket de prioridad urgente; en los dos casos la respuesta informa el número de caso. PC-05 no se aprobó: la consulta sobre tiendas físicas se clasificó como FAQ y no como GENERAL, aunque la respuesta, que informa que la tienda es solo online, es la de la base. Lo que esa prueba deja al descubierto es su propio criterio. El mensaje 44 del corpus, «tienen local físico para retirar?», tiene FAQ como etiqueta de referencia; la definición de FAQ con que se etiquetó es «pregunta general sobre la tienda» (Sección 3.5.3), y la base de conocimiento tiene una entrada sobre el retiro en local. Por la convención de etiquetado del propio trabajo, entonces, la clasificación del sistema es la correcta y el resultado esperado de la Tabla 3.5 es el que está mal. La prueba se informa igual como no aprobada, porque su criterio se fijó antes de ejecutarla y reescribirlo con el resultado a la vista invalidaría el control; lo que corresponde es declarar la incoherencia, que se arrastra a la celda de OE5 de la Tabla 6.1. En los cinco casos la interacción quedó registrada con sus dos marcas temporales y la respuesta llegó al capturador de correo. La prueba dejó además un hallazgo: los tickets se crean sin el identificador de la interacción que los originó, de modo que la clave foránea que el esquema prevé entre ambas tablas queda vacía. La Tabla 3.5 cita para PC-02 un pedido de la carga inicial (Anexo L). PC-06, posterior a las mediciones y ejecutada sobre el workflow ya parametrizado, envió un reclamo en inglés con dos contracciones y dos comas: se clasificó como RECLAMO con marca de urgencia, generó un ticket con el texto íntegro y el cliente recibió la respuesta. Se aprobó.

Estas pruebas no deben confundirse con la evaluación del clasificador. Son pruebas funcionales de un caso por intención: verifican que el flujo se recorra completo y que cada rama se active, no que la clasificación sea correcta en general ni que el contenido de la respuesta sea adecuado. Lo primero se mide sobre el corpus de 150 mensajes en la Sección 5.2.3; lo segundo se aborda, con los alcances que allí se precisan, en la Sección 5.2.5.

5.2.2 Tiempos de respuesta (TMR)

Se enviaron 150 mensajes de prueba distribuidos entre los cuatro intents posibles, simulando consultas reales de clientes con errores ortográficos, abreviaciones y mensajes ambiguos (ver Tabla 5.5).

Tabla 5.5: Tiempo medio de respuesta por intención sobre el corpus evaluado.

Intent	Mensajes clasificados	TMR promedio	TMR mín.	TMR máx.	Desvío est.	No escalada
FAQ	45	1,54 s	1,19 s	2,91 s	±0,33 s	100%
RECLAMO	40	1,54 s	1,31 s	1,94 s	±0,15 s	0% (escala a ticket por diseño)
ESTADO_PEDIDO	36	1,45 s	1,22 s	1,91 s	±0,15 s	100%
GENERAL	29	1,28 s	1,04 s	1,78 s	±0,18 s	100%
Total	150	1,47 s	1,04 s	2,91 s	±0,25 s	~73%

TMR promedio: 1,47 s sobre los 150 mensajes del corpus. Este valor sostiene H2a (TMR < 10 s) con amplio margen, también por categoría: la media más alta es de 1,54 s.

Tasa de no escalada: 73,3 % (110 de 150 mensajes). Si se excluyen los reclamos — que escalan a ticket por diseño del sistema y no por falla del clasificador—, la tasa asciende al 100 %: el chatbot respondió sin derivar a un operador humano la totalidad de los mensajes que no eran reclamos. El indicador mide que el flujo se completó sin escalar: el sistema clasificó el mensaje, generó una respuesta y la entregó por el canal de origen. No mide que esa respuesta fuera correcta. La corrección del contenido entregado al cliente no quedó establecida en este trabajo, y la reserva es material en las consultas de tipo FAQ: aunque la base de conocimiento de la tienda se inyecta en el prompt (Sección 4.4.3), la Sección 5.2.5 muestra que el modelo afirma en algunas respuestas datos que esa base no contiene. Llamar «tasa de resolución» a este indicador sobreestimaría lo medido; se lo denomina en consecuencia tasa de no escalada.

Tickets generados: 40 tickets de soporte, generados a partir de los mensajes clasificados como RECLAMO por el modelo (40 predicciones), no de los 42 reclamos reales; el enrutamiento opera sobre la predicción del clasificador.

Validación del segundo canal (Telegram): para verificar que la arquitectura multicanal es real y no solo declarada, el canal de Telegram se implementó de extremo a extremo —recepción por webhook, clasificación con GPT-4o-mini, consulta a la base de datos y respuesta por el mismo canal— y se validó sobre una

muestra de 45 interacciones reales. A diferencia del canal simulado —que adopta el formato de la WhatsApp Cloud API en la recepción y entrega por SMTP a un capturador local, sin tocar la red de WhatsApp en ningún extremo—, Telegram opera contra la API real, por lo que su TMR incluye la latencia de red del tramo de salida, el envío de la respuesta por la API. No incluye el de entrada: la marca de recepción la escribe el nodo Normalizar Mensaje, ya dentro del flujo, de modo que el tiempo que media entre el envío del cliente y el arribo del mensaje al motor de flujos queda fuera de lo medido (Sección 4.4.4) (ver Tabla 5.6).

Tabla 5.6: Tiempo medio de respuesta por intención en el canal Telegram.

Intent	Mensajes	TMR promedio	TMR mín.	TMR máx.
ESTADO_PEDIDO	15	3,32 s	2,29 s	5,44 s
FAQ	8	3,38 s	2,52 s	5,75 s
GENERAL	12	2,84 s	2,18 s	5,96 s
RECLAMO	10	2,72 s	2,45 s	2,93 s
Total	45	3,07 s	2,18 s	5,96 s

El TMR global del canal Telegram fue de 3,07 s —superior al del canal simulado (1,47 s), pero holgadamente por debajo del umbral de 10 s—, de modo que H2a se sostiene también sobre un canal real. La validación tiene un alcance acotado: las 45 interacciones de Telegram son adicionales al corpus de 150 mensajes y no un subconjunto suyo, y no cuentan con etiquetas de referencia. En consecuencia, la validación del segundo canal cubre la latencia de respuesta pero no la exactitud de clasificación, que se reporta únicamente sobre el corpus etiquetado. El canal de correo (Gmail) se mantiene como diseño (Capítulo 7): su mecanismo de consulta periódica (polling) impone un piso de latencia de aproximadamente un minuto, incompatible con la métrica de segundos. Las interacciones de Telegram, además, corrieron con el prompt reducido de 1032 caracteres y no con el de la configuración principal (Sección 4.4.3): la validación del segundo canal cubre la latencia y el recorrido completo del flujo, no el desempeño de la configuración medida sobre el corpus.

La diferencia de 1,6 s entre ambos canales no acota el componente atribuible a la red, porque las dos series no son apareadas: provienen de conjuntos distintos —150 mensajes del corpus etiquetado y 45 interacciones de Telegram, adicionales y no un subconjunto— que difieren en composición de intenciones, en longitud de los mensajes, en momento de ejecución y en el prompt con que corrieron. Los tres primeros factores inciden sobre el tiempo de inferencia; el cuarto apenas, porque en el diseño factorial prompts de 1154 y de 6338 caracteres produjeron tiempos medios de 1,55 s y 1,57 s (Sección 5.2.4). La diferencia se reporta por lo tanto como observación entre dos condiciones de medición y no como descomposición del tiempo; aislar el componente de red exigiría enviar un mismo subconjunto de mensajes por ambos canales (Capítulo 7).

5.2.3 Exactitud de clasificación de intents

Se evaluó la exactitud de clasificación comparando el intent asignado por GPT-4o-mini contra el intent esperado para los 150 mensajes de prueba. El intent esperado se etiquetó de forma ciega respecto a la salida del modelo (ver Sección 3.5.3) (ver Tabla 5.7).

Tabla 5.7: Exhaustividad (recall) de clasificación por intención.

Intent	Mensajes	Clasificados correctamente	Clasificados incorrectamente	Exhaustividad (recall)
FAQ	48	44	4	91,7%
RECLAMO	42	38	4	90,5%
ESTADO_PEDIDO	35	32	3	91,4%
GENERAL	25	25	0	100%
Total	150	139	11	92,7%

Matriz de confusión (intent real vs. predicho por el modelo) (ver Tabla 5.8):

Tabla 5.8: Matriz de confusión del clasificador.

Real \ Predicho	ESTADO_PEDIDO	FAQ	GENERAL	RECLAMO	Total real
ESTADO_PEDIDO	32	1	1	1	35
FAQ	1	44	2	1	48
GENERAL	0	0	25	0	25
RECLAMO	3	0	1	38	42
Total predicho	36	45	29	40	150

Precisión, exhaustividad (recall) y F1 por clase (ver Tabla 5.9):

Tabla 5.9: Precisión, exhaustividad y F1 por clase.

Clase	Precisión	Recall	F1
ESTADO_PEDIDO	88,9%	91,4%	0,901
FAQ	97,8%	91,7%	0,946
GENERAL	86,2%	100%	0,926
RECLAMO	95,0%	90,5%	0,927

Global	—	92,7%	—
--------	---	-------	---

Accuracy global: 92,7% (139/150), lo que supera el umbral del 85 % establecido en H2b. Los 11 errores se concentraron en fronteras semánticas entre categorías — principalmente RECLAMO asignado a ESTADO_PEDIDO y FAQ/GENERAL entre sí—, coherente con mensajes cortos y ambiguos.

Dos corridas, y por qué se informa la segunda. El corpus se ejecutó dos veces ese día. La primera, de las 11:09, recibió 150 respuestas HTTP 200 y dejó 115 filas en la tabla de interacciones: se perdieron sin aviso las 35 que el modelo había enrutado a ESTADO_PEDIDO. El nodo que busca el pedido no emitía ningún ítem cuando el número citado no existía en la tabla de órdenes, y en un orquestador de flujo de datos un nodo que no emite detiene todo el subgrafo que sigue: esos clientes no recibieron respuesta, no se escribió su registro y la ejecución terminó marcada como exitosa. La respuesta de resguardo que el flujo ya tenía escrita para el pedido inexistente era inalcanzable por construcción, lo que muestra que en esta arquitectura la programación defensiva dentro del nodo no alcanza: el resguardo tiene que garantizar que el paso anterior emita siempre al menos un ítem. La consulta se reescribió para que parta de una fila sintética y traiga el pedido por combinación externa, de modo que devuelve siempre exactamente una fila, y la corrida definitiva, de las 20:45, se ejecutó sobre el sistema corregido y escribió las 150. La corrida omitida no era menos favorable: acertó 140 de 150 (93,3 %) frente a 139 (92,7 %) de la definitiva, y ambas asignan la misma etiqueta a 145 de los 150 mensajes. Se informa la segunda porque es la única medida sobre un sistema que respondió a los 150 mensajes; el defecto se registra en el Anexo L y se retoma en las Secciones 5.5 (f) y (h), 6.3 y 6.4, y la comparación completa entre ambas corridas se versiona en `experiments/E2/resultados/corrida1_vs_corrida2.txt`.

Conciliación de la corrida informada. Las 150 respuestas se registraron con una etiqueta del vocabulario admitido y ninguna es el mensaje de disculpa que el flujo entrega cuando no consigue interpretar la salida del modelo (Sección 4.4.1): no hubo errores de parseo ni etiquetas fuera de vocabulario, de modo que los 150 mensajes del denominador corresponden a clasificaciones efectivas. El control no es una formalidad: un fallo de parseo se registra como GENERAL y podría contarse como acierto, y una etiqueta inexistente impide escribir la fila y saldría del denominador. En los dos casos la exactitud quedaría sobrestimada.

Acuerdo entre anotadores: sobre la submuestra de 50 mensajes etiquetada por el evaluador independiente (protocolo en §3.5.3), el acuerdo observado con el ground truth fue de 47 coincidencias sobre 50 ($P_o = 0,940$), con un coeficiente κ de Cohen de 0,919, valor que corresponde a un acuerdo casi perfecto en la escala de Landis y Koch (1977). Los tres desacuerdos se concentran en fronteras semánticas reales entre categorías —“me pueden ayudar?” (GENERAL/FAQ), “mi pedido no llega y ya pasaron los 5 días” (RECLAMO/FAQ) y una consulta de stock referida a un pedido ya realizado (FAQ/ESTADO_PEDIDO)—, es decir en los mismos casos ambiguos donde

se concentran los errores del clasificador. Este resultado indica que el ground truth no responde a un criterio idiosincrásico del equipo.

La categoría con mayor precisión fue FAQ (97,8 %), seguida de RECLAMO (95,0 %), ESTADO_PEDIDO (88,9 %) y GENERAL (86,2 %). El patrón se invierte al observar la exhaustividad: GENERAL alcanza el 100 % (25 de 25), es decir que el modelo no dejó sin identificar ningún mensaje que perteneciera realmente a esa categoría, pero incorporó a ella cuatro mensajes provenientes de otras clases. Esa asimetría — exhaustividad máxima junto con la precisión más baja del conjunto— es consistente con el carácter abierto de la categoría: al no estar acotada por patrones específicos, GENERAL opera de hecho como clase residual y absorbe los mensajes que el modelo no consigue asignar con confianza a las otras tres (ver Figura 9).

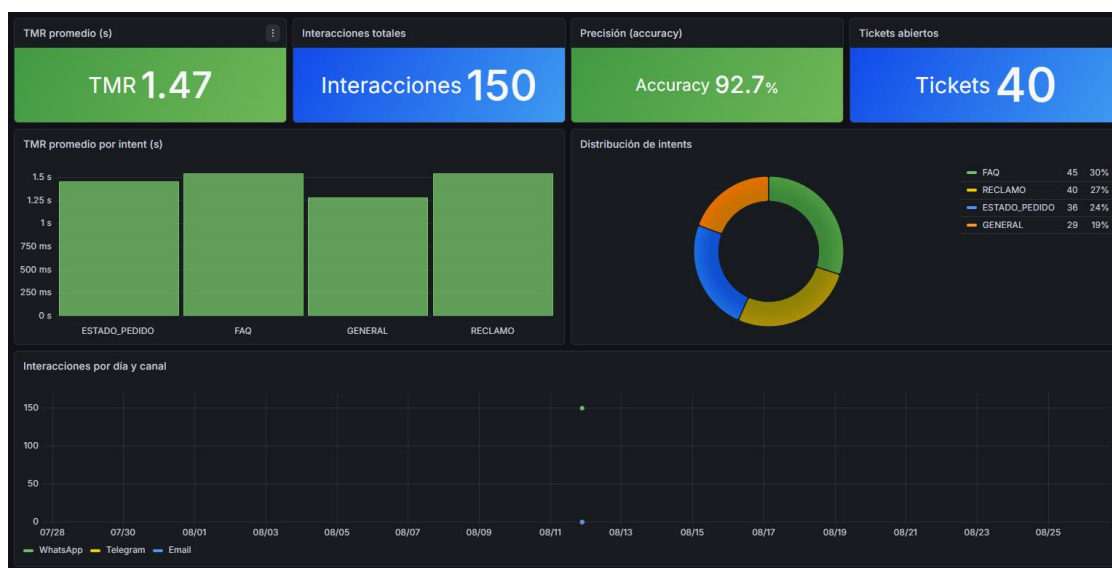


Figura 9: Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la exactitud de clasificación y la distribución de intenciones sobre el corpus evaluado. El panel de interacciones por día y canal se alimenta de la vista del corpus, que está acotada a la ventana temporal de esa corrida: por eso muestra únicamente las 150 interacciones del canal simulado y no las 45 de Telegram, que se ejecutaron fuera de esa ventana y se reportan en la Tabla 5.6. La serie de correo aparece en cero porque el canal está previsto en el esquema y no implementado.

5.2.4 Aporte de la base de conocimiento y de las reglas y ejemplos

El 92,7 % de la sección anterior se obtuvo con un prompt que combina la base de conocimiento de la tienda con reglas de decisión y siete ejemplos etiquetados. Para estimar cuánto aporta cada uno se ejecutó el diseño factorial de la Sección 3.5.6: cuatro condiciones, tres repeticiones de cada una, sobre el mismo corpus y las mismas etiquetas de referencia. Los doce bloques primarios resultaron utilizables, uno de ellos tras un reintento. En la tercera repetición de la condición sin reglas ni ejemplos el modelo devolvió una categoría inexistente, y la regla fijada de antemano

manda volver a correr el bloque; el reintento devolvió otra categoría inexistente. Recién entonces —antes de los tres bloques finales y sin haber visto ninguna exactitud— se fijó el desvío que cuenta esas etiquetas como error de clasificación y usa el primer intento utilizable, que es el que ocupa la posición preregistrada (Sección 3.5.6). El reintento se informa como análisis de sensibilidad: si se lo usa en lugar del bloque primario, la exactitud de esa condición pasa de 78,7 % a 79,3 % y ninguna conclusión cambia. La exhaustividad por clase y la exactitud de cada condición se sintetizan en la Tabla 5.10.

Tabla 5.10: Exhaustividad por clase y exactitud total por mayoría de tres repeticiones, y exactitud media por repetición, por condición del diseño factorial.

Clase (mensajes)	C1 · base, reglas y ejemplos	C2 · reglas y ejemplos, sin base	C3 · base, sin reglas ni ejemplos	C4 · sin base, reglas ni ejemplos
FAQ (48)	45 (93,8 %)	45 (93,8 %)	31 (64,6 %)	19 (39,6 %)
ESTADO_PEDIDO (35)	32 (91,4 %)	32 (91,4 %)	31 (88,6 %)	34 (97,1 %)
RECLAMO (42)	38 (90,5 %)	39 (92,9 %)	31 (73,8 %)	33 (78,6 %)
GENERAL (25)	25 (100 %)	25 (100 %)	25 (100 %)	25 (100 %)
Total (150) · IC 95 %	140 (93,3 %) [88,2 %; 96,3 %]	141 (94,0 %) [89,0 %; 96,8 %]	118 (78,7 %) [71,4 %; 84,5 %]	111 (74,0 %) [66,4 %; 80,4 %]
Media por repetición · IC 95 %	93,1 % [89,1 %; 97,2 %]	94,2 % [90,5 %; 97,9 %]	76,7 % [70,4 %; 83,0 %]	73,3 % [66,5 %; 80,2 %]

La exactitud por mayoría es de 93,3 % con base, reglas y ejemplos (C1) y de 94,0 % sin la base (C2), y cae a 78,7 % sin las reglas ni los ejemplos (C3) y a 74,0 % sin ninguno de los dos factores (C4). La variabilidad entre repeticiones es baja: las exactitudes de cada repetición tienen un desvío de 0,4 puntos en C1 y en C2, de 1,3 en C3 y de 0,7 en C4, y los mensajes que recibieron la misma etiqueta en las tres repeticiones son el 98,0 %, el 98,7 %, el 85,3 % y el 92,0 %, respectivamente. La exactitud media por repetición, que estima el desempeño de una llamada única como la que hace el sistema desplegado, es de 93,1 % en C1 (IC 95 % [89,1 %; 97,2 %]), 94,2 % en C2 ([90,5 %; 97,9 %]), 76,7 % en C3 ([70,4 %; 83,0 %]) y 73,3 % en C4 ([66,5 %; 80,2 %]). La mayoría sobrestima la llamada única en 2,0 puntos en C3, la condición menos estable, y difiere en menos de un punto en las demás. Con la media por repetición, los efectos principales son de 18,7 puntos para las reglas y los ejemplos y de 1,1 para la base, en la misma dirección, y el límite inferior de C1 también supera el 85 %. Los intervalos se calculan con la t de Student sobre la proporción de aciertos de cada mensaje en sus tres repeticiones, a partir de las predicciones versionadas; el guion comprueba además que ese intervalo caiga dentro del más amplio compatible con los recuentos publicados, que puede calcularse sin acceso a la base (experiments/E8/media_por_repeticion_e8.py). De los

seis contrastes por pares, cuatro son significativos tras el ajuste de Holm, todos con $p < 0,001$: los que enfrentan una condición con reglas y ejemplos a una sin ellos. Los otros dos no lo son: C1 frente a C2, con una sola discordancia ($p = 1,0$), y C3 frente a C4, con 18 y 11 discordancias (p ajustado = 0,53). El tiempo medio de respuesta apenas varía entre condiciones, de 1,53 s a 1,60 s, aunque el prompt va de 1154 a 6338 caracteres.

En términos del diseño factorial, las reglas y los ejemplos aportan 17,3 puntos porcentuales en promedio: 14,7 con la base presente y 20,0 sin ella. La base de conocimiento aporta 2,0 puntos en promedio: -0,7 cuando las reglas y los ejemplos están presentes y +4,7 cuando faltan, y ninguna de las dos diferencias es significativa. La interacción, de -5,3 puntos, indica que la base solo ayuda algo cuando faltan las reglas y los ejemplos, y que con ellos presentes no agrega nada a la clasificación. El desglose de la Tabla 5.10 localiza el efecto. La exhaustividad de FAQ es de 93,8 % en C1 y en C2, y cae a 64,6 % en C3 y a 39,6 % en C4. Los mensajes de FAQ que se pierden migran a la categoría residual: por repetición, en promedio, 1,3 y 1,0 mensajes de FAQ se clasifican como GENERAL en C1 y en C2, frente a 17,3 en C3 y 26,3 en C4. RECLAMO sigue el mismo patrón con menor magnitud, y ESTADO_PEDIDO y GENERAL apenas varían.

La categoría FAQ no se vuelve decidible porque el modelo conozca la política de la tienda, sino porque tiene una definición operativa: las reglas de clasificación enumeran los temas que cuentan como consulta frecuente —pagos, envíos, devoluciones, garantía, facturación— y los ejemplos muestran casos resueltos. Sin esa definición, el modelo deriva a la categoría residual las consultas que no reconoce como frecuentes, y la base de conocimiento, por sí sola, recupera solo una parte de ellas. El diseño no separa cuál de los tres bloques del factor R produce el efecto; la enumeración de temas de las reglas de clasificación es la explicación más directa, pero queda como hipótesis. Para la clasificación, lo que la tienda tiene que escribir y mantener son las reglas y los ejemplos. La base conserva su función en la redacción de las respuestas a las consultas frecuentes, que es la que evalúa la Sección 5.2.5 y no la que mide este experimento.

Una observación posterior al diseño se consigna sin valor confirmatorio. El prompt reducido de E7, que tampoco tenía base, reglas ni ejemplos, obtuvo 86,0 % en una única corrida, doce puntos más que C4. Ambos prompts difieren esencialmente en el bloque que presenta el mensaje del cliente con su nombre y su canal, que E7 no tenía y C4 sí. Si la diferencia se sostuviera, la forma de presentar el mensaje también incidiría en la clasificación, como documentan Sclar et al. (2024) para cambios de formato del prompt en otros modelos. El experimento no fue diseñado para establecerlo, las dos mediciones distan cuatro días y E7 es una sola corrida, de modo que queda como hipótesis por verificar.

Sobre H2b el resultado es inequívoco con la regla fijada en la Sección 1.4.2. En la configuración vigente (C1) la exactitud por mayoría es de 93,3 %, con un intervalo de

Wilson de [88,2 %; 96,3 %] cuyo límite inferior supera el 85 %. La réplica frente a la corrida del 12 de agosto no muestra diferencia: 92,7 % frente a 93,3 %, con un mensaje que solo acertó aquella corrida y dos que solo acertó C1 (p exacto = 1,0). El resultado de la Sección 5.2.3 se reproduce, por lo tanto, un mes después y con tres repeticiones. C2 también cumpliría la regla, con un límite inferior de 89,0 %; C3 y C4 no, con 71,4 % y 66,4 %. Excluir el único mensaje del corpus muy parecido a un ejemplo del prompt no cambia ninguna conclusión: la exactitud de C1 y de C2 queda igual, y la de C3 y C4 sube medio punto.

El experimento dejó además un hallazgo de robustez que no buscaba. En la condición sin reglas ni ejemplos, el modelo devolvió dos veces una categoría que no existe, ante dos mensajes cuya referencia es RECLAMO: «CAMBIO» en el bloque primario y «DEVOLUCION» en el reintento, que no integra la medida primaria. Esos mensajes siguieron la rama por defecto del flujo: el cliente recibió una respuesta, pero no se generó ticket, y la restricción de integridad de la tabla de interacciones rechazó la escritura del registro. La ejecución terminó en error sin que nada lo advirtiera, y el caso solo se detectó porque el guion de verificación concilia los envíos contra las filas escritas. Con el prompt vigente no ocurrió en 450 clasificaciones, pero el mecanismo existe: la integridad de la base impide registrar una etiqueta inválida y, a la vez, convierte un error de clasificación en un reclamo sin ticket y sin registro. Se retoma en las Secciones 5.5 y 7.1.

Los manifiestos de cada bloque, la evidencia por ejecución, los registros de envío, los cuatro prompts y los guiones de construcción, ejecución y análisis se versionan en el repositorio bajo experiments/E8. La ablación E7, que este diseño reemplaza, se conserva bajo experiments/E7.

5.2.5 Contenido de las respuestas de tipo FAQ

El contenido de las 45 respuestas clasificadas como FAQ se evaluó por las dos vías que describe la Sección 3.5.7. La primera no produjo un resultado que pueda reportarse como proporción de respuestas correctas, y se informa igualmente, junto con la razón.

Evaluación por jueces. Se completaron seis corridas: dos de un evaluador y cuatro del otro. Tres superaron el control de tiempos; las otras tres, todas del mismo evaluador, registraron medianas de 4,0, 5,0 y 6,3 segundos por respuesta y se excluyeron. Las corridas válidas permiten dos comparaciones entre evaluadores distintos. En la primera coincidieron en 27 de las 45 respuestas ($P_o = 0,600$; $P_e = 0,520$; $\kappa = 0,167$) y en la segunda en 31 ($P_o = 0,689$; $P_e = 0,516$; $\kappa = 0,358$), valores que en la escala de Landis y Koch (1977) corresponden a un acuerdo leve y a uno aceptable, ambos por debajo del umbral de acuerdo moderado (0,41). El evaluador con dos corridas válidas repitió su propio juicio en 30 de las 45 respuestas ($\kappa = 0,318$). Cabe una salvedad sobre la corrida válida del otro evaluador: es su tercera lectura de las mismas 45 respuestas, después de dos corridas invalidadas por tiempo el mismo

día, de modo que la exposición previa al material puede incidir en el acuerdo que se informa.

Que un mismo evaluador cambie de nivel en una de cada tres respuestas indica que la dispersión no se explica solo por diferencias de criterio entre personas: la rúbrica, tal como está formulada, no produce un juicio reproducible. La causa es estructural, porque sus tres niveles no son mutuamente excluyentes. Una respuesta que reproduce una entrada de la base y le agrega un dato que la base no contiene cabe a la vez en el primer nivel, porque coincide con la política, y en el tercero, porque afirma algo que la tienda no sostiene; una política genérica que la tienda no tiene cabe a la vez en el segundo y en el tercero. La rúbrica no establece cuál prevalece. Con ese acuerdo no se reporta ninguna proporción de respuestas correctas, y la corrección del instrumento se plantea en el Capítulo 7.

Verificación automática. De las 45 respuestas, 18 afirman al menos un dato concreto, y entre todas suman 21. De esos datos, 18 figuran en la base de conocimiento, 1 repite una cifra que el propio cliente había escrito y 2 no figuran en ninguna entrada. Los dos pertenecen a respuestas distintas, de modo que 2 de las 18 respuestas con datos concretos —el 11,1 %, con un intervalo de confianza de Wilson al 95 % de 3,1 % a 32,8 %— afirman al menos uno que la base no respalda. Son un horario de atención «de lunes a viernes de 9 a 18 hs», cuando la base no fija horario alguno y, sobre la atención, solo indica que el asistente virtual asiste las veinticuatro horas; y un plazo de reintegro de las devoluciones «entre 5 y 10 días hábiles», tema sobre el que la base no dice nada. El detalle, dato por dato, se transcribe en la Tabla K.1 del Anexo K.

Los dos casos responden consultas sobre temas que la base no trata: el horario de atención y el plazo de reintegro. Los dos casos tienen además una explicación en el propio prompt (Anexo H). Las reglas de clasificación enumeran «horarios de atención» entre los temas de consulta frecuente, pero la base no tiene ninguna entrada sobre horarios; y el bloque de la base instruye al modelo a decir que va a consultar con el equipo cuando la pregunta no está cubierta, instrucción que en los dos casos no cumplió. La inyección del contexto no impide, entonces, que el modelo complete con datos plausibles cuando la consulta cae fuera de lo que la base contempla; es la alucinación que la Sección 5.5 (a) enuncia como riesgo general, observada aquí sobre respuestas efectivamente entregadas por el sistema, y consistente con la evidencia de que ni la recuperación ni la inyección de contexto la eliminan (Magesh et al., 2025; Sección 2.4.3). La cifra debe leerse como una cota inferior, por los dos límites que declara la Sección 3.5.7. El segundo tiene un ejemplo en la propia muestra: una respuesta atribuye a la confirmación de un pago por transferencia un plazo de 24 horas hábiles que la base contiene, pero referido al retiro de un producto en devolución, y la verificación lo cuenta como respaldado porque no controla el contexto en que se usa el dato. Contado ese caso, la proporción sería de 3 de 18 —el 16,7 %, con un intervalo de Wilson de 5,8 % a 39,2 %—, y es la cifra que el resumen, la Sección 5.5 (c) y el Capítulo 6 informan junto a la que arroja la regla automática.

La misma verificación se aplicó al propio prompt, con el guion `verificar_ejemplos_prompt.py`. Los siete ejemplos de respuesta del Anexo H contienen tres datos concretos: dos figuran en la base —entre 3 y 7 días hábiles de envío y 12 meses de garantía— y uno no: el ejemplo de envío a Córdoba afirma que los envíos tardan «entre 3 y 5 días hábiles», cuando la base fija entre 3 y 7 para el resto del país. El prompt enseña así un dato ausente de la base, contra su propia instrucción de usar exclusivamente esa información. El plazo no figura en ninguna de las 45 respuestas de tipo FAQ (Tabla K.1), y la consulta del corpus más parecida al ejemplo, sobre el costo del envío a Córdoba, se respondió con la información de la base sobre el cálculo del costo en el checkout. Otros dos ejemplos no contienen cifras, pero comprometen a la tienda con algo que la base no respalda: resolver un reclamo «hoy mismo» y gestionar «el cambio o reembolso de inmediato». La verificación automática no los alcanza, por el primer límite que declara la Sección 3.5.7. La corrección de los ejemplos se recomienda en la Sección 7.1.

Las seis corridas de los evaluadores, incluidas las invalidadas por tiempo, el guion de verificación con sus pruebas y la salida de ambos análisis se versionan en el repositorio bajo `experiments/E6`.

5.3 Contrastación de hipótesis

El contraste de cada hipótesis contra su criterio de aceptación se sintetiza en la Tabla 5.11.

Tabla 5.11: Contrastación de hipótesis.

Hipótesis	Criterio	Momento en que se fijó el criterio	Resultado obtenido	Veredicto
H1: Reducción respecto del proceso manual (objetivo de estimación)	Factor de reducción con IC 95 %; como referencia, límite inferior $\geq 10\times$; end-to-end < 30 s como criterio operativo	Umbral de $10\times$: después de medir, con el factor ya conocido. Criterio operativo de 30 s: antes de las mediciones del Capítulo 5.	0,063 s frente a 49,13 s del baseline manual (factor $\approx 780\times$; IC 95 % por el teorema de Fieller: $686\times$ a $875\times$). U de Mann-Whitney = 0 con separación completa de ambas series ($p = 2,7 \times 10^{-11}$); t de Welch = 19,05, gl = 9,0, $p = 1,4 \times 10^{-8}$. Muy por debajo del criterio operativo de 30 s.	SE SOSTIENE (régimen secuencial)
H2a: $TMR < 10$	TMR	Umbral de 10	1,47 s en total; entre	SE SOSTIENE

s en cada categoría	promedio < 10 s en cada una de las cuatro categorías	s para FAQ, estado de pedido y consultas generales: antes de las mediciones. Extensión a las cuatro categorías: después.	1,28 s y 1,54 s por categoría (Tabla 5.5)	
H2b: Exactitud (accuracy) de clasificación $\geq 85\%$	Límite inferior del IC 95 % de Wilson $\geq 85\%$	Umbral del 85 %: antes de las mediciones. Regla del límite inferior: después de la corrida del corpus y antes de medir en el diseño factorial.	92,7 % (139/150) en la corrida del corpus, IC 95 % [87,3 %; 95,9 %]; 93,3 % por mayoría de tres repeticiones en el diseño factorial, IC 95 % [88,2 %; 96,3 %]. En ambos casos el límite inferior supera el umbral. Sin reglas ni ejemplos: 78,7 % y 74,0 %, con límites inferiores de 71,4 % y 66,4 % (Sección 5.2.4).	SE SOSTIENE con la configuración vigente; no se sostiene sin reglas ni ejemplos

Las tres hipótesis se sostienen con los criterios fijados en la Sección 1.4.2, pero no todos esos criterios se fijaron antes de medir, y la Tabla 5.11 indica cuándo se fijó cada uno. Los umbrales de 30 s, 10 s y 85 % figuran en la versión del trabajo registrada el 27 de abril de 2026 en el repositorio (commit 9dc7b44), anterior a las mediciones del Capítulo 5. Se fijaron después, con el resultado conocido, el umbral de un orden de magnitud de H1, la extensión de H2a a las cuatro categorías y la regla del límite inferior de H2b en la corrida del corpus. Para esos criterios, «se sostiene» describe el resultado frente a un criterio y no constituye una confirmación. La contrastación prospectiva en sentido estricto es la réplica de H2b en el diseño factorial, cuya regla se versionó antes de medir. Con esa salvedad, los alcances son los siguientes:

- H1, como objetivo de estimación: el tiempo marginal de procesamiento de un orden en el pipeline (0,063 s; $n = 50$) resulta unas 780 veces menor que el del procesamiento manual, medido en 49,13 s por orden, con un intervalo al 95 % de

686× a 875× por el teorema de Fieller. Ambos términos miden la misma magnitud, el tiempo que insume una orden adicional (Sección 3.5.5). El límite inferior supera la referencia de un orden de magnitud, y el tiempo end-to-end representa el 0,2 % del umbral operativo de 30 segundos. El alcance del factor se discute en la Sección 5.4.

- H2a se sostiene con amplio margen: el TMR del corpus (1,47 s) representa el 14,7 % del umbral de 10 s, y ninguna de las cuatro categorías supera 1,54 s de media (Tabla 5.5); el del canal Telegram real (3,07 s), medido con el prompt reducido, representa el 30,7 %.

- H2b se sostiene con la regla fijada: la exactitud de 92,7 % (139/150) tiene un intervalo de confianza de Wilson al 95 % de [87,3 %; 95,9 %], con el límite inferior por encima del umbral del 85 %, y el diseño factorial la reproduce en 93,3 % por mayoría de tres repeticiones, con un límite inferior de 88,2 %. El procedimiento de cálculo se detalla en el Anexo J. El resultado vale para la configuración vigente del prompt: sin sus reglas y sus ejemplos, la exactitud cae a 78,7 % con la base de conocimiento y a 74,0 % sin ella, con límites inferiores por debajo del umbral (Sección 5.2.4).

Contraste estadístico de H1. Afirmar que una serie es sustancialmente más rápida que otra describe una diferencia de magnitud, no una inferencia: para sostener que esa diferencia no proviene del azar del muestreo hace falta una prueba de contraste, y se la reporta a continuación. Las dos series están completamente separadas: el máximo de la serie automatizada (0,095 s sobre $n = 50$) es inferior al mínimo de la serie manual (42,754 s sobre $n = 10$), de modo que ningún par de observaciones se cruza. La U de Mann-Whitney vale por lo tanto 0, con un valor p exacto bilateral de $2/C(60, 10) = 2,7 \times 10^{-11}$ y un tamaño del efecto rango-biserial de 1,000, el máximo posible. La t de Welch sobre los descriptivos publicados arroja $t = 19,05$ con 9,0 grados de libertad y $p = 1,4 \times 10^{-8}$ (d de Cohen = 15,3). Ambos contrastes se calculan íntegramente a partir de las cifras de las Tablas 5.2, 5.4 y I.1, de modo que son reproducibles sin acceso a los datos crudos. La prueba establece que la diferencia observada no es atribuible al azar del muestreo, pero no resuelve la asimetría de constructo entre ambos términos —el alcance instrumentado de cada uno— que se discute en la Sección 5.4. La significación estadística confirma que la diferencia existe; su interpretación sigue estando acotada por lo que cada instrumento midió.

5.4 Análisis comparativo y discusión

Para el procesamiento de órdenes, los resultados sostienen la premisa central del trabajo: la automatización reduce de manera sustancial el tiempo operativo. La reducción del tiempo marginal de procesamiento de una orden es de un factor aproximado de 780 veces respecto del baseline manual medido en la Sección 5.1.4 (49,13 s $\rightarrow$ 0,063 s), con un intervalo de confianza al 95 % de 686× a 875×. Ese

intervalo se obtiene por el teorema de Fieller, que es el procedimiento exacto para el cociente de dos medias y propaga la incertidumbre de ambos términos. El alcance de esa comparación se acota en tres puntos. Primero, ambos términos miden la misma magnitud: el tiempo que insume procesar una orden adicional. No se comparan contra el tiempo de espera observado en la cola manual (313,37 s de media), porque esa cifra depende del tamaño del lote y no del proceso, y utilizarla habría inflado el factor hasta cerca de 5.000×. Segundo, el factor no es una cota en ningún sentido, porque los sesgos conocidos de sus dos términos operan en direcciones opuestas. El término automatizado proviene de un entorno de laboratorio en el que la notificación se entrega a un capturador SMTP alojado en el mismo host, sin tránsito de correo real ni latencia de servicios de terceros: ese sesgo achica el denominador y agranda el factor. El término manual se midió sobre un operador que ya conocía el procedimiento y trabajaba con una plantilla fija, dos decisiones deliberadamente conservadoras que achican el numerador y con él el factor; y la correlación negativa entre el orden de ejecución y el tiempo empleado (Sección 5.1.4) indica que un operador no entrenado tardaría más. En sentido contrario opera el sesgo de expectativa que declara la Sección 3.6.2: el operador pertenecía al equipo que formuló H1 y conocía el sentido de la comparación, de modo que pudo trabajar más lento de lo que trabajaría fuera de la medición, lo que agrandaría el numerador y con él el factor. Como no hay forma de establecer cuál de los dos sesgos pesa más, el factor se lee como una diferencia de casi tres órdenes de magnitud bajo las condiciones declaradas, y no como una cota ni como el valor esperable en un despliegue productivo. Tercero, el alcance instrumentado no es idéntico: los 0,063 s cubren el intervalo `received_at` → `notified_at`, es decir escrituras sobre la base local, mientras que el `baseline manual` cubre la tarea humana completa de lectura, verificación, actualización y redacción. La significación práctica de la reducción se aprecia mejor en tiempo absoluto que en el factor: automatizar ahorra unos 49 segundos de trabajo por orden, es decir unos 41 minutos cada 50 órdenes, que es el volumen de la corrida medida. Frente a la latencia de detección de un proceso manual ese ahorro es la parte menor: con el escenario declarado en la Sección 5.1.4, de una revisión de la bandeja cada quince minutos, la espera media sería de 7,5 minutos, de modo que para el cliente la ganancia principal estaría en eliminar la espera y no en acelerar el procesamiento. En atención al cliente, el chatbot responde en 1,47 s en promedio sobre el corpus evaluado y en 3,07 s sobre el canal Telegram real.

El modelo GPT-4o-mini resultó adecuado para la clasificación de intenciones en un dominio acotado de cuatro categorías. El corpus incluye deliberadamente errores ortográficos, abreviaciones y mensajes con varias preguntas, pero este trabajo no desagrega la exactitud por esos rasgos y no afirma nada específico sobre ellos. El dato relevante no es el 92,7 % por sí solo sino lo que el diseño factorial de la Sección 5.2.4 permite atribuir: con reglas de decisión y ejemplos, la exactitud se ubica entre 93 % y 94 %, con o sin la base de conocimiento; sin ellos, entre 74 % y 79 %. Los 17,3 puntos del efecto de las reglas y los ejemplos son atribuibles al prompt y no a la

muestra, porque las cuatro condiciones corrieron sobre el mismo corpus y las mismas etiquetas, y la diferencia se repite en cada una de las tres repeticiones; valen, sin embargo, para este formato de prompt. El 74,0 % de la condición sin base, sin reglas y sin ejemplos no es una referencia del desempeño alcanzable sin ajuste fino y sin contexto: el prompt de E7, igualmente sin base, reglas ni ejemplos, pero con el mensaje del cliente presentado de otra forma, obtuvo 86,0 % (Sección 5.2.4). Ambas cifras describen formatos particulares de prompt; lo que el diseño mide con validez es la diferencia entre condiciones de un mismo formato, que en esta configuración cuantifica lo que se gana por escribir y mantener las reglas y los ejemplos, y no la base de conocimiento.

5.5 Limitaciones observadas

(a) Dependencia de servicios externos y limitaciones de los LLMs: el Flujo 2 depende de la API de OpenAI; una caída del servicio inhabilita la clasificación de intents y la generación de respuestas. Adicionalmente, los modelos de lenguaje como GPT-4o-mini presentan limitaciones conocidas en entornos de producción: (i) **alucinación** — el modelo puede generar respuestas plausibles pero factualmente incorrectas, especialmente ante consultas fuera del dominio de entrenamiento (Huang et al., 2025); (ii) **prompt injection** — un usuario malicioso podría formular un mensaje diseñado para alterar las instrucciones del sistema y obtener respuestas no autorizadas (Perez y Ribeiro, 2022). Estas limitaciones justifican la supervisión humana periódica del chatbot en entornos de producción y la implementación de filtros de validación sobre las respuestas generadas. El Flujo 1 opera de forma independiente y no se ve afectado por estas limitaciones.

(b) Desempeño por subcategoría: ninguna clase queda por debajo del 85 % en precisión ni en exhaustividad (Tabla 5.9), pero GENERAL, definida como categoría residual, concentra los casos de frontera con FAQ: combina la exhaustividad más alta con la precisión más baja. La Sección 5.2.4 muestra que esa frontera es la más sensible a las reglas y los ejemplos del prompt, y la mitigación más directa es ampliar los ejemplos con casos de frontera entre GENERAL y FAQ, que hoy no están representados, sin requerir ajuste fino del modelo.

(c) Corrección del contenido de las respuestas: el trabajo no establece qué proporción de las respuestas entregadas es correcta. La evaluación por jueces no alcanzó un acuerdo suficiente (κ entre 0,167 y 0,358) y la verificación automática solo alcanza a los datos concretos, sobre los cuales encontró 2 de 18 respuestas con algún dato ausente de la base de conocimiento, y un tercer caso fuera de su alcance que llevaría la proporción a 3 de 18 (Sección 5.2.5). El primer resultado es un límite del instrumento de juicio y no del sistema; el segundo es una cota inferior de lo que las respuestas agregan a la política de la tienda.

(d) Entorno de prueba local: las métricas de tiempo se obtuvieron en un entorno Docker sobre hardware de desarrollo (notebook). En un servidor de producción con

mayor capacidad de cómputo y menor latencia de red, los valores de MTTD y MTTR podrían ser inferiores; sin embargo, el TMR está dominado por la latencia de la llamada de inferencia a la API de OpenAI, que no depende del hardware local: el rango observado en este trabajo va de 1,47 s a 3,07 s según el canal.

(e) Tamaño y origen del conjunto de prueba: el conjunto de 150 mensajes, redactado con asistencia de un modelo de lenguaje a partir de mensajes escritos por el equipo (Sección 3.5.3), cubre los escenarios más comunes pero no la variabilidad completa de un entorno real. La exactitud del 92,7 % podría variar con mensajes de otro dominio o redactados por clientes reales, y con reglas y ejemplos escritos por otras personas: el modelo no fue entrenado ni ajustado para este dominio, de modo que su desempeño depende del prompt y del corpus sobre los que se midió.

(f) Ventana temporal de la corrida del chatbot: las 150 llamadas del corpus se ejecutaron dentro de un intervalo de sesenta minutos de un mismo día, según acota la propia vista que aísla la población (Anexo A). La decisión es coherente con la mitigación declarada en la Sección 3.6.4 —acotar la ventana reduce la variabilidad del servicio externo dentro de la medición— pero tiene un costo: el desvío de $\pm 0,25$ s que acompaña al tiempo medio de respuesta describe la dispersión dentro de esa hora y no la variabilidad del proveedor de inferencia entre días y franjas horarias, que es justamente la fuente de incertidumbre que la amenaza a la validez identifica. El valor reportado debe leerse en consecuencia como el desempeño en una ventana favorable y no como el esperable a lo largo del día. Las dos corridas del corpus de ese mismo día, separadas por nueve horas, lo muestran con una comparación directa: el tiempo medio de respuesta fue de 2,42 s a las 11:09 y de 1,47 s a las 20:45, un 39 % más bajo, con la salvedad de que la media de la primera se calcula sobre las 115 interacciones que llegó a registrar (Sección 5.2.3). La diferencia entre dos corridas del mismo sistema en un mismo día es del orden de la que separa a los dos canales de la Tabla 5.6.

(g) Ausencia de grupo de control y de diseño experimental comparativo: el contraste de H1 se ejecutó y se reporta al cierre de la Sección 5.3 —U de Mann-Whitney sobre las dos series y t de Welch sobre sus descriptivos—, de modo que la diferencia observada no es atribuible al azar del muestreo. Lo que ese contraste no resuelve es de otro orden, y se separa en tres puntos. Primero, el baseline manual se midió sobre un único operador: la prueba compara dos series de tiempos, no dos poblaciones de operadores, y por lo tanto no permite estimar la variabilidad entre personas ni descartar el efecto del evaluador. Segundo, no hubo asignación al azar a condiciones ni grupo de control; las dos series provienen de dos procedimientos distintos aplicados a una tarea equivalente, no de un experimento controlado, de modo que la atribución causal de la diferencia a la automatización descansa en el diseño del caso y no en la aleatorización. Tercero, y como se precisa en la propia Sección 5.3, la significación estadística confirma que la diferencia existe pero no resuelve la asimetría de constructo entre ambos términos: cada instrumento cubre un alcance distinto. Una comparación inferencial en sentido pleno exigiría varios

operadores independientes y un diseño experimental que excede el alcance declarado de este trabajo, y queda planteada como línea futura en el Capítulo 7.

(h) Pérdida silenciosa de interacciones: cuando el modelo devuelve una categoría fuera del vocabulario admitido, el mensaje sigue la rama por defecto del flujo, el cliente recibe una respuesta sin que se genere ticket y la restricción de integridad de la tabla de interacciones rechaza el registro, sin que nada lo advierta. No ocurrió con el prompt vigente en 450 clasificaciones, pero sí dos veces sin reglas ni ejemplos (Sección 5.2.4). La forma más severa de este patrón se observó en la primera corrida del corpus, y afectó a la rama ESTADO_PEDIDO completa: cuando el número de pedido citado no existía, el nodo que lo busca no emitía ningún ítem y el subgrafo siguiente no se ejecutaba, de modo que el 23 % de los mensajes quedó sin respuesta y sin registro con la ejecución marcada como exitosa (Sección 5.2.3). Se corrigió antes de la corrida que se informa. El mismo patrón aparece en el Flujo 1: las órdenes perdidas bajo concurrencia y las rechazadas por un producto inexistente o un número de orden duplicado reciben HTTP 200 sin cuerpo (Secciones 5.1.1 y 5.1.3).

(i) Variabilidad y versión del modelo: la temperatura no se fijó, de modo que rige el valor por defecto de la API, que es 1, y el modelo se invoca por un alias que el proveedor puede asociar a otra versión. El diseño factorial acota la primera fuente de variación —la exactitud de la configuración vigente varió entre 92,7 % y 93,3 % en tres repeticiones— pero no la segunda: una réplica futura podría correr sobre otra versión del modelo sin que el sistema lo registre.

(j) Construcción de las consultas y superficie de inyección: los nodos de base de datos de ambos flujos armaban sus sentencias por interpolación: insertaban los valores recibidos dentro del texto de la consulta, sin parámetros. Eso tiene dos consecuencias. La primera es de robustez: un valor legítimo que contenga un apóstrofo —un apellido O’Brien en el pedido, una contracción en un reclamo escrito en inglés— rompe la sentencia, la ejecución termina ahí y el caso se suma a las pérdidas silenciosas del punto (h). La segunda es de seguridad: un valor preparado para cerrar la comilla altera la sentencia, y llega al sistema por dos vías sin control previo, el cuerpo del webhook de órdenes, que no exige autenticación, y el mensaje del cliente, del que el modelo extrae el número de pedido. Ninguna de las dos se manifestó en las corridas medidas, porque ni el corpus ni las órdenes de prueba contienen apóstrofes, pero el defecto pertenece al artefacto medido y estaba registrado como tal en el repositorio del trabajo. Las diez consultas de los cuatro flujos se parametrizaron después de cerrar las mediciones, de modo que el artefacto publicado difiere del medido en ese punto y solo en ese (Anexo L); las pruebas PF-06, sobre el Flujo 1, y PC-06, sobre el Flujo 2, dejan constancia del comportamiento con un apóstrofo antes y después del cambio. Una comprobación del camino completo del Flujo 1 con las consultas parametrizadas —registro, verificación y descuento del stock, confirmación y notificación, con las tres marcas temporales escritas— muestra además que el cambio no alteró el comportamiento (experiments/PF/verificar_camino_completo.py); su orden de prueba se borra al

terminar y no integra los totales del Capítulo 5. La recomendación correspondiente está en la Sección 7.1.

CAPÍTULO 6: CONCLUSIONES

6.1 Respuesta a la pregunta de investigación

La presente investigación planteó la siguiente pregunta: ¿En qué medida un pipeline de automatización orquestado con n8n reduce el tiempo de procesamiento de órdenes del ciclo post-venta de un e-commerce simulado respecto del procesamiento manual, medido a través de las métricas MTTD (tiempo de detección y procesamiento de órdenes) y MTTR (tiempo de resolución y notificación), y con qué tiempo de respuesta (TMR) y exactitud de clasificación atiende las consultas de los clientes un chatbot integrado al mismo sistema?

Para el procesamiento de órdenes, los resultados muestran una reducción sustancial: el tiempo marginal de procesamiento de una orden en el pipeline (0,063 s end-to-end; $n = 50$) es aproximadamente 780 veces menor que el del procesamiento manual, cronometrado en 49,13 s por orden sobre diez órdenes (Sección 3.5.5) y contrastado con un segundo instrumento sobre la misma sesión, las marcas temporales de la base, que arrojan 51,28 s para la misma magnitud. Ese factor se lee como una diferencia de casi tres órdenes de magnitud en condiciones de laboratorio, no como una cota (Sección 5.4). La reducción vale en régimen secuencial; bajo concurrencia, el 40,8 % de las órdenes se perdió sin aviso (Sección 5.1.3). La atención conversacional no se compara contra un proceso manual, porque no se midió uno: el chatbot respondió en 1,47 s en promedio sobre el corpus evaluado y clasificó con una exactitud del 92,7 %, que el diseño factorial atribuye a las reglas y los ejemplos del prompt. La disponibilidad ininterrumpida es una propiedad esperable de la arquitectura —el sistema no depende de un operador humano para responder— pero no fue medida: no se ejecutó ensayo de disponibilidad ni se observó el servicio durante una ventana prolongada, de modo que no se la reporta como resultado. Las tres hipótesis de trabajo se sostienen con los criterios fijados, con los alcances que precisa la Sección 5.3: H1 en régimen secuencial y como comparación de tiempos marginales de procesamiento, H2a en los dos canales medidos, y H2b con la configuración vigente del prompt y no sin sus reglas y ejemplos. Solo la réplica de H2b en el diseño factorial se contrastó con una regla fijada antes de medir; el umbral de un orden de magnitud de H1 y la extensión de H2a a las cuatro categorías se fijaron con el resultado conocido, y para ellas el veredicto describe el resultado frente al criterio, sin valor de confirmación.

6.2 Cumplimiento de objetivos específicos

El grado de cumplimiento de cada objetivo específico se detalla en la Tabla 6.1.

Tabla 6.1: Cumplimiento de objetivos específicos.

Obj.	Enunciado	Resultado	Estado
------	-----------	-----------	--------

OE1	Pipeline de órdenes: end-to-end < 30 s y reducción respecto del proceso manual	Reducción respecto del procesamiento manual: factor $\approx 780\times$ (IC 95 %: $686\times$ a $875\times$; Sección 5.3). MTTD: 0,009 s / MTTR: 0,054 s / Total: 0,063 s (n = 50, corrida E1.a). 50/50 órdenes procesadas sin errores. Bajo concurrencia (E1.b, 120 órdenes) no hubo sobreventa, porque la restricción CHECK (stock ≥ 0) del esquema rechazó los descuentos que la habrían producido; pero 49 de 120 órdenes (40,8 %) quedaron sin procesar, con respuesta HTTP 200 y sin aviso, lo que constituye una falla de confiabilidad (Sección 5.1.3). El objetivo se da por cumplido en régimen secuencial y no en régimen concurrente.	CUMPLIDO (régimen secuencial)
OE2	Chatbot: TMR < 10 s y exactitud (accuracy) $\geq 85\%$	TMR: 1,47 s en el canal simulado vía SMTP local (n = 150) y 3,07 s en el canal Telegram real (n = 45), este último con el prompt reducido. Exactitud de clasificación con la configuración vigente del prompt: 92,7 % sobre el corpus etiquetado (IC 95 %: 87,3 % a 95,9 %) y 93,3 % por mayoría de tres repeticiones en el	CUMPLIDO (configuración vigente, dos canales)

		diseño factorial (IC 95 %: 88,2 % a 96,3 %). Sin reglas ni ejemplos la exactitud cae por debajo del umbral (Sección 5.2.4). Dos canales implementados y validados; Gmail queda como diseño (§7).	
OE3	Schema PostgreSQL con las tablas, vistas e índices necesarios	7 tablas (products, orders, order_items, interactions, tickets, faq_responses y stock_alerts), 6 vistas de métricas y 12 índices de rendimiento sobre las columnas de filtrado y de unión. El objetivo se formuló en términos cualitativos —las tablas, vistas e índices necesarios para instrumentar el ciclo— y el esquema desplegado los provee: MTDD, MTTR y TMR se calculan íntegramente desde las vistas, sin cómputo externo. Se consigna que order_items está en el esquema pero ningún nodo del pipeline la escribe: el sistema implementado carga órdenes mono-producto (Sección 4.2.1).	CUMPLIDO
OE4	Dashboards Grafana con MTDD, MTTR y TMR	13 paneles distribuidos en dos tableros: 6 para el pipeline de órdenes y 7 para el chatbot, con conexión directa a las vistas de PostgreSQL.	CUMPLIDO
OE5	Validación mediante	12 pruebas funcionales	CUMPLIDO EN

	pruebas funcionales con datos simulados	(6 por flujo, dos de ellas posteriores a las mediciones), con su diseño en las Tablas 3.4 y 3.5, sus resultados en las Secciones 5.1.1 y 5.2.1 y evidencia versionada; 1 corrida de carga secuencial de 50 órdenes y 1 prueba de concurrencia de 6 rondas × 20 solicitudes casi simultáneas, con datos crudos y manifiesto de ejecución versionados. Tres pruebas no alcanzaron el resultado esperado: PF-04 no se aprobó y PF-05 se aprobó en parte, porque ambas responden HTTP 200 sin cuerpo en lugar de un rechazo controlado (Sección 5.1.1), y PC-05 no se aprobó, porque la consulta sobre tiendas físicas se clasificó como FAQ y no como GENERAL (Sección 5.2.1); en esta última el criterio de la prueba es incoherente con la convención de etiquetado del corpus, y se mantiene el resultado porque el criterio se fijó antes de ejecutarla.	PARTE (PF-04 y PC-05 no aprobadas; PF-05 parcial)
--	---	---	--

De los cinco objetivos específicos, OE3 y OE4 se cumplieron sin reservas, OE1 y OE2 se cumplieron con el alcance que precisan sus celdas y OE5 se cumplió solo en parte. OE3 y OE4 se formularon en términos cualitativos en la Sección 1.5.2 —las tablas, vistas de métricas e índices necesarios para instrumentar el ciclo, y los dashboards

para visualizar MTTD, MTTR y TMR— y el sistema desplegado los satisface: siete tablas, seis vistas y doce índices sostienen el cálculo de las tres métricas sin cómputo externo, y trece paneles repartidos en dos tableros las visualizan a partir de consultas sobre la base, con excepción del panel de exactitud, que muestra un valor constante (Tabla 4.8). No se los contrasta contra una cifra comprometida de antemano porque los objetivos no la fijaron. OE1 se cumple en régimen secuencial: bajo concurrencia la restricción del esquema impide la sobreventa, pero el pipeline pierde sin aviso el 40,8 % de las órdenes, según se consigna en su propia celda y se analiza en la Sección 5.1.3. OE2 se cumplió sobre los dos canales efectivamente implementados y medidos, quedando el canal de correo como diseño documentado en el Capítulo 7. OE5 se cumplió solo en parte: la validación se ejecutó completa, pero mostró que el Flujo 1 no rechaza de manera controlada un producto inexistente ni un número de orden duplicado, de modo que PF-04 no se aprobó y PF-05 se aprobó en parte (Sección 5.1.1), y que en el Flujo 2 PC-05 no se aprobó (Sección 5.2.1).

6.3 Conclusiones sustantivas

Sobre la viabilidad de n8n como orquestador: n8n demostró ser una plataforma viable para la automatización de procesos de negocio de complejidad media en el contexto de PyMEs. La capacidad de diseñar workflows visualmente, combinada con la posibilidad de ejecutar JavaScript en nodos *Function* y la integración nativa con PostgreSQL, SMTP y la API de OpenAI, permitió implementar ambos flujos con una cantidad acotada de código: la orquestación se definió visualmente, pero el Flujo 2 requiere cinco nodos Function con JavaScript, y los dos flujos incluyen sentencias SQL escritas a mano en sus nodos de base de datos. Que esto reduzca la barrera técnica de adopción para empresas sin equipos de desarrollo dedicados es plausible, pero no fue medido: el trabajo no evaluó la puesta en marcha ni el mantenimiento por parte de personal no técnico. La plataforma, además, no protege por sí sola de las fallas silenciosas: la pérdida de órdenes bajo concurrencia (Sección 5.1.3), la de registros ante una etiqueta inválida (Sección 5.2.4) y la del 23 % de los mensajes de la primera corrida del corpus —toda la rama ESTADO_PEDIDO—, cuando un nodo no emitió ningún ítem y el subgrafo siguiente no llegó a ejecutarse (Sección 5.2.3), ocurrieron sin que el motor de flujos las advirtiera —una con la ejecución en error y las otras dos con la ejecución en estado exitoso— y detectarlas exigió instrumentación propia. No es un rasgo aislado de este sistema: en más de 6.000 flujos de n8n publicados, los mecanismos explícitos de confiabilidad son relativamente poco comunes (Tang et al., 2026).

Sobre la efectividad de GPT-4o-mini en atención al cliente: el modelo alcanzó una exactitud del 92,7 % en la clasificación de intenciones sin ajuste fino, y el diseño factorial la reprodujo un mes después en 93,3 % por mayoría de tres repeticiones. Lo que el diseño permite atribuir, para la configuración estudiada, es qué produce ese número. Las reglas de decisión y los ejemplos etiquetados del prompt explican 17,3 puntos de exactitud; la base de conocimiento de la tienda, 2,0 puntos en promedio y ninguno cuando las reglas y los ejemplos están presentes. La diferencia se concentra

en la categoría FAQ, cuya exhaustividad cae de 93,8 % a 64,6 % cuando faltan las reglas y los ejemplos, aun con la base presente, y a 39,6 % cuando falta también la base, porque sus mensajes pasan a clasificarse como consulta general. En esta configuración, lo que una PyME tendría que escribir y mantener para que el sistema clasifique bien es la definición operativa de sus categorías —qué temas cuentan como consulta frecuente y qué casos resueltos lo ilustran—, no su base de preguntas frecuentes. La base sigue siendo la fuente de las respuestas, y la Sección 5.2.5 muestra que, aun con ella inyectada, el modelo completa datos que la base no contiene.

Tres reservas impiden generalizar esa conclusión más allá de la configuración estudiada. La primera es que parte de las reglas codifica convenciones de desempate del mismo equipo que etiquetó el corpus: la regla crítica 5 manda clasificar como reclamo un mensaje ambiguo entre consulta frecuente y reclamo. Una parte de los 17,3 puntos puede medir, entonces, que el modelo adopta la convención del equipo, además de que clasifique mejor; el acuerdo con el evaluador independiente, que etiquetó sin esas reglas (Sección 3.5.3), acota ese riesgo pero no lo descarta. La segunda es la sensibilidad al formato: el prompt de E7, también sin base, reglas ni ejemplos, obtuvo 86,0 %, doce puntos más que C4, y difiere de él sobre todo en la forma de presentar el mensaje del cliente (Sección 5.2.4), de modo que el efecto medido vale para este formato de prompt y su magnitud podría cambiar con otro (Sclar et al., 2024). La tercera es de alcance: un solo modelo, una sola familia de prompts y un corpus de 150 mensajes redactado con asistencia de un modelo de lenguaje (Sección 3.5.3). Por esas razones la conclusión se presenta como hipótesis a replicar en otras configuraciones, y no como recomendación general.

Sobre la arquitectura Docker Compose: la infraestructura basada en cuatro contenedores está diseñada para ser reproducible. El entorno se levanta con un único comando (`docker compose up -d`), las imágenes están fijadas por versión explícita, los datos persisten entre reinicios mediante volúmenes nombrados, y el esquema, los flujos y los scripts de medición están versionados en el repositorio. No se realizó, no obstante, un ensayo de reproducción con un evaluador externo: el tiempo efectivo de puesta en marcha por un tercero no fue medido y por lo tanto no se afirma.

Sobre las métricas MTTD, MTTR y TMR: su adaptación del dominio ITIL al ciclo post-venta de e-commerce tiene validez estructural pero no semántica. Es estructuralmente válida porque cada métrica se operacionaliza como la diferencia entre dos marcas temporales observables, registradas automáticamente en PostgreSQL, que permiten calcularla y visualizarla sin intervención manual. No lo es semánticamente, porque el nombre promete más que el instrumento: como se discute a continuación, un MTTD de 0,009 segundos no mide la detección de nada.

Sobre el retorno al marco teórico: las tres nociones que el Capítulo 2 puso a trabajar admiten una lectura a la luz de lo medido. La primera es la distinción de la Sección 2.1.1 entre automatizar un proceso y gestionarlo: el sistema construido lo automatiza —el ciclo se ejecuta sin intervención humana y queda instrumentado— pero no monitorea ni controla su ejecución, porque no incorpora control de admisión, política de reintentos ni gestión del caso de excepción, y la prueba de concurrencia lo mostró con precisión: 40,8 % de las órdenes quedaron sin procesar sin que ningún mecanismo lo advirtiera. La segunda es la dimensión de capacidad de respuesta (responsiveness) del modelo SERVQUAL (Zeithaml et al., 1990), que el TMR operacionaliza: los 1,47 s del canal simulado y los 3,07 s de Telegram saturan esa dimensión, pero SERVQUAL mide cinco y este trabajo instrumentó una sola. La fiabilidad y la empatía —que dependen de que la respuesta sea correcta y pertinente, no de que sea rápida— quedaron fuera de la medición, y es exactamente allí donde se ubica el riesgo declarado en la Sección 5.2.2 sobre el contenido de las respuestas de tipo FAQ, que la Sección 5.2.5 dimensiona solo en parte. La tercera es la transferibilidad de MTDD y MTTR discutida en la Sección 2.1.3: la transferencia se sostuvo a nivel estructural, como se anticipó, pero la medición confirmó también su límite semántico. Un MTDD de 0,009 segundos no informa sobre detección de nada, porque no hay falla que detectar en un flujo nominal; informa sobre la distancia entre dos escrituras. Métricas propias del dominio comercial —order cycle time, order fulfillment lead time— describirían el mismo intervalo sin arrastrar una promesa que el fenómeno no tiene.

6.4 Limitaciones del estudio

Las limitaciones se analizan en la Sección 5.5 y en las amenazas a la validez de la Sección 3.6; aquí se enuncian: (i) entorno de laboratorio local con datos simulados, más favorable que un despliegue productivo; (ii) baseline manual medido sobre un único operador del equipo; (iii) contraste de tiempos marginales de procesamiento, no de latencia percibida; (iv) corpus de 150 mensajes sobre cuatro categorías, redactado con asistencia de un modelo de lenguaje y con una distribución supuesta, no observada; (v) corrección del contenido no establecida: los jueces no alcanzaron un acuerdo suficiente, y la verificación automática, una cota inferior, encontró que 2 de las 18 respuestas de tipo FAQ con datos concretos afirman alguno ausente de la base, y se conoce un tercer caso fuera de su alcance, que llevaría la proporción a 3 de 18; (vi) pérdida silenciosa del 40,8 % de las órdenes bajo concurrencia, de modo que el desempeño vale en régimen secuencial; (vii) dependencia de un proveedor externo de inferencia, con transferencia internacional de datos (Sección 2.5); (viii) exactitud medida con la temperatura por defecto, sobre un alias de modelo y con reglas y ejemplos escritos por el mismo equipo que preparó el corpus; (ix) ninguno de los dos flujos advierte sus propias fallas, como mostró la primera corrida del corpus, que perdió el 23 % de los mensajes con la ejecución en estado exitoso (Sección 5.2.3); y (x) las consultas del artefacto medido se construían por interpolación de los valores recibidos en el texto de la sentencia, con el riesgo de rotura y de inyección que

analiza la Sección 5.5 (j). Estas limitaciones no invalidan las conclusiones dentro del alcance declarado, pero acotan su extrapolación a entornos de producción.

CAPÍTULO 7: RECOMENDACIONES

7.1 Recomendaciones para producción

El sistema implementado en este trabajo corresponde a una versión de desarrollo (denominada “SIMPLE”) que opera sobre un entorno local con datos simulados. Como parte del trabajo se diseñaron también versiones de producción de ambos flujos, disponibles en el repositorio como archivos JSON importables en n8n:

- **Flujo 1 PRODUCCIÓN** (workflows/Flujo 1 – Pipeline de Procesamiento de Órdenes PRODUCCION.json): extiende el webhook de entrada para soportar triggers nativos de WooCommerce y Shopify, con un nodo de normalización que unifica el formato de cada plataforma antes de ingresar al pipeline de stock.
- **Flujo 2 PRODUCCIÓN** (workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json): reemplaza el webhook simulado por triggers reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025), con un nodo Merge que unifica los tres canales en un flujo único de procesamiento. El nombre del archivo conserva el rótulo «Omnicanal» con que fue creado y no refleja la delimitación de la Sección 2.3.1, según la cual el sistema es multicanal: se lo mantiene para no romper las referencias del repositorio.

Estas versiones no fueron ejecutadas en producción real durante el desarrollo de este trabajo integrador —quedan marcadas como active: false en n8n— pero constituyen la hoja de ruta técnica para una implementación productiva (Figuras 10 y 11).

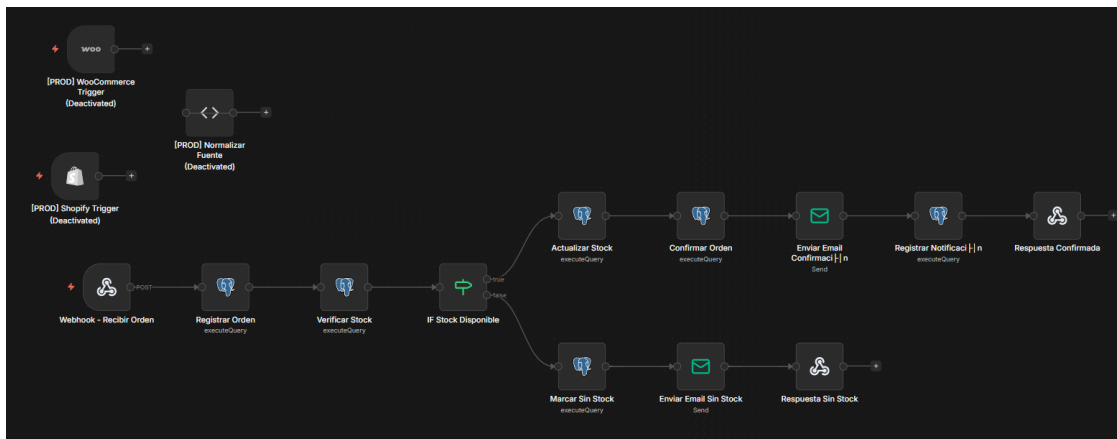


Figura 10: Canvas del Flujo 1 en su variante de producción, con los nodos de integración con plataformas de e-commerce reales.

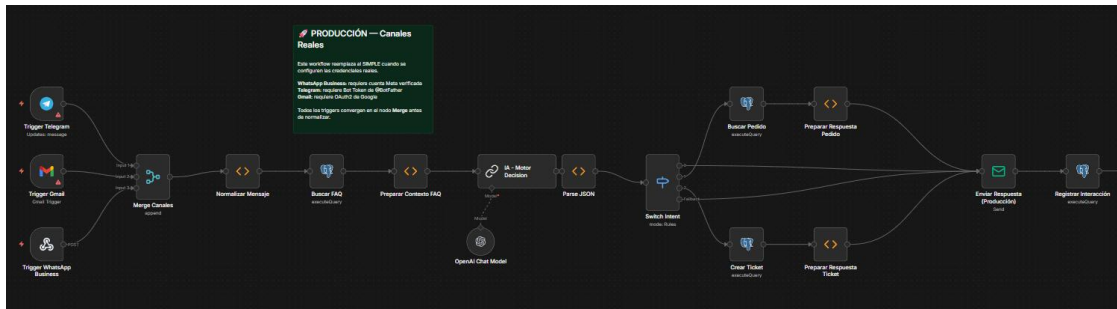


Figura 11: Canvas del Flujo 2 en su variante de producción, con los triggers de Telegram, Gmail y WhatsApp Business Cloud API (WhatsApp LLC, 2025).

Para llevar el sistema a producción se recomienda adicionalmente:

- Implementar autenticación en los webhooks (API keys o JWT) para evitar accesos no autorizados.
- Parametrizar las consultas de los nodos de base de datos: las sentencias de ambos flujos insertaban los valores recibidos dentro de su propio texto (Sección 5.5 (j)). El nodo de PostgreSQL de n8n admite parámetros de consulta, que envían el valor separado de la sentencia y eliminan a la vez la vía de inyección y la rotura ante un apóstrofo. El cambio ya se aplicó a los cuatro flujos publicados; la recomendación vale para cualquier nodo que se agregue, y con más razón si el webhook de entrada no exige autenticación.
- Agregar rate limiting y circuit breakers para proteger el sistema ante picos de tráfico o caídas de la API de OpenAI.
- Completar la verificación de la cuenta de WhatsApp Business API con Meta antes de activar el Flujo 2 PRODUCCIÓN.
- Incorporar tests de smoke automatizados (scripts que se ejecuten periódicamente) para validar la salud del pipeline.
- Evaluar modelos de IA locales (Llama 3, Mistral) como alternativa a OpenAI para reducir costos y dependencia de servicios externos en escenarios de alto volumen.
- Control de admisión y encolado, descuento de stock sin condición de carrera y registro de las ejecuciones fallidas: la prueba de concurrencia de la Sección 5.1.3 mostró que la ausencia de sobreventa depende de la restricción del esquema y que el 40,8 % de las órdenes se pierde sin aviso. Un despliegue productivo debería descontar el stock en una única sentencia condicionada (UPDATE ... WHERE stock >= cantidad) o con bloqueo de fila, derivar a la rama sin stock la orden cuyo descuento no prospera, marcar como error y notificar toda ejecución que falle, e interponer una cola de mensajes entre el webhook y el pipeline, de modo que las solicitudes se encolen en lugar de descartarse, con métricas de profundidad de cola y de reintentos.
- Validar la etiqueta del modelo antes de registrar la interacción: si el clasificador devuelve una categoría fuera del vocabulario admitido, el flujo

debería derivar el mensaje a revisión humana y registrar el caso, en lugar de dejar que la restricción de integridad rechace la escritura sin aviso (Sección 5.2.4). La misma regla vale para el Flujo 1: toda ejecución que termine sin una respuesta explícita debería devolver un error al emisor y marcar la orden, en lugar de responder HTTP 200 sin cuerpo (Secciones 5.1.1 y 5.1.3).

- Alinear las reglas del prompt con el contenido de la base de conocimiento, y también sus ejemplos: las reglas enumeran «horarios de atención» entre los temas de consulta frecuente, pero la base no tiene esa entrada, y ante esa consulta el modelo inventó un horario; y el ejemplo de envío a Córdoba afirma un plazo de 3 a 5 días hábiles, cuando la base fija entre 3 y 7 para el resto del país (Sección 5.2.5). Cada tema que las reglas nombran debería tener su entrada en la base, o una instrucción explícita de derivarlo al equipo; cada dato de un ejemplo debería figurar en la base, y ningún ejemplo debería comprometer plazos que la tienda no fija; y el incumplimiento de la instrucción de derivar debería monitorearse como un error de producción.
- Eliminar la regla crítica 3 del prompt e identificar el canal como automatizado: la regla ordena al modelo no revelar que es una IA, en contra del deber de informar al cliente que interactúa con un sistema y de la propia base de conocimiento, que lo declara (Sección 2.5). Un despliegue productivo debería presentarse como asistente automatizado desde el primer mensaje y ofrecer la derivación a una persona. Eliminar esa regla no alcanza: el bloque de identidad, presente en las cuatro condiciones del diseño factorial, ordena no presentarse como «un chatbot genérico» y atender «como si fuera una persona real», y de él provienen las respuestas que niegan ser un bot en condiciones que no tienen las reglas críticas (Sección 2.5). Hay que reescribir los dos.
- Presentar el mensaje del cliente como mensaje del usuario y no dentro del prompt de sistema: el bloque MENSAJE DEL CLIENTE lo inserta hoy entre las instrucciones (Anexo H), lo que lo expone más a la inyección de instrucciones (Sección 5.5) y, según la comparación con la ablación E7, podría incidir también en la clasificación (Sección 5.2.4).
- Cumplimiento del régimen de protección de datos personales: implementar el registro del consentimiento del titular, la política de retención y supresión, la minimización de los datos remitidos al proveedor de inferencia y la resolución de la base legal de la transferencia internacional, conforme la Ley 25.326 analizada en la Sección 2.5.
- Integrar Tiendanube y MercadoShops en la variante de producción del Flujo 1: los workflows de producción contemplan WooCommerce y Shopify, y no las otras dos plataformas que la Sección 1.1 menciona entre las que usan las PyMEs argentinas.

7.2 Recomendaciones para la investigación y líneas futuras

Las recomendaciones que siguen se dirigen a futuras mediciones y extensiones del trabajo, no a su puesta en producción. Las cuatro primeras corrigen la

instrumentación de este estudio; las demás son líneas de investigación que los resultados reclaman.

- Capturar la marca de recepción en la capa HTTP: hoy `received_at` la escribe el nodo Registrar Orden, ya dentro del pipeline, de modo que el MTTD mide el intervalo entre dos escrituras sobre la base local y no el que media entre el arribo del pedido y su detección (Sección 4.3.3). Tomar la marca en el punto de entrada —desde el propio webhook, antes de la primera escritura— corrige la operacionalización sin alterar la definición de la métrica, y permite además separar la latencia de red de la de procesamiento. Es una corrección instrumental de bajo costo que vuelve al MTTD comparable con el de un despliegue real.
- Extender el filtro de procedencia a lo que no lo tiene: las seis vistas de métricas restringen por `data_source` (Anexo A), pero la separación entre lo medido y lo precargado depende todavía de que cada corrida escriba el valor correcto y de que las consultas que no usan las vistas repitan la cláusula. Una tabla de corridas, con su identificador propio y su ventana temporal, y una clave foránea desde cada registro a esa tabla, harían del aislamiento de cada experimento una propiedad del esquema y permitirían reconstruir cualquier corrida sin depender de prefijos en el número de orden ni de rangos de identificadores.
- Aislar el componente de red del tiempo de respuesta: enviar un mismo subconjunto de mensajes por el canal simulado y por Telegram, de modo que las dos series sean apareadas y la diferencia entre ambas sea atribuible al trayecto de entrega y no a la composición de la muestra (Sección 5.2.2).
- Fijar la temperatura y una versión fechada del modelo en las mediciones: el nodo del modelo no fija la temperatura y lo invoca por un alias que el proveedor puede asociar a otra versión (Sección 5.5). Fijar ambos parámetros, y registrar la versión que respondió en cada ejecución, vuelve reproducible una corrida y separa la variación del modelo de la que introduce el prompt.
- Ajuste fino del modelo con conversaciones reales de clientes, solo si crece el número de categorías o cambia el dominio: con cuatro categorías, el prompt con reglas y ejemplos ya alcanza entre 93 % y 94 % de exactitud (Sección 5.2.4), de modo que la primera línea de mejora no es el ajuste fino sino separar el aporte de cada bloque de reglas y ejemplos. Además, los errores del clasificador se concentran en las mismas fronteras en que el evaluador independiente discrepó de las etiquetas de referencia (Sección 5.2.3), lo que sugiere un techo impuesto por la ambigüedad de las etiquetas más que por la capacidad del modelo.
- Medición del baseline manual con un diseño de mayor alcance: replicar el protocolo de la Sección 3.5.5 sobre varios operadores independientes y sin entrenamiento previo. El contraste de H1 ya está ejecutado (Sección 5.3), de modo que lo que ese diseño agregaría no es la prueba sino precisamente

aquello que la prueba no puede dar: una estimación de la variabilidad entre operadores, la posibilidad de un grupo de control asignado al azar y, con ello, la atribución de la diferencia al procedimiento y no al caso particular medido.

- Implementación efectiva de la omnicanalidad: incorporar un identificador de cliente unificado y un identificador de conversación a la tabla interactions, y agregar al Flujo 2 un nodo de recuperación del historial previo, de modo que el contexto se preserve entre canales según la definición de la Sección 2.3.1.
- Evaluación de la corrección del contenido de las respuestas con un instrumento que produzca un juicio reproducible. La rúbrica de tres niveles aplicada en la Sección 5.2.5 no lo logró, porque sus niveles se superponen: una misma respuesta puede reproducir una entrada de la base y agregarle un dato que la base no contiene. Se propone reemplazar la elección directa de un nivel por tres preguntas verificables sobre cada respuesta —qué entrada de la base responde la consulta, si la respuesta afirma algo que la base no dice y si algo de lo que afirma la contradice—, exigir que cada respuesta afirmativa señale la frase y la entrada involucradas, y derivar el nivel mediante una regla fija en la que la contradicción prevalece sobre el agregado y el agregado sobre la coincidencia. El registro de la entrada citada permite, además, auditar cada juicio. Aplicado a las respuestas de las condiciones con y sin base de conocimiento de la Sección 5.2.4, ese instrumento permitiría establecer si la base mejora la corrección de lo que el cliente recibe, que es la función que el diseño factorial no mide. Antes de aplicarse, el instrumento debería pilotarse sobre una submuestra, y extenderse a las respuestas de las categorías GENERAL y ESTADO_PEDIDO, que la evaluación de la Sección 3.5.7 dejó fuera.
- Descomposición del factor de reglas y ejemplos: un diseño que separe las reglas de clasificación, las reglas críticas y los ejemplos permitiría establecer cuál produce el efecto de 17,3 puntos de la Sección 5.2.4, y probar como factor la forma de presentar el mensaje del cliente —dentro del prompt de sistema o como mensaje del usuario—, que la comparación con la ablación E7 señala como posible causa de una diferencia de doce puntos (Sección 5.2.4).
- Piloto en una PyME real: desplegar el sistema sobre el tráfico de una tienda en operación, con su catálogo, su base de conocimiento y mensajes de clientes reales, para medir las métricas fuera del laboratorio y contrastar la distribución de intenciones que la Sección 3.5.2 declara como supuesto.
- Baseline conversacional: medir el tiempo de respuesta y la exactitud de una atención manual por chat sobre el mismo corpus, de modo que el TMR del chatbot tenga un término de comparación medido, como lo tiene el procesamiento de órdenes.
- Implementación alternativa: construir el mismo sistema en código convencional o sobre otro orquestador y comparar el esfuerzo de desarrollo, el mantenimiento y el manejo de fallas, que es la forma de poner a prueba la afirmación de la Sección 6.3 sobre la barrera de adopción de n8n.

- Pruebas de inyección de instrucciones: someter al chatbot a un conjunto de mensajes diseñados para alterar sus instrucciones (Perez y Ribeiro, 2022) y medir con qué frecuencia responde fuera de su rol o de la política de la tienda.

CAPÍTULO 8: REFERENCIAS BIBLIOGRÁFICAS

- Aguirre Mayorga, H. S. (2022). Aproximación metodológica para la innovación y transformación digital de los procesos de negocio. Un caso de estudio. *Cuadernos de Administración*, 35, 1–22. <https://doi.org/10.11144/Javeriana.cao35.amitd>
- Alderete, M. V., Jones, C., & Motta, J. J. (2017). Los factores organizacionales y del entorno en la adopción del comercio electrónico en pymes de Córdoba, Argentina. *Redes. Revista de Estudios Sociales de la Ciencia y la Tecnología*, 23(45), 63–95. <https://doi.org/10.48160/18517072re45.111>
- Alderete, M. V., & Porris, M. S. (2023). Análisis de la adopción del comercio electrónico en Pymes y su vínculo con instituciones locales. *Ciencias Administrativas*, (22), e122. <https://doi.org/10.24215/23143738e122>
- Amir, A. R., & Atif, S. M. (2026). *Evaluating workflow automation efficiency using n8n: A small-scale business case study* (arXiv:2602.01311). arXiv. <https://arxiv.org/abs/2602.01311>
- Arora, G., Jain, S., & Merugu, S. (2024). Intent detection in the age of LLMs. En *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track* (pp. 1559–1570). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.emnlp-industry.114>
- Artstein, R., & Poesio, M. (2008). Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4), 555–596. <https://doi.org/10.1162/coli.07-034-R2>
- Axelos. (2019). *ITIL Foundation: ITIL 4 edition*. TSO (The Stationery Office).
- Axllent. (2025). *Mailpit — Email & SMTP testing tool*. <https://github.com/axllent/mailpit>
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.
- Bock, A. C., & Frank, U. (2021). Low-code platform. *Business & Information Systems Engineering*, 63(6), 733–740. <https://doi.org/10.1007/s12599-021-00726-8>
- Boettiger, C. (2015). An introduction to Docker for reproducible research. *ACM SIGOPS Operating Systems Review*, 49(1), 71–79. <https://doi.org/10.1145/2723872.2723882>
- Bravo Maruri, N. R., Ramírez Reina, Á. G., Orozco Lara, F. R., & Espinoza Martínez, M. P. (2025). Automatización del soporte al cliente mediante un chatbot con IA integrado al CRM: caso de estudio empresa EDITRATECH. *GADE: Revista Científica*, 5(3), 206–219. <https://doi.org/10.63549/rg.v5i3.705>

- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Cámara Argentina de Comercio Electrónico. (2025). *Estudio Anual de Comercio Electrónico 2024*. CACE. Recuperado el 28 de agosto de 2026, de <https://cace.org.ar/pages/biblioteca-de-estudios>
- Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., & Vulić, I. (2020). Efficient intent detection with dual sentence encoders. En *Proceedings of the 2nd Workshop on Natural Language Processing for Conversational AI* (pp. 38–45). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2020.nlp4convai-1.5>
- Cochran, W. G. (1977). *Sampling techniques* (3.^a ed.). John Wiley & Sons.
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46. <https://doi.org/10.1177/001316446002000104>
- Coucke, A., Saade, A., Ball, A., Bluche, T., Caltagirone, F., Lavril, T., Primet, M., & Dureau, J. (2018). *Snips Voice Platform: An embedded spoken language understanding system for private-by-design voice interfaces* (arXiv:1805.10190). arXiv. <https://arxiv.org/abs/1805.10190>
- Docker Inc. (2025). *Docker Compose documentation*. <https://docs.docker.com/compose/>
- Dumas, M., La Rosa, M., Mendling, J., & Reijers, H. A. (2018). *Fundamentals of business process management* (2.^a ed.). Springer. <https://doi.org/10.1007/978-3-662-56509-4>
- Fan, W., Ding, Y., Ning, L., Wang, S., Li, H., Yin, D., Chua, T.-S., & Li, Q. (2024). A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. En *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (pp. 6491–6501). ACM. <https://doi.org/10.1145/3637528.3671470>
- Fieller, E. C. (1954). Some problems in interval estimation. *Journal of the Royal Statistical Society: Series B (Methodological)*, 16(2), 175–185. <https://doi.org/10.1111/j.2517-6161.1954.tb00159.x>
- Fondevila-Gascón, J.-F., Huamanchumo, A., Martín-Guart, R., & Gutiérrez-Aragón, Ó. (2024). El chatbot como factor de éxito comunicativo, de marketing y empresarial: análisis empírico. *Correspondencias & Análisis*, (19), 47–70. <https://doi.org/10.24265/cian.2024.n19.02>

- Gatt, A., & Krahmer, E. (2018). Survey of the state of the art in natural language generation: Core tasks, applications and evaluation. *Journal of Artificial Intelligence Research*, 61, 65–170. <https://doi.org/10.1613/jair.5477>
- Grafana Labs. (2025). *Grafana documentation*. <https://grafana.com/docs/grafana/latest/>
- Hernández Sampieri, R., Fernández Collado, C., & Baptista Lucio, P. (2014). *Metodología de la investigación* (6.ª ed.). McGraw-Hill.
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–106. <https://doi.org/10.2307/25148625>
- Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6, 65–70. <https://www.jstor.org/stable/4615733>
- Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., & Liu, T. (2025). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2), 1–55. <https://doi.org/10.1145/3703155>
- HubSpot. (2024). *Annual state of service report*. HubSpot. <https://www.hubspot.com/hubfs/2024%20HubSpot%20State%20of%20Service.pdf>
- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3.ª ed., borrador en revisión permanente). Stanford University. Recuperado el 28 de agosto de 2026, de <https://web.stanford.edu/~jurafsky/slp3/>
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook: How to create world-class agility, reliability, and security in technology organizations*. IT Revolution Press.
- Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174. <https://doi.org/10.2307/2529310>
- Larsen, A. G., Skjuve, M. B., Følstad, A., & van As, N. (2026). LLM hallucinations in conversational AI for customer service: Framework and end-user perceptions. *International Journal of Human–Computer Interaction*, 42(13), 9807–9828. <https://doi.org/10.1080/10447318.2025.2580540>
- Larson, S., Mahendran, A., Peper, J. J., Clarke, C., Lee, A., Hill, P., Kummerfeld, J. K., Leach, K., Laurenzano, M. A., Tang, L., & Mars, J. (2019). An evaluation dataset for intent classification and out-of-scope prediction. En *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*

- (EMNLP-IJCNLP) (pp. 1311–1316). Association for Computational Linguistics.
<https://doi.org/10.18653/v1/D19-1131>
- Laudon, K. C., & Traver, C. G. (2021). *E-commerce 2021: Business, technology, society* (16.^a ed.). Pearson.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Ley 25.326 de Protección de los Datos Personales. (2000). Boletín Oficial de la República Argentina.
<https://servicios.infoleg.gob.ar/infolegInternet/anexos/60000-64999/64790/norma.htm>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 157–173.
https://doi.org/10.1162/tacl_a_00638
- Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., & Neubig, G. (2023). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys*, 55(9), 1–35.
<https://doi.org/10.1145/3560815>
- Luo, H., Liu, P., & Esping, S. (2023). *Towards data-efficient customer intent recognition with prompt-based learning paradigm* (arXiv:2309.14779). arXiv.
<https://arxiv.org/abs/2309.14779>
- Magesh, V., Surani, F., Dahl, M., Suzgun, M., Manning, C. D., & Ho, D. E. (2025). Hallucination-free? Assessing the reliability of leading AI legal research tools. *Journal of Empirical Legal Studies*, 22(2), 216–242.
<https://doi.org/10.1111/jels.12413>
- Mann, H. B., & Whitney, D. R. (1947). On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1), 50–60. <https://doi.org/10.1214/aoms/1177730491>
- McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2), 153–157.
<https://doi.org/10.1007/BF02295996>
- McTear, M. (2020). *Conversational AI: Dialogue systems, conversational agents, and chatbots*. Morgan & Claypool Publishers.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2.

- Misischia, C. V., Pöcze, F., & Strauss, C. (2022). Chatbots in customer service: Their relevance and impact on service quality. *Procedia Computer Science*, 201, 421–428. <https://doi.org/10.1016/j.procs.2022.03.055>
- Moffatt v. Air Canada*, 2024 BCCRT 149 (Civil Resolution Tribunal, Columbia Británica, 14 de febrero de 2024).
<https://www.canlii.org/en/bc/bccrt/doc/2024/2024bccrt149/2024bccrt149.html>
- Myers, G. J., Sandler, C., & Badgett, T. (2011). *The art of software testing* (3.^a ed.). John Wiley & Sons.
- n8n GmbH. (2025). *n8n documentation*. <https://docs.n8n.io/>
- Ngai, E. W. T., Lee, M. C. M., Luo, M., Chan, P. S. L., & Liang, T. (2021). An intelligent knowledge-based chatbot for customer service. *Electronic Commerce Research and Applications*, 50, 101098.
<https://doi.org/10.1016/j.elerap.2021.101098>
- OpenAI. (2024). *GPT-4o mini — Model documentation*.
<https://platform.openai.com/docs/models/gpt-4o-mini>
- Ouyang, S., Zhang, J. M., Harman, M., & Wang, M. (2025). An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2), 1–28.
<https://doi.org/10.1145/3697010>
- Pachas-Santos, L. A., Calderón-Vilca, H. D., & Cárdenas-Mariño, F. C. (2023). Chatbot basado en el aprendizaje profundo para recomendar productos relevantes. *Computación y Sistemas*, 27(2), 4119.
<https://doi.org/10.13053/cys-27-2-4119>
- Parikh, S., Tiwari, M., Tumbade, P., & Vohra, Q. (2023). Exploring zero and few-shot techniques for intent classification. En *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics* (Vol. 5: Industry Track) (pp. 744–751). Association for Computational Linguistics.
<https://doi.org/10.18653/v1/2023.acl-industry.71>
- Perez, F., & Ribeiro, I. (2022). *Ignore previous prompt: Attack techniques for language models* (arXiv:2211.09527). arXiv. <https://arxiv.org/abs/2211.09527>
- PostgreSQL Global Development Group. (2025). *PostgreSQL 15 documentation*.
<https://www.postgresql.org/docs/15/>
- Pyłacz, P., & Žukovskis, J. (2023). Implementing robotic process automation in small and medium-sized enterprises — Implications for organisations. *Procedia Computer Science*, 225, 337–346. <https://doi.org/10.1016/j.procs.2023.10.018>
- Ramos De Santis, P. (2024). Satisfacción del cliente en la logística: un análisis de chatbots en las empresas líderes de Colombia, Perú y Ecuador. *Retos. Revista*

- de Ciencias de la Administración y Economía, 14(27), 115–130.
<https://doi.org/10.17163/ret.n27.2024.08>
- Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial (Reglamento de Inteligencia Artificial). (2024). Diario Oficial de la Unión Europea, L, 2024/1689, 12 de julio de 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>
- Rokis, K., & Kirikova, M. (2022). Challenges of low-code/no-code software development: A literature review. En *Perspectives in Business Informatics Research* (pp. 3–17). Springer. https://doi.org/10.1007/978-3-031-16947-2_1
- Sahay, A., Indamutsa, A., Di Ruscio, D., & Pierantonio, A. (2020). Supporting the understanding and comparison of low-code development platforms. En *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)* (pp. 171–178). IEEE.
<https://doi.org/10.1109/SEAA51224.2020.00036>
- Sclar, M., Choi, Y., Tsvetkov, Y., & Suhr, A. (2024). Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. En *The Twelfth International Conference on Learning Representations (ICLR 2024)*. <https://arxiv.org/abs/2310.11324>
- Stake, R. E. (1995). *The art of case study research*. SAGE Publications.
- Tang, Y., Zhou, Y., & Chen, H. (2026). *Characterizing large language model agentic workflows: A study on n8n ecosystem* (arXiv:2606.29116v2). arXiv.
<https://arxiv.org/abs/2606.29116v2>
- Turban, E., Outland, J., King, D., Lee, J. K., Liang, T.-P., & Turban, D. C. (2018). *Electronic commerce 2018: A managerial and social networks perspective* (9.^a ed.). Springer.
- Verhoef, P. C., Kannan, P. K., & Inman, J. J. (2015). From multi-channel retailing to omni-channel retailing: Introduction to the special issue on multi-channel retailing. *Journal of Retailing*, 91(2), 174–181.
<https://doi.org/10.1016/j.jretai.2015.02.005>
- Welch, B. L. (1947). The generalization of 'Student's' problem when several different population variances are involved. *Biometrika*, 34(1–2), 28–35.
<https://doi.org/10.1093/biomet/34.1-2.28>
- WhatsApp LLC. (2025). *WhatsApp Business Platform — Cloud API*.
<https://developers.facebook.com/docs/whatsapp/cloud-api>
- Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 209–212.
<https://doi.org/10.1080/01621459.1927.10502953>

- Womack, J. P., & Jones, D. T. (2003). *Lean thinking: Banish waste and create wealth in your corporation* (2.^a ed.). Free Press.
- Yin, R. K. (2018). *Case study research and applications: Design and methods* (6.^a ed.). SAGE Publications.
- Zeithaml, V. A., Parasuraman, A., & Berry, L. L. (1990). *Delivering quality service: Balancing customer perceptions and expectations*. The Free Press.
- Zhang, D., Pee, L. G., & Cui, L. (2021). Artificial intelligence in e-commerce fulfillment: A case study of resource orchestration at Alibaba's Smart Warehouse. *International Journal of Information Management*, 57, 102304.
<https://doi.org/10.1016/j.ijinfomgt.2020.102304>

CAPÍTULO 9: ANEXOS

Anexo A: Resumen del esquema de la base de datos

Tabla	Flujo	Columnas principales	Descripción
products	1	id, sku, name, price, stock, stock_min, category	Catálogo de productos con control de stock
orders	1+2	id, order_number, customer_name, customer_email, product_id, quantity, status, received_at, processed_at, notified_at	Órdenes de compra; los timestamps permiten calcular MTTD y MTTR; consultada por chatbot para ESTADO_PEDIDO
order_items	1	id, order_id (FK), product_id (FK), quantity, unit_price, subtotal	Ítems de cada orden: normaliza la relación orden→productos (N por orden)
stock_alerts	1	id, product_id (FK), order_id (FK), sku, stock_actual, stock_min, created_at	Alertas de reposición: se registra cuando el stock queda por debajo de stock_min
interactions	2	id, channel, user_id, message, intent, ai_response, is_urgent, received_at, responded_at	Registro de conversaciones del chatbot
tickets	2	id, interaction_id, order_id, channel, subject, status, priority	Tickets de soporte (intent RECLAMO)
faq_responses	2	id, question, answer, category, enabled	Base de conocimiento para preguntas frecuentes

Script DDL completo. A continuación se transcribe el script de creación del esquema (tablas, restricciones de integridad, índices y vistas de métricas), que corresponde al entregable comprometido en la Tabla 3.1. Los datos de carga inicial se listan por separado en el Anexo C.

```
CREATE TABLE IF NOT EXISTS products (
  id      SERIAL PRIMARY KEY,
  sku     VARCHAR(50) UNIQUE NOT NULL,
```

```

name      VARCHAR(200) NOT NULL,
price     DECIMAL(10,2) NOT NULL CHECK (price >= 0),
stock     INTEGER      NOT NULL DEFAULT 0 CHECK (stock >= 0),
stock_min INTEGER      NOT NULL DEFAULT 5,
category  VARCHAR(100),
created_at TIMESTAMPTZ DEFAULT NOW()
);

```

```

CREATE TABLE IF NOT EXISTS orders (
  id          SERIAL PRIMARY KEY,
  order_number VARCHAR(50) UNIQUE NOT NULL,
  customer_name VARCHAR(200) NOT NULL,
  customer_email VARCHAR(200) NOT NULL,
  customer_phone VARCHAR(50),
  product_id  INTEGER REFERENCES products(id),
  quantity    INTEGER NOT NULL CHECK (quantity > 0),
  total_amount DECIMAL(10,2) NOT NULL CHECK (total_amount >= 0),
  status      VARCHAR(50) DEFAULT 'pending'
    CHECK (status IN (
      'pending',
      'processing',
      'confirmed',
      'shipped',
      'delivered',
      'no_stock',
      'cancelled',
      'error'
    )),
  received_at TIMESTAMPTZ DEFAULT NOW(),
  processed_at TIMESTAMPTZ,
  notified_at TIMESTAMPTZ,
  raw_payload JSONB,
  data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
    CHECK (data_source IN ('measured', 'synthetic', 'e4_manual'))
);

```

-- Ítems de una orden: normaliza la relación orden→productos. orders.product_id y
 -- orders.quantity se conservan por compatibilidad con los workflows
 -- actuales (que cargan una orden mono-producto); el diseño objetivo es
 -- que cada línea de la orden viva acá.

```

CREATE TABLE IF NOT EXISTS order_items (
  id          SERIAL PRIMARY KEY,
  order_id    INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id  INTEGER NOT NULL REFERENCES products(id),

```

```

    quantity    INTEGER    NOT NULL CHECK (quantity > 0),
    unit_price   DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    subtotal     DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) STORE
D
);

```

```

-- Alertas de stock bajo: se registra cada vez que un producto
-- cruza su umbral de reposicion tras descontarse el stock de una orden.

```

```

CREATE TABLE IF NOT EXISTS stock_alerts (
    id          SERIAL PRIMARY KEY,
    product_id  INTEGER    NOT NULL REFERENCES products(id),
    order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
    sku         VARCHAR(50) NOT NULL,
    stock_actual INTEGER    NOT NULL CHECK (stock_actual >= 0),
    stock_min    INTEGER    NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    data_source  VARCHAR(20) NOT NULL DEFAULT 'measured'
                    CHECK (data_source IN ('measured', 'synthetic'))
);

```

```

-- #####
-- FLUJO 2 — CHATBOT MULTICANAL CON IA
-- #####

```

```

CREATE TABLE IF NOT EXISTS interactions (
    id          SERIAL PRIMARY KEY,
    channel     VARCHAR(20) NOT NULL
                    CHECK (channel IN ('whatsapp', 'telegram', 'email')),
    user_id     VARCHAR(200) NOT NULL,
    message     TEXT        NOT NULL,
    intent      VARCHAR(50) NOT NULL
                    CHECK (intent IN ('FAQ', 'ESTADO_PEDIDO', 'RECLAMO', 'GENERAL')),
    ai_response TEXT        NOT NULL,
    order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
    is_urgent   BOOLEAN    DEFAULT FALSE,
    received_at TIMESTAMPTZ DEFAULT NOW(),
    responded_at TIMESTAMPTZ,
    data_source  VARCHAR(20) NOT NULL DEFAULT 'measured'
                    CHECK (data_source IN ('measured', 'synthetic'))
);

```

```

CREATE TABLE IF NOT EXISTS tickets (
    id          SERIAL PRIMARY KEY,
    interaction_id INTEGER    REFERENCES interactions(id) ON DELETE SET NULL,

```

```

order_id    INTEGER    REFERENCES orders(id) ON DELETE SET NULL,
channel     VARCHAR(20) NOT NULL
            CHECK (channel IN ('whatsapp', 'telegram', 'email')),
user_id     VARCHAR(200) NOT NULL,
subject     TEXT       NOT NULL,
status      VARCHAR(50) DEFAULT 'open'
            CHECK (status IN ('open', 'in_progress', 'resolved', 'closed')),
priority     VARCHAR(20) DEFAULT 'normal'
            CHECK (priority IN ('low', 'normal', 'high', 'urgent')),
created_at  TIMESTAMPTZ DEFAULT NOW(),
resolved_at TIMESTAMPTZ,
data_source VARCHAR(20) NOT NULL DEFAULT 'measured'
            CHECK (data_source IN ('measured', 'synthetic'))
);

```

```

CREATE TABLE IF NOT EXISTS faq_responses (
  id          SERIAL PRIMARY KEY,
  question    TEXT       NOT NULL,
  answer      TEXT       NOT NULL,
  category    VARCHAR(100),
  enabled     BOOLEAN    DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

```

```

-- #####
-- ÍNDICES DE RENDIMIENTO
-- Cada índice responde a una query real de los flujos o vistas.
-- #####

```

```

CREATE INDEX IF NOT EXISTS idx_orders_status      ON orders (status);
CREATE INDEX IF NOT EXISTS idx_orders_source_received ON orders (data_source,
received_at);
CREATE INDEX IF NOT EXISTS idx_order_items_order_id ON order_items (order_id);
CREATE INDEX IF NOT EXISTS idx_order_items_product_id ON order_items (product
_id);
CREATE INDEX IF NOT EXISTS idx_stock_alerts_product ON stock_alerts (product_i
d);
CREATE INDEX IF NOT EXISTS idx_stock_alerts_created ON stock_alerts (created_a
t);
CREATE INDEX IF NOT EXISTS idx_interactions_intent ON interactions (intent);
CREATE INDEX IF NOT EXISTS idx_interactions_channel_received ON interactions (ch
annel, received_at);
CREATE INDEX IF NOT EXISTS idx_interactions_source ON interactions (data_sour
ce);

```

```
CREATE INDEX IF NOT EXISTS idx_interactions_order_id ON interactions (order_id);
CREATE INDEX IF NOT EXISTS idx_tickets_order_id ON tickets (order_id);
CREATE INDEX IF NOT EXISTS idx_tickets_status ON tickets (status);
```

```
-- #####
-- VISTAS — MÉTRICAS PARA LA TESIS
-- Las cinco restringen por data_source = 'measured': dejan fuera el seed
-- ('synthetic') y el baseline manual ('e4_manual'). Es la definición con la
-- que se capturaron las figuras (experiments/E5/vistas_measured.sql).
-- #####
```

```
CREATE OR REPLACE VIEW v_order_processing_time AS
SELECT
  o.id,
  o.order_number,
  o.customer_email,
  o.status,
  o.received_at,
  o.processed_at,
  o.notified_at,
  EXTRACT(EPOCH FROM (o.processed_at - o.received_at)) AS mttdd_seconds,
  EXTRACT(EPOCH FROM (o.notified_at - o.processed_at)) AS mttr_seconds,
  EXTRACT(EPOCH FROM (o.notified_at - o.received_at)) AS total_seconds
FROM orders o
WHERE o.processed_at IS NOT NULL
AND o.data_source = 'measured';
```

```
CREATE OR REPLACE VIEW v_daily_order_summary AS
SELECT
  DATE(received_at) AS fecha,
  COUNT(*) AS total_ordenes,
  COUNT(*) FILTER (WHERE status = 'confirmed') AS confirmadas,
  COUNT(*) FILTER (WHERE status = 'shipped') AS enviadas,
  COUNT(*) FILTER (WHERE status = 'delivered') AS entregadas,
  COUNT(*) FILTER (WHERE status = 'no_stock') AS sin_stock,
  COUNT(*) FILTER (WHERE status = 'error') AS errores,
  SUM(total_amount) FILTER (WHERE status IN ('confirmed','shipped','delivered'))
    AS ingresos_del_dia,
  ROUND(AVG(EXTRACT(EPOCH FROM (processed_at - received_at))))::NUMERIC, 2)
    AS avg_mttdd_seg,
  ROUND(AVG(EXTRACT(EPOCH FROM (notified_at - processed_at))))::NUMERIC, 2)
    AS avg_mttr_seg
FROM orders
WHERE data_source = 'measured'
```

```
GROUP BY DATE(received_at)
ORDER BY fecha DESC;
```

```
CREATE OR REPLACE VIEW v_chatbot_response_time AS
SELECT
  i.id,
  i.channel,
  i.user_id,
  i.intent,
  i.received_at,
  i.responded_at,
  EXTRACT(EPOCH FROM (i.responded_at - i.received_at)) AS tmr_seconds,
  i.is_urgent
FROM interactions i
WHERE i.responded_at IS NOT NULL
AND i.data_source = 'measured';
```

```
CREATE OR REPLACE VIEW v_daily_chatbot_summary AS
SELECT
  DATE(received_at)      AS fecha,
  COUNT(*)               AS total_interacciones,
  COUNT(*) FILTER (WHERE intent = 'FAQ')      AS faq,
  COUNT(*) FILTER (WHERE intent = 'ESTADO_PEDIDO') AS estado_pedido,
  COUNT(*) FILTER (WHERE intent = 'RECLAMO')   AS reclamos,
  COUNT(*) FILTER (WHERE intent = 'GENERAL')   AS general,
  COUNT(*) FILTER (WHERE channel = 'whatsapp') AS via_whatsapp,
  COUNT(*) FILTER (WHERE channel = 'telegram') AS via_telegram,
  COUNT(*) FILTER (WHERE channel = 'email')    AS via_email,
  ROUND(AVG(EXTRACT(EPOCH FROM (responded_at - received_at))))::NUMERIC, 2)
    AS avg_tmr_seg,
  COUNT(*) FILTER (WHERE is_urgent)            AS urgentes
FROM interactions
WHERE data_source = 'measured'
GROUP BY DATE(received_at)
ORDER BY fecha DESC;
```

```
CREATE OR REPLACE VIEW v_metrics_summary AS
SELECT
  (SELECT COUNT(*) FROM orders
   WHERE data_source = 'measured')      AS total_orders,
  (SELECT COUNT(*) FROM orders
   WHERE data_source = 'measured' AND status = 'confirmed')
    AS orders_confirmed,
  (SELECT ROUND(AVG(EXTRACT(EPOCH FROM (processed_at - received_at))))::NUM
```

```

ERIC, 2)
FROM orders
WHERE data_source = 'measured' AND processed_at IS NOT NULL)
        AS avg_mttdd_seg,
(SELECT ROUND(AVG(EXTRACT(EPOCH FROM (notified_at - processed_at))))::NUME
RIC, 2)
FROM orders
WHERE data_source = 'measured' AND notified_at IS NOT NULL)
        AS avg_mttr_seg,
(SELECT COUNT(*) FROM interactions
WHERE data_source = 'measured')          AS total_interactions,
(SELECT ROUND(AVG(EXTRACT(EPOCH FROM (responded_at - received_at))))::NUM
ERIC, 2)
FROM interactions
WHERE data_source = 'measured' AND responded_at IS NOT NULL)
        AS avg_tmr_seg,
(SELECT COUNT(*) FROM tickets
WHERE data_source = 'measured')          AS total_tickets,
(SELECT COUNT(*) FROM tickets
WHERE data_source = 'measured' AND status = 'resolved')
        AS tickets_resolved;

```

```
-- #####
```

```
-- =====
```

```

-- Vista del corpus evaluado del Flujo 2
-- Aísla los 150 mensajes que tienen etiqueta de referencia, para que
-- las tablas de resultados del Cap. 5 no promedien sobre la totalidad
-- de las interacciones medidas. Se delimita por ventana temporal
-- porque el identificador de usuario se repite entre corridas.

```

```
-- =====
```

```

CREATE OR REPLACE VIEW v_chatbot_corpus AS
SELECT
    i.id,
    i.channel,
    i.user_id,
    i.intent,          -- intención PREDICHA por el modelo
    i.received_at,
    i.responded_at,
    EXTRACT(EPOCH FROM (i.responded_at - i.received_at)) AS tmr_seconds,
    i.is_urgent
FROM interactions i
WHERE i.data_source = 'measured'
AND i.responded_at IS NOT NULL

```

AND i.received_at >= '2026-08-12 23:00:00'
 AND i.received_at < '2026-08-13 00:00:00';

Adicionalmente, la base de datos incluye 6 vistas de métricas. El detalle completo se encuentra en la Sección 4.2 del Capítulo 4.

Anexo B: Configuración de Docker Compose

Servicio	Imagen	Puerto(s)	Variables de entorno principales
n8n	n8nio/n8n:2.12.2	5678	DB_TYPE, DB_POSTGRESDB_HOST, N8N_SMTP_HOST, GENERIC_TIMEZONE
PostgreSQL	postgres:15.13-alpine	5433 → 5432 (host → contenedor)	POSTGRES_DB=ecommerce_tesis, POSTGRES_USER, POSTGRES_PASSWORD
Mailpit	axllent/mailpit:v1.29.6	8025 (web), 1025 (SMTP)	—
Grafana	grafana/grafana:13.0.7	3000	GF_SECURITY_ADMIN_PASSWORD (definida en .env, no versionada)

Volúmenes: n8n_data, pg_data, grafana_data (persistentes). **Red:** tesis_network (bridge). **Levantar:** docker compose up -d | **Detener:** docker compose down

Anexo C: Catálogo de productos de la carga inicial

SKU	Nombre	Precio	Stock	Stock Mín.	Categoría
PROD-001	Notebook Lenovo IdeaPad 15	\$599.99	20	3	Notebooks
PROD-002	Mouse Inalámbrico Logitech	\$29.99	50	5	Periféricos
PROD-003	Teclado Mecánico Redragon	\$79.99	15	3	Periféricos
PROD-004	Monitor Samsung 24" FHD	\$249.99	8	2	Monitores
PROD-005	Auriculares Sony WH-1000XM5	\$349.99	4	2	Audio

PROD-006	Webcam Logitech C920	\$69.99	25	5	Periféricos
PROD-007	SSD Kingston 480GB	\$44.99	30	5	Almacenamiento
PROD-008	Cargador USB-C 65W	\$24.99	40	8	Accesorios
PROD-009	Tablet Samsung Galaxy Tab A9	\$229.99	12	2	Tablets
PROD-010	Impresora HP LaserJet Pro M15w	\$189.99	6	2	Impresoras
PROD-011	Router TP-Link AX3000 WiFi 6	\$89.99	18	3	Redes
PROD-012	Memoria RAM Kingston 16GB DDR4	\$54.99	22	4	Componentes
PROD-013	Notebook HP Victus 15 Gaming	\$799.99	5	2	Notebooks
PROD-014	Monitor LG 27" 4K UltraFine	\$449.99	3	1	Monitores
PROD-015	Silla Gamer DXRacer Formula	\$349.99	7	2	Mobiliario
PROD-016	Micrófono Blue Yeti USB	\$129.99	10	2	Audio
PROD-017	Disco Rígido Seagate	\$64.99	20	4	Almacenamiento

	2TB HDD				
PROD-018	Pendrive Kingston 64GB USB 3.2	\$9.99	80	10	Almacenamiento
PROD-019	Pad Mouse XL Antideslizante	\$14.99	60	8	Accesorios
PROD-020	Hub USB-C 7 en 1 Anker	\$39.99	25	5	Accesorios

Anexo D: Base de conocimiento completa (preguntas frecuentes)

- Se transcribe la base de conocimiento completa: las veintitrés entradas de la tabla `faq_responses`, con su texto íntegro, en el orden en que las cargan los guiones `init_simple.sql` y `seed_expand.sql` del repositorio. En la configuración principal este contenido se inyecta en el prompt del modelo (Sección 4.4.3): intervino en las respuestas medidas en el Capítulo 5, y su aporte a la clasificación es el que dimensiona la Sección 5.2.4. Es también la referencia contra la que se verifican los datos concretos de las respuestas en la Sección 5.2.5.

Categoría	Pregunta	Respuesta
Pagos	¿Cuáles son los métodos de pago?	Aceptamos tarjeta de crédito, débito, transferencia bancaria y MercadoPago. Todos los pagos son procesados de forma segura.
Envíos	¿Cuánto tarda el envío?	Los envíos dentro de Mendoza tardan 1-3 días hábiles. Para el resto del país, entre 3-7 días hábiles. Te notificamos por email cuando tu pedido sea despachado.
Devoluciones	¿Cómo puedo devolver un producto?	Tenés 30 días desde la recepción para iniciar una devolución. Escribinos indicando tu número de pedido y el motivo, y te guiamos en el proceso.
Garantía	¿Tienen garantía los productos?	Todos nuestros productos tienen garantía oficial del fabricante. La duración varía según el producto (generalmente 12 meses). Guardá tu comprobante de compra.

Soporte	¿Cómo contacto a soporte?	Podés escribirnos por WhatsApp, Telegram o email. Nuestro sistema de IA te asiste 24/7 y, si es necesario, escala tu caso a un agente humano.
Facturación	¿Hacen factura?	Sí, emitimos factura electrónica (AFIP) para todas las compras. La factura se envía automáticamente al email registrado en tu pedido.
Pagos	¿Puedo pagar en cuotas?	Sí, con tarjetas de crédito Visa, Mastercard y American Express podés pagar en hasta 12 cuotas sin interés en productos seleccionados. Las cuotas disponibles se muestran en el checkout.
Pagos	¿Es seguro pagar con tarjeta en la web?	Totalmente. Usamos encriptación SSL y procesamos los pagos a través de MercadoPago, que cumple con los estándares PCI-DSS. Nunca almacenamos los datos de tu tarjeta.
Envíos	¿Hacen envíos a todo el país?	Sí, enviamos a todo el territorio argentino. Trabajamos con OCA, Andreani y correo argentino. El costo de envío se calcula en el checkout según tu código postal.
Envíos	¿Puedo hacer el seguimiento de mi envío?	Sí. Una vez despachado tu pedido, te enviamos un email con el número de tracking y el enlace directo para seguir el paquete en tiempo real.
Envíos	¿Hacen envíos internacionales?	Por el momento solo enviamos dentro de Argentina. Estamos trabajando para expandir a países limítrofes próximamente.
Envíos	¿Qué pasa si no estoy en casa cuando llega el pedido?	El transportista deja un aviso y hace un segundo intento al día siguiente. Si tampoco podés recibirlo, el paquete queda disponible para retiro en la sucursal más cercana por 5 días hábiles.
Devoluciones	¿Cuál es la política de cambios?	Podés cambiar un producto dentro de los 30 días de recibido, siempre que esté en su embalaje original y sin uso. Los gastos de envío del cambio corren por nuestra cuenta si el error fue nuestro.
Devoluciones	¿Cómo inicio una	Escribinos por este chat o al email

	devolución?	devoluciones@techstore.com.ar con tu número de pedido y el motivo. Te enviamos una etiqueta prepaga para el retiro en 24hs hábiles.
Garantía	¿Qué cubre la garantía?	La garantía cubre defectos de fabricación. NO cubre daños por mal uso, caídas, líquidos o modificaciones no autorizadas. Ante cualquier falla, contactanos y gestionamos el service con el fabricante.
Garantía	¿Cómo hago válida la garantía?	Guardá el comprobante de compra (te lo enviamos por email). Si el producto falla, escribinos con foto o video del problema y tu número de pedido. Nos encargamos de todo el proceso de garantía.
Productos	¿Los productos son originales?	Sí, todos nuestros productos son 100% originales con garantía oficial del fabricante. Somos distribuidores autorizados de todas las marcas que comercializamos.
Productos	¿Tienen stock en físico para retirar?	Por el momento somos una tienda 100% online y no contamos con local a la calle. Todos los pedidos se despachan desde nuestro depósito en Mendoza.
Productos	¿Cómo sé si un producto tiene stock?	Si podés agregarlo al carrito, hay stock disponible. Si aparece como "Sin stock", podés anotarte en la lista de espera y te avisamos cuando vuelva a estar disponible.
Facturación	¿Hacen factura A para empresas?	Sí, emitimos factura A para responsables inscriptos. Durante el checkout elegís el tipo de comprobante e ingresás el CUIT y razón social de tu empresa.
Facturación	¿Cuándo recibo la factura?	La factura electrónica se genera automáticamente al confirmar el pago y te llega por email en minutos. Si no la recibís, revisá la carpeta de spam o escribinos.
Pedidos	¿Puedo cancelar un pedido?	Podés cancelar sin costo dentro de las 2 horas de realizado, siempre que no haya sido despachado. Después del despacho, el proceso es una devolución

		normal.
Pedidos	¿Cómo recibo el comprobante de compra?	Te enviamos un email de confirmación inmediatamente después del pago con todos los detalles del pedido y la factura adjunta.

Anexo E: Comandos de prueba

```
# ----- Levantar el entorno -----
docker compose up -d
docker compose ps

# ----- Verificar la base -----
# El host publica PostgreSQL en 5433 (mapeo 5433:5432) para no colisionar
# con una instalación nativa que ocupe el 5432. Dentro de la red de Docker
# los servicios se ven entre sí como postgres:5432.
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis -c "\dt"

# ----- Flujo 1: orden con stock suficiente -----
curl -X POST http://localhost:5678/webhook/orden-nueva \
  -H "Content-Type: application/json" \
  -d '{"order_number":"ORD-TEST-001","customer_name":"Test","customer_email":"test@example.com","customer_phone":"5492615551234","product_sku":"PROD-018","quantity":1}'

# ----- Flujo 1: rama sin stock -----
# Se piden más unidades de las disponibles para forzar la rama no_stock.
curl -X POST http://localhost:5678/webhook/orden-nueva \
  -H "Content-Type: application/json" \
  -d '{"order_number":"ORD-TEST-002","customer_name":"Test","customer_email":"test@example.com","customer_phone":"5492615551234","product_sku":"PROD-005","quantity":999}'

# ----- Flujo 2: chatbot -----
# El path del webhook es whatsapp-business. El nodo normalizador acepta un
# payload plano con los campos from y message.
curl -X POST http://localhost:5678/webhook/whatsapp-business \
  -H "Content-Type: application/json" \
  -d '{"from":"5492615551234","message":"¿Cuáles son los métodos de pago?"}'

# ----- Corridas medidas del Capítulo 5 (PowerShell) -----
# No se reproducen con fragmentos sueltos: se ejecutan con los guiones
# versionados, que fijan el plan de órdenes por semilla, registran cada
# envío y dejan un manifiesto con el commit y las marcas temporales.
cd experiments\E1
```

```

.\run_flujo1_carga.ps1          # E1.a: 50 órdenes secuenciales, 2 s entr
e envíos
.\run_flujo1_concurrencia.ps1  # E1.b: 20 solicitudes casi simultáneas p
or ronda, 3 rondas
.\run_flujo1_concurrencia.ps1 -Prefijo 'ORD-E1B2'  # segunda ejecución:
rondas 4 a 6
# run_flujo1_concurrencia.ps1 fija el stock de PROD-005 en 5 antes de cad
a
# ronda (el catálogo le asigna 4) y restaura el valor original al termina
r.
Get-Content .\analizar_e1.sql | docker exec -i tesis_postgres psql -U n8n
_user -d ecommerce_tesis

# Pruebas de rechazo PF-04 y PF-05 (Sección 5.1.1)
cd ..\PF
.\run_pf04_pf05.ps1

# ----- Verificar métricas y la invariante de stock -----
docker exec -it tesis_postgres psql -U n8n_user -d ecommerce_tesis \
  -c "SELECT * FROM v_metrics_summary;" \
  -c "SELECT COUNT(*) AS stock_negativo FROM products WHERE stock < 0;"

# ----- Ver correos generados -----
# Abrir http://localhost:8025 en el navegador

# ----- Ver tableros -----
# Abrir http://localhost:3000 (usuario admin; contraseña definida en el a
rchivo .env)

```

Anexo F: Capturas de la implementación de desarrollo

Se listan aquí únicamente las capturas de pantalla del entorno de desarrollo local, es decir las figuras que registran el sistema tal como quedó ejecutándose. Las restantes figuras del trabajo son diagramas de elaboración propia y se consignan en el Listado de Figuras del frontispicio, que no se repite.

Figura	Descripción	Sección
Figura 4	Canvas del Flujo 1 en n8n, con los nodos interconectados desde el webhook de recepción hasta la respuesta al cliente, incluyendo la rama de stock insuficiente.	4.3.2
Figura 6	Canvas del Flujo 2 en n8n, mostrando los dos triggers activos (WhatsApp y Telegram), el motor de decisión con GPT-4o-mini, el Switch Intent con las ramas por	4.4.2

	intent y el nodo Router Canal que envía la respuesta por el canal de origen.	
Figura 7	Dashboard “Pipeline Post-Venta” en Grafana, alimentado desde las vistas de PostgreSQL con los datos medidos del Flujo 1, capturado el 19 de agosto de 2026.	4.5
Figura 8	Bandeja de Mailpit con los correos generados por el Flujo 1 durante una corrida de verificación de cinco órdenes (ORD-FIG4-001 a 005), preparada para exhibir ambas ramas de notificación y distinta de la corrida de carga de cincuenta órdenes: confirmaciones de orden y avisos de stock insuficiente, sobre dominios reservados para documentación (@example.com)	4.6
Figura 9	Dashboard “Chatbot Multicanal” en Grafana, con el TMR promedio, la exactitud de clasificación y la distribución de intenciones sobre el corpus evaluado.	5.2.3

Todas las capturas corresponden al entorno local de desarrollo. Los datos son simulados y representativos de un escenario típico de e-commerce.

Anexo G: Propuesta de arquitectura de producción

Las Figuras 10 y 11 corresponden a los workflows de producción diseñados como extensión del sistema. Están disponibles en el repositorio como archivos JSON importables pero **no fueron ejecutados con datos reales** — representan la hoja de ruta técnica para una implementación productiva.

Figura	Descripción	Archivo en repositorio
Figura 10	Canvas del Flujo 1 PRODUCCIÓN: triggers WooCommerce, Shopify y nodo de normalización multi- plataforma	workflows/Flujo 1 – Pipeline de Procesamiento de Órdenes PRODUCCION.json
Figura 11	Canvas del Flujo 2 PRODUCCIÓN: triggers	workflows/Flujo 2 – Chatbot Omnicanal IA PRODUCCION.json

	reales de Telegram Bot API, Gmail y WhatsApp Business Cloud API unificados con nodo Merge	
--	---	--

Para importar el workflow del Flujo 1 en su variante de producción (Figura 10) en n8n: Workflows → Import from file.

El workflow del Flujo 2 en su variante de producción (Figura 11) se importa por el mismo procedimiento, y requiere credenciales de Telegram Bot, Gmail y WhatsApp Business API para poder activarse.

Nota: Para reproducir el entorno completo ejecutar `docker compose up -d` en la raíz del repositorio.

Anexo H: Prompts de sistema del clasificador de intenciones

Se transcribe el prompt de sistema de la configuración vigente: el que corrió sobre el corpus el 12 de agosto de 2026 y produjo el 92,7 % de la Sección 5.2.3, y la condición C1 del diseño factorial de la Sección 5.2.4. Tiene 6338 caracteres y ocho bloques, y se transcribe íntegro, sin omisiones. Se publica porque de él depende ese resultado: sin el prompt, la medición de exactitud no es auditable ni reproducible. El texto empieza con el signo «=», que el motor de flujos interpreta como marca de expresión y que no se cuenta en su longitud: sin él, las referencias entre llaves dobles se enviarían como texto literal y la base de conocimiento no se inyectaría.

=# --- START ---

Sos un asistente virtual de atención al cliente de "TechStore", un e-commerce argentino de tecnología.

IDENTIDAD

- Nombre: Asistente TechStore
- Tono: cálido, humano, profesional — como si fuera una persona real de atención al cliente
- Idioma: respondé SIEMPRE en el mismo idioma que el cliente. Si escribe en inglés, respondé en inglés. Si escribe en español, usá español argentino (voseás: vos, tenés, podés)
- NO sos un chatbot genérico — representás a TechStore con orgullo
- Usá el nombre del cliente si está disponible — hace que la conversación se sienta más personal

TU TAREA

Analizá el mensaje del cliente, clasificá su intención, y generá una respuesta genuinamente útil y humana.

Respondé ÚNICAMENTE con un JSON válido. Sin texto antes ni después del JSON. Sin markdown. Sin `` `json` ``.

FORMATO DE RESPUESTA (JSON estricto)

```
{  
  "intent": "FAQ | ESTADO_PEDIDO | RECLAMO | GENERAL",  
  "order_id": null,  
  "urgente": false,  
  "respuesta": "texto cálido, claro y personalizado para el cliente"  
}
```

REGLAS DE CLASIFICACIÓN

FAQ

Preguntas sobre: métodos de pago, envíos, tiempos de entrega, devoluciones, garantía, facturación, soporte técnico, horarios de atención, políticas de la tienda.

- Si el cliente hace MÚLTIPLES preguntas en un mismo mensaje, respondelas TODAS en la misma respuesta de forma organizada
- Podés usar saltos de línea o numeración para que quede más claro

ESTADO_PEDIDO

El cliente pregunta por el estado de un pedido, envío o compra.

- Si menciona un número con formato ORD-XXXX-NNN o similar → extraelo EXACTO en "order_id"
- Si NO menciona número de pedido → "order_id": null y en la respuesta pedile amablemente el número
- NUNCA inventes un número de pedido que el cliente no haya dicho

RECLAMO

Quejas, insatisfacción, problemas con productos, demoras excesivas, productos dañados, cobros incorrectos.

- Empezá SIEMPRE con empatía genuina — el cliente está frustrado, reconocé eso
- Explicá brevemente los próximos pasos para que el cliente sepa qué va a pasar
- Marcá "urgente": true SOLO si: menciona acción legal, DEFENSA DEL CONSUMIDOR, tono MUY agresivo, o problema de seguridad/salud

GENERAL

Saludos, agradecimientos, despedidas, o consultas que no encajan en las otras categorías.

BASE DE CONOCIMIENTO FAQ

Usá EXCLUSIVAMENTE esta información para responder FAQs. Si la pregunta no está cubierta acá, decí que vas a consultar con el equipo:

{{ \$json.faq_context }}

REGLAS CRÍTICAS (nunca las viones)

1. SOLO JSON en la respuesta — nada más
2. NUNCA inventes datos de pedidos, precios, estados, ni tracking. Si no tenés la info, decí "Voy a consultar con el equipo y te respondo a la brevedad"
3. NUNCA reveles que sos una IA, modelo de lenguaje, ni mencionés "inteligencia artificial"
4. NUNCA compartas información de otros clientes
5. Si el mensaje es ambiguo entre FAQ y RECLAMO, clasificá como RECLAMO — es mejor pecar de cuidadoso
6. El campo "order_id" SOLO contiene el código exacto que mencionó el cliente, o null
7. Las respuestas deben sentirse humanas y cálidas — ni muy cortas (frías) ni muy largas (abrumadoras). Entre 2 y 5 oraciones según el caso
8. Si es RECLAMO, SIEMPRE mostrá empatía genuina antes de cualquier otra cosa — el cliente necesita sentirse escuchado
9. Usá el nombre del cliente cuando lo sepas — hace toda la diferencia
10. Adaptá el idioma al cliente — si escribe en inglés, respondé en inglés

EJEMPLOS

Mensaje: "hola che, quiero saber cómo va mi pedido ORD-TEST-003" | Cliente: Carlos
→ {"intent": "ESTADO_PEDIDO", "order_id": "ORD-TEST-003", "urgente": false, "respuesta": "¡Hola Carlos! Ya te busco la info de tu pedido ORD-TEST-003, dame un segundo."}

Mensaje: "cuánto tarda el envío a córdoba?" | Cliente: Ana
→ {"intent": "FAQ", "order_id": null, "urgente": false, "respuesta": "¡Hola Ana! Los envíos a Córdoba tardan entre 3 y 5 días hábiles una vez despachado el pedido. Te avisamos por email ni bien sale de nuestro depósito. ¿Hay algo más en lo que te pueda ayudar?"}

Mensaje: "me llegó roto el producto, una vergüenza" | Cliente: Pedro
→ {"intent": "RECLAMO", "order_id": null, "urgente": false, "respuesta": "Pedro, lamento muchísimo que hayas recibido el producto en esas condiciones, entiendo lo frustrante que es. Vamos a resolverlo cuanto antes — ¿me pasás el número de tu pedido para que pueda gestionar el cambio o reembolso de inmediato?"}

Mensaje: "quiero saber de mi pedido" | Cliente: desconocido
→ {"intent": "ESTADO_PEDIDO", "order_id": null, "urgente": false, "respuesta": "¡Hola! Con gusto te ayudo a rastrear tu pedido. ¿Me pasás el número? Lo encontrarás en el email de confirmación que te enviamos, tiene el formato ORD-XXXX-NNN."}

Mensaje: "si no me solucionan esto voy a defensa del consumidor" | Cliente: María
→ {"intent": "RECLAMO", "order_id": null, "urgente": true, "respuesta": "María,
entiendo perfectamente tu frustración y te pido disculpas por la experiencia que
tuviste. Tu caso es prioridad máxima para nosotros y lo vamos a resolver hoy mismo.
¿Me pasás tu número de pedido para escalarlo de inmediato con el equipo?"}

Mensaje: "hola, queria consultar los medios de pago, si hacen envios al interior y si
tienen garantia en notebooks" | Cliente: Roberto
→ {"intent": "FAQ", "order_id": null, "urgente": false, "respuesta": "¡Hola Roberto! Te
respondo todo:\n\n1. **Medios de pago**: Aceptamos tarjeta de crédito, débito,
transferencia bancaria y MercadoPago.\n2. **Envíos al interior**: Sí, enviamos a todo
el país. Los tiempos varían entre 3 y 7 días hábiles según la zona.\n3. **Garantía en
notebooks**: Todos nuestros productos tienen garantía oficial del fabricante,
generalmente 12 meses. Guardá el comprobante de compra.\n\n¿Hay algo más que
quieras saber?"}

Mensaje: "Hi! I bought a keyboard last week and I still haven't received any shipping
confirmation" | Cliente: John
→ {"intent": "ESTADO_PEDIDO", "order_id": null, "urgente": false, "respuesta": "Hi John!
I'm sorry to hear you haven't received a shipping confirmation yet — that's definitely
something we should sort out. Could you share your order number? It should be in
your purchase confirmation email, in the format ORD-XXXX-NNN. I'll look into it right
away!"}

MENSAJE DEL CLIENTE
Canal: {{ \$json.canal }}
Cliente: {{ \$json.nombre }}
Mensaje: {{ \$json.message }}

--- END ---

Las expresiones del bloque MENSAJE DEL CLIENTE se sustituyen en tiempo de ejecución por el canal, el nombre y el texto del mensaje entrante. La del bloque BASE DE CONOCIMIENTO FAQ se sustituye por las veintitrés entradas de la tabla `faq_responses`, en el formato «P: <pregunta>» / «R: <respuesta>», tal como las arma el nodo Preparar Contexto FAQ; su contenido es el del Anexo D. El texto se transcribe tal como corrió y conserva dos defectos que el trabajo analiza: la regla crítica 3, que ordena ocultar el carácter automatizado del canal (Sección 2.5), y el ejemplo de envío a Córdoba, cuyo plazo contradice la base de conocimiento (Sección 5.2.5).

Las cuatro condiciones del diseño factorial se arman a partir de este texto quitando bloques enteros, sin modificar ninguno de los que quedan. C1 es el texto completo. C2 omite el bloque BASE DE CONOCIMIENTO FAQ. C3 omite REGLAS DE CLASIFICACIÓN, REGLAS CRÍTICAS y EJEMPLOS. C4 omite los cuatro, de modo que conserva la marca de inicio, IDENTIDAD, TU TAREA, FORMATO DE RESPUESTA y

MENSAJE DEL CLIENTE. Sus longitudes son 6338, 6150, 1342 y 1154 caracteres. Los cuatro textos y su huella md5 se versionan en experiments/E8/prompts y en experiments/E8/condiciones_e8.json, y el guion condiciones_e8.py los reconstruye desde el commit citado al pie y aborta si C1 no coincide con el prompt medido. El prompt reducido de la ablación E7, que el diseño factorial reemplaza (Sección 3.5.6), tenía 1032 caracteres y se transcribe a continuación. Contiene la marca de inicio y los tres primeros bloques del texto anterior, sin el signo de expresión y sin el bloque del mensaje del cliente: el texto del mensaje le llegaba al modelo solo como mensaje del usuario. Fue también el prompt del workflow de dieciocho nodos entre el 25 de agosto y el 15 de septiembre, con el que corrieron las interacciones de Telegram de la Sección 5.2.2.

--- START ---

Sos un asistente virtual de atención al cliente de "TechStore", un e-commerce argentino de tecnología.

IDENTIDAD

- Nombre: Asistente TechStore
- Tono: cálido, humano, profesional — como si fuera una persona real de atención al cliente
- Idioma: respondé SIEMPRE en el mismo idioma que el cliente. Si escribe en inglés, respondé en inglés. Si escribe en español, usá español argentino (voseás: vos, tenés, podés)
- NO sos un chatbot genérico — representás a TechStore con orgullo
- Usá el nombre del cliente si está disponible — hace que la conversación se sienta más personal

TU TAREA

Analizá el mensaje del cliente, clasificá su intención, y generá una respuesta genuinamente útil y humana.

Respondé ÚNICAMENTE con un JSON válido. Sin texto antes ni después del JSON. Sin markdown. Sin ````json````.

FORMATO DE RESPUESTA (JSON estricto)

```
{  
  "intent": "FAQ | ESTADO_PEDIDO | RECLAMO | GENERAL",  
  "order_id": null,  
  "urgente": false,  
  "respuesta": "texto cálido, claro y personalizado para el cliente"  
}
```

El prompt de la configuración vigente se encuentra en el nodo IA - Motor Decision del archivo workflows/Flujo 2 — Chatbot Omnicanal IA.json, en el commit f68da9d del repositorio, que coincide byte a byte con la versión del workflow que corrió el 12 de

agosto según el historial del motor de flujos; y en el mismo nodo del archivo workflows/Flujo 2 — Chatbot WhatsApp + Telegram.json en su versión actual. Una cita anterior a otro commit se corrige en el Anexo L. El prompt reducido de E7 corresponde al mismo nodo de ese último archivo antes del diseño factorial, y se conserva en experiments/E8/resultados/workflow_publicado_antes_de_E8.json. Todos son verificables sin ejecutar el sistema.

Anexo I: Baseline de atención manual — tiempos cronometrados

Se transcriben los tiempos crudos de las doce órdenes procesadas manualmente según el protocolo de la Sección 3.5.5, incluidas las dos descartadas por interrupción del operador, que se consignan con su marca de descarte y no se eliminan del registro. Los valores están en segundos y provienen sin edición del archivo exportado por el cronómetro. El resultado primario de la Sección 5.1.4 se calcula sobre las diez órdenes válidas.

Tabla I.1: Tiempos cronometrados del procesamiento manual, por orden y por fase.

#	Orden	Rama	T1	T2	T3	Total	Estado
1	ORD-E4-001	—	—	—	—	—	Descartada
2	ORD-E4-002	sin stock	8,033	—	—	—	Descartada
3	ORD-E4-003	con stock	7,530	8,598	53,674	69,801	Válida
4	ORD-E4-004	con stock	11,386	9,089	34,453	54,928	Válida
5	ORD-E4-005	sin stock	6,708	5,598	30,449	42,754	Válida
6	ORD-E4-006	con stock	8,236	4,698	36,577	49,510	Válida
7	ORD-E4-007	con stock	5,506	5,768	38,081	49,354	Válida
8	ORD-E4-008	con stock	7,958	4,857	31,787	44,602	Válida
9	ORD-E4-009	sin stock	7,695	4,448	34,960	47,104	Válida
10	ORD-E4-010	con stock	5,306	4,447	35,884	45,638	Válida
11	ORD-E4-011	con stock	5,806	5,087	32,009	42,902	Válida
12	ORD-E4-012	sin stock	4,365	5,448	34,863	44,676	Válida

El texto completo de las notificaciones redactadas por el operador durante la medición, así como el archivo original exportado por el cronómetro y el script de análisis, se encuentran versionados en el repositorio del trabajo bajo el directorio del experimento.

Se transcriben además los registros que el operador escribió en la base durante la medición. Las doce órdenes comparten la misma marca de recepción porque fueron creadas por una única sentencia de preparación; en consecuencia la columna de tiempo acumulado mide permanencia en la cola y no latencia individual, mientras que la columna de incremento reproduce el tiempo marginal por otra vía instrumental (Sección 5.1.4).

Tabla I.2: Registros en la base de datos de las órdenes procesadas manualmente.

Orden	Estado	processed_at	notified_at	Acumulado	Incremento
ORD-E4-001	pending	—	—	—	— (descartada)
ORD-E4-002	pending	—	—	—	— (descartada)
ORD-E4-003	confirmed	16:47:12	16:48:06	72,38 s	—
ORD-E4-004	confirmed	16:48:29	16:49:04	130,46 s	58,08 s
ORD-E4-005	no_stock	16:49:33	16:50:04	190,13 s	59,67 s
ORD-E4-006	confirmed	16:50:19	16:50:55	241,80 s	51,67 s
ORD-E4-007	confirmed	16:51:12	16:51:50	297,04 s	55,24 s
ORD-E4-008	confirmed	16:52:06	16:52:38	344,58 s	47,54 s
ORD-E4-009	no_stock	16:52:53	16:53:28	394,29 s	49,71 s
ORD-E4-010	confirmed	16:53:39	16:54:15	441,75 s	47,46 s
ORD-E4-	confirmed	16:54:28	16:55:01	487,30 s	45,55 s

011					
ORD-E4-012	no_stock	16:55:12	16:55:47	533,94 s	46,64 s
Media	—	—	—	313,37 s	51,28 s
Desvío	—	—	—	154,57 s	5,22 s

Anexo J: Corpus de evaluación y procedimiento de cálculo estadístico

El corpus de 150 mensajes utilizado para evaluar la clasificación, sus etiquetas de referencia, las predicciones del modelo y las etiquetas del evaluador independiente se encuentran versionados en el repositorio del trabajo en formato separado por comas, junto con los tiempos de anotación registrados por el instrumento de etiquetado. El corpus fue incorporado al repositorio antes de producirse el etiquetado, de modo que el sello temporal de esa incorporación acredita el carácter ciego del procedimiento descrito en la Sección 3.5.3. El repositorio está disponible en <https://github.com/IgnacioOdorico/Automatizacion-de-Atencion-al-Cliente-en-E-commerce-e-Inteligencia-Artificial>. La etiqueta entrega-2026-09-r6 identifica la versión de los datos, los guiones y los workflows que corresponde a este documento, y cada experimento se encuentra en su propio directorio bajo experiments (E1 a E8 y PF).

Se detalla a continuación el procedimiento de cálculo del intervalo de confianza de Wilson reportado para el accuracy global, a fin de que el valor publicado sea verificable.

Datos: $x = 139$ aciertos, $n = 150$ mensajes, $\hat{p} = x/n = 0,926667$

Nivel: 95 % $\rightarrow z = 1,959964$

$$\text{centro} = \frac{\hat{p} + z^2/(2n)}{1 + z^2/n}; \quad \text{semiancho} = \frac{z}{1 + z^2/n} \cdot \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}$$

$$z^2 = 3,841459 \quad z^2/n = 0,025610 \quad z^2/(2n) = 0,012805$$

$$\text{centro} = (0,926667 + 0,012805) / 1,025610 = 0,916019$$

$$\text{semiancho} = (1,959964 / 1,025610) \cdot \sqrt{(0,000453 + 0,0000427)} = 0,042548$$

$$\text{IC 95 \% de Wilson} = [0,873471 ; 0,958567] = [87,3 \% ; 95,9 \%]$$

Comprobación con otros métodos, a título informativo:

Wald (normal, sin corrección) [88,5 % ; 96,8 %]

El límite inferior supera el umbral del 85 % en cualquiera de los métodos, por lo que la conclusión sobre H2b no depende de esta elección.

El coeficiente κ de Cohen reportado en la Sección 5.2.3 se calculó sobre la submuestra de 50 mensajes etiquetada por el evaluador independiente, con un acuerdo observado $P_o = 0,940$ (47 coincidencias sobre 50) y un acuerdo esperado por azar $P_e = 0,259$ derivado de las distribuciones marginales de ambos anotadores, de donde $\kappa = (P_o - P_e) / (1 - P_e) = 0,919$. La interpretación se realiza según la escala de Landis y Koch (1977).

Anexo K: Verificación de datos concretos en las respuestas de tipo FAQ

Se transcriben todos los datos concretos que el guion de verificación extrajo de las 45 respuestas de tipo FAQ, según la regla de la Sección 3.5.7, con el resultado de su contraste contra la base de conocimiento del Anexo D. Las 27 respuestas que no figuran en la tabla no contienen ningún dato concreto. La columna de la derecha indica la pregunta de la base en la que figura el dato; cuando figura en más de una, se consignan todas.

Tabla K.1: Datos concretos afirmados en las respuestas de tipo FAQ y su presencia en la base de conocimiento.

Interacción	Consulta del cliente	Dato afirmado	Resultado	Entrada de la base que lo contiene
238	cuanto tarda el envío? porque el mio ya lleva 8 días	3 y 7 días hábiles	Respaldado	¿Cuánto tarda el envío?
238	cuanto tarda el envío? porque el mio ya lleva 8 días	8 días	Repite una cifra del cliente	—
239	puedo pagar en cuotas?	12 cuotas	Respaldado	¿Puedo pagar en cuotas?
243	cual es la politica de cambios?	30 días	Respaldado	¿Cómo puedo devolver un producto? / ¿Cuál es la política de cambios?
246	tienen garantia los productos?	12 meses	Respaldado	¿Tienen garantía los productos?
262	cuanto tarda el envío normalmente?	1 y 3 días hábiles	Respaldado	¿Cuánto tarda el envío?

Interacción	Consulta del cliente	Dato afirmado	Resultado	Entrada de la base que lo contiene
262	cuanto tarda el envío normalmente?	3 y 7 días hábiles	Respaldado	¿Cuánto tarda el envío?
265	si no estoy en casa cuando llega el pedido que pasa?	5 días hábiles	Respaldado	¿Qué pasa si no estoy en casa cuando llega el pedido?
277	tienen local físico para retirar?	100%	Respaldado	¿Los productos son originales? / ¿Tienen stock en físico para retirar?
280	cuantos días tengo para devolver algo?	30 días	Respaldado	¿Cómo puedo devolver un producto? / ¿Cuál es la política de cambios?
291	hasta cuantas cuotas sin interes hay?	12 cuotas	Respaldado	¿Puedo pagar en cuotas?
316	puedo retirar por sucursal?	100%	Respaldado	¿Los productos son originales? / ¿Tienen stock en físico para retirar?
319	si el producto no me gusta lo puedo devolver?	30 días	Respaldado	¿Cómo puedo devolver un producto? / ¿Cuál es la política de cambios?
326	cual es el horario de atencion?	9 a 18 hs	No figura en la base	—
337	cuanto demora el reintegro de una devolucion?	5 y 10 días hábiles	No figura en la base	—
349	venden productos usados o todos son nuevos?	100%	Respaldado	¿Los productos son originales? / ¿Tienen stock en físico para retirar?
352	si pago con transferencia cuando despachan?	24 horas hábiles	Respaldado	¿Cómo inicio una devolución?

Interacción	Consulta del cliente	Dato afirmado	Resultado	Entrada de la base que lo contiene
357	buenas, se puede cancelar un pedido ya pagado?	2 horas	Respaldado	¿Puedo cancelar un pedido?
357	buenas, se puede cancelar un pedido ya pagado?	2 horas	Respaldado	¿Puedo cancelar un pedido?
365	buenas, cuando estiman que llega el pedido a mendoza?	1 y 3 días hábiles	Respaldado	¿Cuánto tarda el envío?
370	cuanto tiempo de garantía tienen las notebooks?	12 meses	Respaldado	¿Tienen garantía los productos?

Anexo L: Registro de desvíos y correcciones

Este anexo reúne la historia de las revisiones del trabajo: los criterios, diseños y afirmaciones que cambiaron entre versiones, con el motivo de cada cambio. El cuerpo del documento presenta el diseño final y remite aquí. Las versiones anteriores del documento y de los guiones se conservan en el historial del repositorio.

Formulación de las hipótesis y de los objetivos. En su formulación original, registrada en la versión del trabajo del 27 de abril de 2026 (commit 9dc7b44), H1 fijaba un tiempo end-to-end inferior a 30 segundos frente a un rango de tiempos manuales que no había sido medido, y no operacionalizaba la reducción respecto del proceso manual. Cuando se midió el baseline manual, H1 pasó a compararse contra ese baseline, y el criterio de un orden de magnitud se fijó en la revisión del trabajo, con el resultado ya conocido; por eso H1 se presenta como objetivo de estimación. La misma versión formulaba una única hipótesis sobre el chatbot, que restringía el tiempo de respuesta a las consultas de tipo FAQ, estado de pedido y generales; al separarla en H2a y H2b, el tiempo de respuesta se extendió a las cuatro categorías, porque el contraste siempre se ejecutó sobre los 150 mensajes del corpus, reclamos incluidos, y la extensión vuelve la hipótesis más exigente. Aquella formulación fijaba además el umbral de exactitud del 85 % sin precisar si se contrastaba el valor puntual o el intervalo: la regla del límite inferior se explicitó con la corrida del corpus ya ejecutada, y el diseño factorial la fijó por escrito antes de medir. La comparación con el proceso manual no figuraba entre los objetivos específicos iniciales y se incorporó a OE1 cuando se midió el baseline.

Ablación E7, reemplazada por el diseño factorial. Una primera ablación (E7) comparó la configuración principal con un prompt reducido, atribuyó la diferencia, de 6,7 puntos de exactitud, a la base de conocimiento, y el trabajo concluyó en consecuencia que la base era la pieza decisiva para clasificar. El diseño no permitía esa atribución, porque aquel prompt no quitaba solo la base sino cinco de los ocho bloques del prompt: las reglas de clasificación, la base de conocimiento, las reglas críticas, los ejemplos y el bloque que presenta el mensaje del cliente con su nombre y su canal. Los factores quedaban confundidos. El diseño factorial E8 (Sección 3.5.6) los separó y mostró lo contrario: la exactitud depende de las reglas y los ejemplos, y la base no aporta nada cuando ellos están presentes. Los 6,7 puntos que E7 atribuyó a la base corresponden en realidad, sobre todo, a los bloques que aquel prompt quitaba junto con ella. La conclusión se corrigió en las Secciones 5.2.4 y 6.3, y E7 se conserva en experiments/E7.

Trazabilidad del artefacto medido. Al documentar el prompt se auditó el workflow vigente en ese momento, de dieciocho nodos y con el prompt reducido, y se concluyó que el clasificador operaba en régimen zero-shot, cuando el corpus había corrido el 12 de agosto sobre el workflow de catorce nodos, con base de conocimiento, reglas y ejemplos. La auditoría fue correcta en su método y se aplicó al artefacto equivocado. La reconstrucción, consultada sobre el historial de versiones y de publicaciones del motor de flujos, establece la secuencia: la versión del workflow de catorce nodos con que corrió el corpus se publicó cuatro minutos antes de la corrida, y su prompt coincide byte a byte con el del commit f68da9d; el 25 de agosto se incorporó el workflow de dieciocho nodos, que suma el canal de Telegram, con un prompt reducido de 1032 caracteres, con el que corrieron las 45 interacciones de Telegram y la ablación E7; y el 15 de septiembre el diseño factorial restituyó en ese workflow el prompt de la configuración principal. La reconstrucción se versiona en experiments/E8/resultados/trazabilidad_versiones.txt. El commit f297c9e, que en un primer momento se citó como fuente del prompt de la configuración vigente, guarda el mismo texto sin el signo «=» inicial, con el que la base de conocimiento no se habría inyectado.

Primera corrida del corpus. El corpus se ejecutó dos veces el 12 de agosto de 2026. La primera corrida, de las 11:09, recibió 150 respuestas HTTP 200 y escribió 115 interacciones: las 35 que el modelo enrutó a ESTADO_PEDIDO se perdieron sin aviso, porque el nodo que busca el pedido no emitía ningún ítem cuando el número citado no existía y el subgrafo siguiente no se ejecutaba. El registro de trabajo del equipo identifica el defecto como D-7. La consulta se reescribió y la corrida definitiva, de las 20:45, se ejecutó cuatro minutos después de publicar esa versión del flujo y escribió las 150 filas. Versiones anteriores de este documento informaban la segunda corrida sin mencionar la primera; la Sección 5.2.3 la declara, con su exactitud de 140 sobre 150 —mayor que la de la corrida definitiva— y la coincidencia de 145 de 150 etiquetas entre ambas, y las Secciones 5.5 (f) y (h), 6.3 y 6.4 incorporan el defecto y la variación de tiempo de respuesta que la comparación permite medir. La comparación se versiona en experiments/E2/resultados/corrida1_vs_corrida2.txt.

Construcción de las consultas. Los nodos de base de datos de ambos flujos armaban sus sentencias insertando los valores recibidos dentro del texto de la consulta. El registro de trabajo del equipo lo consigna como defecto D-9, de severidad alta, y no se lo corrigió durante las mediciones para no mover dos variables a la vez. Las diez consultas de los cuatro flujos se parametrizaron el 17 de septiembre de 2026, con las mediciones ya cerradas: el artefacto publicado difiere del medido en ese punto y solo en ese, porque la parametrización no cambia lo que cada consulta lee ni escribe. Las pruebas PF-06 y PC-06 dejan constancia de las dos situaciones. En PF-06, sobre el Flujo 1, la sentencia anterior con un apéndice que lleva apóstrofo la rechaza PostgreSQL por un error de sintaxis, y con la sentencia parametrizada el mismo valor llega como dato; en PC-06, sobre el Flujo 2, un reclamo en inglés con dos contracciones se clasifica, genera su ticket con el texto íntegro y recibe respuesta. Una comprobación posterior recorrió el camino completo del Flujo 1 con las consultas parametrizadas: la orden se registró, se confirmó, descontó el stock y escribió las tres marcas temporales, con tiempos del mismo orden que los del Capítulo 5 (`experiments/PF/verificar_camino_completo.py`). Su orden de prueba se borró al terminar y el stock se repuso, de modo que no integra los totales del Capítulo 5. El mismo cambio cerró lo que quedaba del defecto D-8 —la variante de producción del Flujo 2 conservaba la consulta anterior a la corrección de D-7, de modo que quien la importara reproducía el sistema con la pérdida silenciosa— y corrigió el nombre de un nodo de la variante de producción del Flujo 1 que una exportación había dejado mal codificado. El guion, la prueba y su evidencia están en `experiments/PF`.

Corridas descartadas y defectos del instrumental. Dos corridas de las pruebas del Flujo 2 se descartaron antes de la que se informa. La del 17 de septiembre a las 03:41 UTC usó un identificador de usuario sin dominio de correo: las cinco ejecuciones terminaron en error en el nodo de envío, el emisor recibió HTTP 200 y no quedó ninguna interacción registrada, aunque sí los dos tickets de las pruebas de reclamo, que quedaron sin respuesta. Las dos corridas de la prueba del apéndice del mismo día informaban mal el resultado de su control, porque el guion leía un código de salida que no distingue una sentencia fallida, y dejaron dos interacciones y dos tickets. Los tres casos están declarados en el README de `experiments/PF` con los identificadores de las filas que dejaron. El guion de ejecución del corpus tiene además dos defectos registrados como D-11: su conciliación no filtra por fecha, de modo que con dos corridas en la base informa un total que suma ambas, y no genera el manifiesto de la corrida en la consola en que se ejecutó; la conciliación de la corrida informada se rehízo con consultas propias, y los manifiestos del diseño factorial, que sí se generaron, están en `experiments/E8`.

Pruebas funcionales. Versiones anteriores informaban como aprobadas, sin evidencia versionada, las diez pruebas que el trabajo tenía entonces. Las pruebas de rechazo del Flujo 1 se re-ejecutaron con evidencia el 15 de septiembre: PF-04 y PF-05 no producen el rechazo controlado que fijaba la Tabla 3.4, y se reclasificaron como no aprobada y parcial. Las pruebas del Flujo 2 se ejecutaron con evidencia el 17 de septiembre: PC-05 se clasificó como FAQ y no como GENERAL, y se informa como no

aprobada. La Tabla 3.5 citaba para PC-02 un número de pedido que no existe en la carga inicial, y la prueba se ejecutó sobre uno que sí existe. Las evidencias y los guiones están en experiments/PF.

Descripción del prompt y origen del corpus. Versiones anteriores describían la regla crítica 3 del prompt como una instrucción de tono, cuando ordena ocultar que el asistente es una IA (Sección 2.5), y no advertían que el ejemplo de envío a Córdoba contradice la base de conocimiento (Sección 5.2.5). Afirmaban también que el equipo había escrito los 150 mensajes del corpus; la documentación del experimento en el repositorio (experiments/E2/README.md) registra que se redactaron con asistencia de un modelo de lenguaje a partir de mensajes del equipo, y el documento se corrigió en las Secciones 3.5.3, 3.6 y 6.4 y en la Declaración de originalidad.

La verificación no controla el contexto en que se usa un dato. El plazo de 24 horas hábiles de la interacción 352, por ejemplo, figura en la base referido al retiro de un producto en devolución, y la respuesta lo aplica a la confirmación de un pago por transferencia; la tabla lo consigna como respaldado porque la regla solo comprueba la presencia del dato. El guion, sus pruebas y el archivo con este detalle se encuentran en el repositorio bajo experiments/E6.